

Листинг программного кода модуля app

Всего файлов: 121

Листинг 1 – Код файла app/build.gradle.kts

```
import java.util.Properties

plugins {
    alias(libs.plugins.android.application)
    alias(libs.plugins.kotlin.android)
    alias(libs.plugins.kotlin.compose)
    alias(libs.plugins.hilt.android)
    id("org.jetbrains.kotlin.kapt")
}

private fun String.asBuildConfigString(): String {
    return "\"\${replace("\\", "\\\").replace("'", "\\'")}\\""
}

val localProperties = Properties().apply {
    val localPropertiesFile = rootProject.file("local.properties")
    if (localPropertiesFile.exists()) {
        localPropertiesFile.inputStream().use(::load)
    }
}

val apiNinjasApiKey = (
    localProperties.getProperty("apiNinjasApiKey")
    ?: providers.gradleProperty("apiNinjasApiKey").orNull
).orEmpty()

android {
    namespace = "com.example.poster"
    compileSdk = 36

    defaultConfig {
        applicationId = "com.example.poster"
        minSdk = 31
        targetSdk = 35
        versionCode = 1
        versionName = "1.0"
        buildConfigField(
            "String",
            "API_NINJAS_RANDOM_IMAGE_URL",
            "https://api.api-ninjas.com/v1/randomimage".asBuildConfigString()
        )
        buildConfigField(
            "String",
            "API_NINJAS_API_KEY",
            apiNinjasApiKey.asBuildConfigString()
        )

        testInstrumentationRunner = "androidx.test.runner.AndroidJUnitRunner"
    }
}
```

```

buildTypes {
    release {
        isMinifyEnabled = false
        proguardFiles(
            getDefaultProguardFile("proguard-android-optimize.txt"),
            "proguard-rules.pro"
        )
    }
}

compileOptions {
    sourceCompatibility = JavaVersion.VERSION_11
    targetCompatibility = JavaVersion.VERSION_11
}

kotlinOptions {
    jvmTarget = "11"
}

buildFeatures {
    compose = true
    buildConfig = true
}

}

kapt {
    correctErrorTypes = true
}

dependencies {

    implementation(libs.androidx.core.ktx)
    implementation(libs.androidx.lifecycle.runtime.ktx)
    implementation(libs.androidx.activity.compose)
    implementation(platform(libs.androidx.compose.bom))
    implementation(libs.androidx.ui)
    implementation("androidx.compose.animation:animation")
    implementation(libs.androidx.ui.graphics)
    implementation(libs.androidx.ui.tooling.preview)
    implementation(libs.androidx.material3)
    implementation(libs.androidx.material.icons.extended)
    implementation(libs.hilt.android)
    kapt(libs.hilt.compiler)
    implementation("io.coil-kt:coil-compose:2.7.0")
    testImplementation(libs.junit)
    androidTestImplementation(libs.androidx.junit)
    androidTestImplementation(libs.androidx.espresso.core)
    androidTestImplementation(platform(libs.androidx.compose.bom))
    androidTestImplementation(libs.androidx.ui.test.junit4)
    debugImplementation(libs.androidx.ui.tooling)
    debugImplementation(libs.androidx.ui.test.manifest)
}

```

Листинг 2 – Код файла app/proguard-rules.pro

```
# Add project specific ProGuard rules here.
# You can control the set of applied configuration files using the
# proguardFiles setting in build.gradle.
#
# For more details, see
# http://developer.android.com/guide/developing/tools/proguard.html

# If your project uses WebView with JS, uncomment the following
# and specify the fully qualified class name to the JavaScript interface
# class:
#-keepclassmembers class fqcn.of.javascript.interface.for.webview {
#   public *;
#}

# Uncomment this to preserve the line number information for
# debugging stack traces.
#-keepattributes SourceFile,LineNumberTable

# If you keep the line number information, uncomment this to
# hide the original source file name.
#-renamesourcefileattribute SourceFile
```

Листинг 3 – Код файла `app/src/androidTest/java/com/example/poster/ExampleInstrumentedTest.kt`

```
package com.example.poster

import androidx.test.platform.app.InstrumentationRegistry
import androidx.test.ext.junit.runners.AndroidJUnit4

import org.junit.Test
import org.junit.runner.RunWith

import org.junit.Assert.*

/**
 * Instrumented test, which will execute on an Android device.
 *
 * See [testing documentation](http://d.android.com/tools/testing).
 */
@RunWith(AndroidJUnit4::class)
class ExampleInstrumentedTest {
    @Test
    fun useAppContext() {
        // Context of the app under test.
        val appContext = InstrumentationRegistry.getInstrumentation().targetContext
        assertEquals("com.example.poster", appContext.packageName)
    }
}
```

Листинг 4 – Код файла app/src/main/AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <uses-permission android:name="android.permission.INTERNET" />

    <application
        android:name=".PosterApplication"
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:usesCleartextTraffic="true"
        android:supportsRtl="true"
        android:theme="@style/Theme.Posters"
        tools:targetApi="31">
        <activity
            android:name=".MainActivity"
            android:exported="true"
            android:label="@string/app_name"
            android:theme="@style/Theme.Posters">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>

</manifest>
```

Листинг 5 – Код файла `app/src/main/java/com/example/poster/core/common/AppResult.kt`

```
package com.example.poster.core.common

sealed interface AppResult<out T> {
    data class Success<T>(val data: T) : AppResult<T>

    data class Error(
        val message: String,
        val cause: Throwable? = null,
    ) : AppResult<Nothing>
}
```

Листинг 6 – Код файла `app/src/main/java/com/example/poster/core/media/RemoteImageConfig.kt`

```
package com.example.poster.core.media

import android.content.Context
import coil.request.ImageRequest
import com.example.poster.BuildConfig

object RemoteImageConfig {
    private const val API_NINJAS_KEY_HEADER = "X-API-Key"
    private const val API_NINJAS_ACCEPT_HEADER = "Accept"
    private const val API_NINJAS_ACCEPT_VALUE = "image/jpg"

    val randomImageUrl: String = BuildConfig.API_NINJAS_RANDOM_IMAGE_URL

    fun buildImageModel(
        context: Context,
        remoteUrl: String,
    ): Any? {
        if (requiresApiNinjasHeaders(remoteUrl) && BuildConfig.API_NINJAS_API_KEY.isBlank())
        {
            return null
        }

        return ImageRequest.Builder(context)
            .data(remoteUrl)
            .crossfade(true)
            .apply {
                if (requiresApiNinjasHeaders(remoteUrl)) {
                    setHeader(API_NINJAS_KEY_HEADER, BuildConfig.API_NINJAS_API_KEY)
                    setHeader(API_NINJAS_ACCEPT_HEADER, API_NINJAS_ACCEPT_VALUE)
                }
            }
            .build()
    }

    private fun requiresApiNinjasHeaders(remoteUrl: String): Boolean {
        return remoteUrl.startsWith(randomImageUrl)
    }
}
```

Листинг 7 – Код файла `app/src/main/java/com/example/poster/core/network/NetworkConfig.kt`

```
package com.example.poster.core.network

object NetworkConfig {
    const val BASE_URL = "http://10.0.2.2:8080/"
    const val CONNECT_TIMEOUT_SECONDS = 15L
    const val READ_TIMEOUT_SECONDS = 15L
}
```

Листинг 8 – Код файла `app/src/main/java/com/example/poster/core/security/AuthInterceptor.kt`

```
package com.example.poster.core.security

interface AuthInterceptor {
    suspend fun authorizationHeader(): String?
}

class BearerAuthInterceptor(
    private val tokenStorage: TokenStorage,
) : AuthInterceptor {
    override suspend fun authorizationHeader(): String? {
        val accessToken = tokenStorage.getAccessToken() ?: return null
        return "Bearer $accessToken"
    }
}
```

Листинг 9 – Код файла `app/src/main/java/com/example/poster/core/security/MailAccessTokenStorage.kt`

```
package com.example.poster.core.security

import com.example.poster.core.storage.StorageKeys

interface MailAccessTokenStorage {
    suspend fun hasToken(): Boolean
    suspend fun saveToken(token: String)
    suspend fun deleteToken()
}

class SecureMailAccessTokenStorage(
    private val secureStorage: SecureStorage,
) : MailAccessTokenStorage {
    override suspend fun hasToken(): Boolean {
        return secureStorage.getString(StorageKeys.MAIL_ACCESS_TOKEN) != null
    }

    override suspend fun saveToken(token: String) {
        secureStorage.putString(StorageKeys.MAIL_ACCESS_TOKEN, token)
    }

    override suspend fun deleteToken() {
        secureStorage.remove(StorageKeys.MAIL_ACCESS_TOKEN)
    }
}
```

Листинг 10 – Код файла `app/src/main/java/com/example/poster/core/security/SecureStorage.kt`

```
package com.example.poster.core.security

interface SecureStorage {
    suspend fun getString(key: String): String?
    suspend fun putString(key: String, value: String)
    suspend fun remove(key: String)
    suspend fun clear()
}

class InMemorySecureStorage : SecureStorage {
    private val values = mutableMapOf<String, String>()

    override suspend fun getString(key: String): String? {
        return values[key]
    }

    override suspend fun putString(key: String, value: String) {
        values[key] = value
    }

    override suspend fun remove(key: String) {
        values.remove(key)
    }

    override suspend fun clear() {
        values.clear()
    }
}
```

Листинг 11 – Код файла app/src/main/java/com/example/poster/core/security/TokenStorage.kt

```
package com.example.poster.core.security

import com.example.poster.core.storage.StorageKeys

interface TokenStorage {
    suspend fun getAccessToken(): String?
    suspend fun getRefreshToken(): String?
    suspend fun saveTokens(accessToken: String, refreshToken: String)
    suspend fun clearTokens()
}

class SecureTokenStorage(
    private val secureStorage: SecureStorage,
) : TokenStorage {
    override suspend fun getAccessToken(): String? {
        return secureStorage.getString(StorageKeys.ACCESS_TOKEN)
    }

    override suspend fun getRefreshToken(): String? {
        return secureStorage.getString(StorageKeys.REFRESH_TOKEN)
    }

    override suspend fun saveTokens(accessToken: String, refreshToken: String) {
        secureStorage.putString(StorageKeys.ACCESS_TOKEN, accessToken)
        secureStorage.putString(StorageKeys.REFRESH_TOKEN, refreshToken)
    }

    override suspend fun clearTokens() {
        secureStorage.remove(StorageKeys.ACCESS_TOKEN)
        secureStorage.remove(StorageKeys.REFRESH_TOKEN)
    }
}
```

Листинг 12 – Код файла `app/src/main/java/com/example/poster/core/storage/StorageKeys.kt`

```
package com.example.poster.core.storage

object StorageKeys {
    const val AUTH_SESSION = "auth_session"
    const val ACCESS_TOKEN = "access_token"
    const val REFRESH_TOKEN = "refresh_token"
    const val USER_ID = "user_id"
    const val MAIL_ACCESS_TOKEN = "mail_access_token"
}
```

Листинг 13 – Код файла `app/src/main/java/com/example/poster/data/local/PosterLocalDataSource.kt`

```
package com.example.poster.data.local

data class LocalAuthSessionEntity(
    val accessToken: String,
    val refreshToken: String,
    val userId: String,
)

interface PosterLocalDataSource {
    suspend fun getAuthSession(): LocalAuthSessionEntity?
    suspend fun saveAuthSession(session: LocalAuthSessionEntity)
    suspend fun clearAuthSession()
}

class InMemoryPosterLocalDataSource : PosterLocalDataSource {
    private var session: LocalAuthSessionEntity? = null

    override suspend fun getAuthSession(): LocalAuthSessionEntity? {
        return session
    }

    override suspend fun saveAuthSession(session: LocalAuthSessionEntity) {
        this.session = session
    }

    override suspend fun clearAuthSession() {
        session = null
    }
}
```

Листинг 14 – Код файла `app/src/main/java/com/example/poster/data/mapper/AuthMapper.kt`

```
package com.example.poster.data.mapper

import com.example.poster.data.remote.dto.auth.AuthResponseDto
import com.example.poster.domain.model.AuthSession

fun AuthResponseDto.toDomainSession(): AuthSession {
    return AuthSession(
        accessToken = accessToken,
        refreshToken = refreshToken,
        userId = profile.id,
    )
}
```

**Листинг 15 – Код файла `app/src/main/java/com/example/poster/data/mapper/ChatMapper.k`
`t`**

```
package com.example.poster.data.mapper

import com.example.poster.data.remote.dto.chat.ChatDto
import com.example.poster.domain.model.Chat

fun ChatDto.toDomain(): Chat {
    return Chat(
        id = id,
        title = title,
        initials = initials,
        lastMessage = lastMessage.orEmpty(),
        lastMessageTime = lastMessageTime.orEmpty(),
        unreadCount = unreadCount,
    )
}
```

Листинг 16 – Код файла `app/src/main/java/com/example/poster/data/mapper/DomainMappers.kt`

```
package com.example.poster.data.mapper

import com.example.poster.data.local.LocalAuthSessionEntity
import com.example.poster.data.remote.RemoteAttachmentDto
import com.example.poster.data.remote.RemoteAuthSessionDto
import com.example.poster.domain.model.Attachment
import com.example.poster.domain.model.AttachmentType
import com.example.poster.domain.model.AttachmentUploadStatus
import com.example.poster.domain.model.AuthSession

fun RemoteAttachmentDto.toDomain(): Attachment {
    return Attachment(
        id = id,
        localUri = localUri,
        remoteUrl = remoteUrl,
        type = type.toAttachmentType(),
        fileName = fileName,
        mimeType = mimeType,
        sizeBytes = sizeBytes,
        uploadStatus = uploadStatus.toAttachmentUploadStatus(),
    )
}

fun RemoteAuthSessionDto.toDomain(): AuthSession {
    return AuthSession(
        accessToken = accessToken,
        refreshToken = refreshToken,
        userId = userId,
    )
}

fun Attachment.toRemote(): RemoteAttachmentDto {
    return RemoteAttachmentDto(
        id = id,
        type = type.name,
        localUri = localUri,
        remoteUrl = remoteUrl,
        fileName = fileName,
        mimeType = mimeType,
        sizeBytes = sizeBytes,
        uploadStatus = uploadStatus.name,
    )
}

private fun String.toAttachmentType(): AttachmentType {
    return when (uppercase()) {
        "IMAGE" -> AttachmentType.IMAGE
        "VIDEO" -> AttachmentType.VIDEO
    }
}
```

```

        "AUDIO" -> AttachmentType.AUDIO
        "DOCUMENT", "FILE" -> AttachmentType.DOCUMENT
        else -> AttachmentType.UNKNOWN
    }
}

private fun String.toAttachmentUploadStatus(): AttachmentUploadStatus {
    return when (uppercase()) {
        "LOCAL_ONLY" -> AttachmentUploadStatus.LOCAL_ONLY
        "UPLOADING" -> AttachmentUploadStatus.UPLOADING
        "UPLOADED" -> AttachmentUploadStatus.UPLOADED
        "FAILED" -> AttachmentUploadStatus.FAILED
        else -> AttachmentUploadStatus.UPLOADED
    }
}

fun AuthSession.toLocal(): LocalAuthSessionEntity {
    return LocalAuthSessionEntity(
        accessToken = accessToken,
        refreshToken = refreshToken,
        userId = userId,
    )
}

fun LocalAuthSessionEntity.toDomain(): AuthSession {
    return AuthSession(
        accessToken = accessToken,
        refreshToken = refreshToken,
        userId = userId,
    )
}

```

Листинг 17 – Код файла `app/src/main/java/com/example/poster/data/mapper/MessageMapper.kt`

```
package com.example.poster.data.mapper

import com.example.poster.data.remote.dto.message.MessageDto
import com.example.poster.data.remote.dto.message.AttachmentDto
import com.example.poster.domain.model.Attachment
import com.example.poster.domain.model.AttachmentType
import com.example.poster.domain.model.AttachmentUploadStatus
import com.example.poster.domain.model.Message
import com.example.poster.domain.model.MessageStatus

fun MessageDto.toDomain(): Message {
    return Message(
        id = id,
        chatId = chatId,
        text = bodyText.orEmpty(),
        time = createdAt,
        isMine = isMine,
        status = status.toMessageStatus(),
        attachments = attachments.map { it.toDomain() },
    )
}

fun AttachmentDto.toDomain(): Attachment {
    return Attachment(
        id = id,
        localUri = null,
        remoteUrl = url,
        fileName = fileName,
        mimeType = mimeType,
        sizeBytes = sizeBytes,
        type = type.toAttachmentType(mimeType),
        uploadStatus = AttachmentUploadStatus.UPLOADED,
    )
}

private fun String.toMessageStatus(): MessageStatus {
    return when (uppercase()) {
        "SENDING" -> MessageStatus.SENDING
        "SENT" -> MessageStatus.SENT
        "DELIVERED" -> MessageStatus.DELIVERED
        "READ" -> MessageStatus.READ
        "FAILED" -> MessageStatus.FAILED
        else -> MessageStatus.SENT
    }
}

private fun String.toAttachmentType(mimeType: String): AttachmentType {
    return when {
```

```
    equals("IMAGE", ignoreCase = true) || mimeType.startsWith("image/") -> AttachmentType.IMAGE
    equals("VIDEO", ignoreCase = true) || mimeType.startsWith("video/") -> AttachmentType.VIDEO
    equals("AUDIO", ignoreCase = true) || mimeType.startsWith("audio/") -> AttachmentType.AUDIO
    equals("DOCUMENT", ignoreCase = true) || equals("FILE", ignoreCase = true) -> AttachmentType.DOCUMENT
    mimeType.contains("pdf") || mimeType.contains("text") -> AttachmentType.DOCUMENT
    else -> AttachmentType.UNKNOWN
}
}
```

Листинг 18 – Код файла `app/src/main/java/com/example/poster/data/mapper/ProfileMapper.kt`

```
package com.example.poster.data.mapper

import com.example.poster.data.remote.dto.profile.ProfileDto
import com.example.poster.domain.model.Profile

fun ProfileDto.toDomain(): Profile {
    return Profile(
        id = id,
        name = name,
        username = username,
        email = email,
        bio = bio.orEmpty(),
        avatarUrl = avatarUrl,
        isOnline = isOnline,
    )
}
```

Листинг 19 – Код файла `app/src/main/java/com/example/poster/data/mapper/SettingsMapper.kt`

```
package com.example.poster.data.mapper

import com.example.poster.data.remote.dto.profile.ProfileDto
import com.example.poster.data.remote.dto.settings.SettingsDto
import com.example.poster.domain.model.UserSettings

fun SettingsDto.toDomain(profile: ProfileDto): UserSettings {
    return UserSettings(
        name = profile.name,
        username = profile.username,
        email = profile.email,
        birthday = profile.birthday.orEmpty(),
        bio = profile.bio.orEmpty(),
        language = language,
        hasMailAccessToken = hasMailAccessToken,
        smtpHost = smtpHost,
        smtpPort = smtpPort,
        imapHost = imapHost,
        imapPort = imapPort,
    )
}
```

Листинг 20 – Код файла `app/src/main/java/com/example/poster/data/remote/api/AttachmentApi.kt`

```
package com.example.poster.data.remote.api

import com.example.poster.data.remote.dto.message.AttachmentDto

interface AttachmentApi {
    suspend fun uploadAttachment(chatId: String, localUri: String): AttachmentDto
    suspend fun getAttachment(attachmentId: String): AttachmentDto
    suspend fun deleteAttachment(attachmentId: String)
}
```

Листинг 21 – Код файла `app/src/main/java/com/example/poster/data/remote/api/AuthApi.kt`

```
package com.example.poster.data.remote.api

import com.example.poster.data.remote.dto.auth.AuthResponseDto
import com.example.poster.data.remote.dto.auth.RefreshRequestDto
import com.example.poster.data.remote.dto.auth.RegisterResponseDto
import com.example.poster.data.remote.dto.auth.SignInRequestDto
import com.example.poster.data.remote.dto.auth.SignUpRequestDto
import com.example.poster.data.remote.dto.auth.VerifyOtpRequestDto

interface AuthApi {
    suspend fun signUp(request: SignUpRequestDto): RegisterResponseDto
    suspend fun signIn(request: SignInRequestDto): AuthResponseDto
    suspend fun verifyOtp(request: VerifyOtpRequestDto): AuthResponseDto
    suspend fun refresh(request: RefreshRequestDto): AuthResponseDto
    suspend fun logout()
}
```

Листинг 22 – Код файла app/src/main/java/com/example/poster/data/remote/api/ChatApi.kt

```
package com.example.poster.data.remote.api

import com.example.poster.data.remote.dto.chat.ChatDto
interface ChatApi {
    suspend fun getOrCreateChat(contactId: Long): ChatDto
    suspend fun markChatAsRead(chatId: String)
}
```

Листинг 23 – Код файла `app/src/main/java/com/example/poster/data/remote/api/MessageApi.kt`

```
package com.example.poster.data.remote.api

import com.example.poster.data.remote.dto.message.MessageDto
import com.example.poster.data.remote.dto.message.MessageListResponseDto
import com.example.poster.data.remote.dto.message.SendMessageRequestDto

interface MessageApi {
    suspend fun getMessages(chatId: String): MessageListResponseDto
    suspend fun sendMessage(chatId: String, request: SendMessageRequestDto): MessageDto
    suspend fun deleteMessage(messageId: String)
    suspend fun markMessageAsRead(messageId: String)
}
```

Листинг 24 – Код файла `app/src/main/java/com/example/poster/data/remote/api/PosterApiRoutes.kt`

```
package com.example.poster.data.remote.api

object PosterApiRoutes {
    const val SIGN_UP = "/auth/sign-up"
    const val SIGN_IN = "/auth/sign-in"
    const val VERIFY_OTP = "/auth/verify-otp"
    const val RESEND_OTP = "/auth/resend-otp"
    const val REFRESH = "/auth/refresh"
    const val LOGOUT = "/auth/logout"
    const val AUTH_ME = "/auth/me"

    const val PROFILE_ME = "/profile/me"
    const val PROFILE_AVATAR = "/profile/avatar"

    const val SETTINGS = "/settings"
    const val MAIL_ACCESS_TOKEN = "/settings/mail-access-token"
    const val MAIL_ACCESS_TOKEN_STATUS = "/settings/mail-access-token/status"

    const val CHATS = "/chats"
    const val CHATS_SYNC = "/chats/sync"

    const val WEB_SOCKET = "/ws"

    fun profile(userId: String): String = "/profile/$userId"
    fun chat(chatId: String): String = "/chats/$chatId"
    fun chatRead(chatId: String): String = "/chats/$chatId/read"
    fun chatMessages(chatId: String): String = "/chats/$chatId/messages"
    fun message(messageId: String): String = "/messages/$messageId"
    fun messageRead(messageId: String): String = "/messages/$messageId/read"
    fun messageRetry(messageId: String): String = "/messages/$messageId/retry"
}
```

Листинг 25 – Код файла `app/src/main/java/com/example/poster/data/remote/api/ProfileApi.kt`

```
package com.example.poster.data.remote.api

import com.example.poster.data.remote.dto.profile.ProfileDto
import com.example.poster.data.remote.dto.profile.UpdateProfileRequestDto
import com.example.poster.data.remote.dto.profile.UploadAvatarResponseDto

interface ProfileApi {
    suspend fun getMyProfile(): ProfileDto
    suspend fun updateMyProfile(request: UpdateProfileRequestDto): ProfileDto
    suspend fun getProfile(userId: String): ProfileDto
    suspend fun uploadAvatar(avatarPng: ByteArray): UploadAvatarResponseDto
}
```

Листинг 26 – Код файла `app/src/main/java/com/example/poster/data/remote/api/SettingsApi.kt`

```
package com.example.poster.data.remote.api

import com.example.poster.data.remote.dto.settings.MailAccessTokenStatusDto
import com.example.poster.data.remote.dto.settings.SaveMailAccessTokenRequestDto
import com.example.poster.data.remote.dto.settings.SettingsDto

interface SettingsApi {
    suspend fun getSettings(): SettingsDto
    suspend fun updateSettings(language: String): SettingsDto
    suspend fun saveMailAccessToken(request: SaveMailAccessTokenRequestDto)
    suspend fun deleteMailAccessToken()
    suspend fun getMailAccessTokenStatus(): MailAccessTokenStatusDto
}
```

Листинг 27 – Код файла `app/src/main/java/com/example/poster/data/remote/dto/auth/AuthDtos.kt`

```
package com.example.poster.data.remote.dto.auth
```

```
data class SignInRequestDto(  
    val email: String,  
    val password: String,  
)
```

```
data class SignUpRequestDto(  
    val email: String,  
    val password: String,  
    val displayName: String? = null,  
)
```

```
data class RegisterResponseDto(  
    val message: String,  
)
```

```
data class VerifyOtpRequestDto(  
    val email: String,  
    val code: String,  
)
```

```
data class RefreshRequestDto(  
    val refreshToken: String,  
)
```

```
data class AuthResponseDto(  
    val accessToken: String,  
    val refreshToken: String,  
    val profile: UserDto,  
)
```

```
data class UserDto(  
    val id: String,  
    val name: String,  
    val username: String,  
    val email: String,  
    val birthday: String? = null,  
    val bio: String? = null,  
    val avatarUrl: String? = null,  
    val isOnline: Boolean = false,  
)
```

Листинг 28 – Код файла `app/src/main/java/com/example/poster/data/remote/dto/chat/ChatDtos.kt`

```
package com.example.poster.data.remote.dto.chat
```

```
data class ChatDto(  
    val id: String,  
    val title: String,  
    val initials: String,  
    val lastMessage: String?,  
    val lastMessageTime: String?,  
    val unreadCount: Int,  
    val isOnline: Boolean,  
)
```

```
data class ChatListResponseDto(  
    val items: List<ChatDto>,  
)
```

```
data class CreateChatRequestDto(  
    val recipient: String,  
)
```

Листинг 29 – Код файла app/src/main/java/com/example/poster/data/remote/dto/message/MessageDtos.kt

```
package com.example.poster.data.remote.dto.message

data class MessageDto(
    val id: String,
    val chatId: String,
    val senderId: String,
    val bodyText: String?,
    val createdAt: String,
    val isMine: Boolean,
    val status: String,
    val attachments: List<AttachmentDto> = emptyList(),
)

data class SendMessageRequestDto(
    val bodyText: String,
)

data class MessageListResponseDto(
    val items: List<MessageDto>,
)

data class AttachmentDto(
    val id: String,
    val fileName: String,
    val mimeType: String,
    val sizeBytes: Long,
    val url: String,
    val type: String,
)
```

Листинг 30 – Код файла `app/src/main/java/com/example/poster/data/remote/dto/profile/ProfileDtos.kt`

```
package com.example.poster.data.remote.dto.profile
```

```
data class ProfileDto(  
    val id: String,  
    val name: String,  
    val username: String,  
    val email: String,  
    val birthday: String?,  
    val bio: String?,  
    val avatarUrl: String?,  
    val isOnline: Boolean,  
)
```

```
data class UpdateProfileRequestDto(  
    val name: String? = null,  
    val username: String? = null,  
    val bio: String? = null,  
    val birthday: String? = null,  
)
```

```
data class UploadAvatarResponseDto(  
    val avatarUrl: String,  
)
```

Листинг 31 – Код файла app/src/main/java/com/example/poster/data/remote/dto/settings/SettingsDtos.kt

```
package com.example.poster.data.remote.dto.settings
```

```
data class SettingsDto(  
    val language: String,  
    val hasMailAccessToken: Boolean,  
    val smtpHost: String? = null,  
    val smtpPort: Int? = null,  
    val imapHost: String? = null,  
    val imapPort: Int? = null,  
)
```

```
data class SaveMailAccessTokenRequestDto(  
    val token: String,  
    val smtpHost: String? = null,  
    val smtpPort: Int? = null,  
    val imapHost: String? = null,  
    val imapPort: Int? = null,  
)
```

```
data class MailAccessTokenStatusDto(  
    val configured: Boolean,  
)
```

Листинг 32 – Код файла `app/src/main/java/com/example/poster/data/remote/PosterRemoteDataSource.kt`

```
package com.example.poster.data.remote

import com.example.poster.core.network.NetworkConfig
import com.example.poster.data.remote.dto.chat.ChatDto
import com.example.poster.data.remote.dto.message.MessageDto
import com.example.poster.data.remote.dto.profile.ProfileDto
import com.example.poster.data.remote.dto.settings.SettingsDto
import com.example.poster.domain.model.MailAccessSettings
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.withContext
import org.json.JSONObject
import java.io.OutputStreamWriter
import java.net.HttpURLConnection
import java.net.URL
import java.util.UUID

data class RemoteAuthSessionDto(
    val accessToken: String,
    val refreshToken: String,
    val userId: String,
)

data class RemoteAttachmentDto(
    val id: String,
    val type: String,
    val localUri: String?,
    val remoteUrl: String?,
    val fileName: String,
    val mimeType: String,
    val sizeBytes: Long,
    val uploadStatus: String,
)

interface PosterRemoteDataSource {
    suspend fun login(email: String, password: String): RemoteAuthSessionDto
    suspend fun register(email: String, password: String, displayName: String)
    suspend fun verifyOtp(email: String, code: String): RemoteAuthSessionDto
    suspend fun refresh(refreshToken: String): RemoteAuthSessionDto
    suspend fun logout(accessToken: String?)
    suspend fun getChats(accessToken: String?): List<ChatDto>
    suspend fun getMessages(accessToken: String?, chatId: String): List<MessageDto>
    suspend fun sendMessage(accessToken: String?, chatId: String, text: String): MessageDto
    suspend fun markChatAsRead(accessToken: String?, chatId: String)
    suspend fun markMessageAsRead(accessToken: String?, messageId: String)
    suspend fun deleteMessage(accessToken: String?, messageId: String)
    suspend fun getMyProfile(accessToken: String?): ProfileDto
    suspend fun getProfile(accessToken: String?, userId: String): ProfileDto
    suspend fun updateProfile(accessToken: String?, name: String? = null, username: String? = null,
```

```

    bio: String? = null): ProfileDto
    suspend fun uploadAvatar(accessToken: String?, pngBytes: ByteArray): String
    suspend fun getSettings(accessToken: String?): SettingsDto
    suspend fun changeLanguage(accessToken: String?, language: String): SettingsDto
    suspend fun saveMailAccessToken(accessToken: String?, settings: MailAccessSettings)
    suspend fun deleteMailAccessToken(accessToken: String?)
    suspend fun hasMailAccessToken(accessToken: String?): Boolean
}

class HttpPosterRemoteDataSource(
    private val baseUrl: String = NetworkConfig.BASE_URL,
) : PosterRemoteDataSource {
    override suspend fun login(email: String, password: String): RemoteAuthSessionDto {
        return authSession(
            request(
                method = "POST",
                path = "auth/login",
                body = jsonObject("email" to email, "password" to password),
            )
        )
    }

    override suspend fun register(email: String, password: String, displayName: String) {
        request(
            method = "POST",
            path = "auth/register",
            body = jsonObject("email" to email, "password" to password, "displayName" to displayName),
            expectedCodes = setOf(HttpURLConnection.HTTP_CREATED),
        )
    }

    override suspend fun verifyOtp(email: String, code: String): RemoteAuthSessionDto {
        return authSession(
            request(
                method = "POST",
                path = "auth/verify-otp",
                body = jsonObject("email" to email, "code" to code),
            )
        )
    }

    override suspend fun refresh(refreshToken: String): RemoteAuthSessionDto {
        return authSession(
            request(
                method = "POST",
                path = "auth/refresh",
                body = jsonObject("refreshToken" to refreshToken),
            )
        )
    }
}

```

```

override suspend fun logout(accessToken: String?) {
    request(
        method = "POST",
        path = "auth/logout",
        accessToken = accessToken,
        expectedCodes = setOf(HttpURLConnection.HTTP_NO_CONTENT),
    )
}

override suspend fun getChats(accessToken: String?): List<ChatDto> {
    val contacts = request(method = "GET", path = "contacts", accessToken = accessToken)
        .getJSONArray("items")

    return buildList {
        for (index in 0 until contacts.length()) {
            val contact = contacts.getJSONObject(index)
            val contactId = contact.getLong("id")
            add(
                request(
                    method = "POST",
                    path = "contacts/\$contactId/chat",
                    accessToken = accessToken,
                ).toChatDto()
            )
        }
    }
}

override suspend fun getMessages(accessToken: String?, chatId: String): List<MessageDto> {
    val items = request(
        method = "GET",
        path = "chats/\$chatId/messages?limit=50&offset=0",
        accessToken = accessToken,
    ).getJSONArray("items")

    return buildList {
        for (index in 0 until items.length()) {
            add(items.getJSONObject(index).toMessageDto())
        }
    }
}

override suspend fun sendMessage(accessToken: String?, chatId: String, text: String): MessageDto {
    return request(
        method = "POST",
        path = "chats/\$chatId/messages",
        accessToken = accessToken,
        body = jsonObject("bodyText" to text),
        expectedCodes = setOf(HttpURLConnection.HTTP_CREATED),
    )
}

```

```

        ).getJSONObject("message").toMessageDto()
    }

    override suspend fun markChatAsRead(accessToken: String?, chatId: String) {
        request(
            method = "PATCH",
            path = "chats/$chatId/messages/read",
            accessToken = accessToken,
            expectedCodes = setOf(HttpURLConnection.HTTP_NO_CONTENT),
        )
    }

    override suspend fun markMessageAsRead(accessToken: String?, messageId: String) {
        request(
            method = "PATCH",
            path = "messages/$messageId/read",
            accessToken = accessToken,
            expectedCodes = setOf(HttpURLConnection.HTTP_NO_CONTENT),
        )
    }

    override suspend fun deleteMessage(accessToken: String?, messageId: String) {
        request(
            method = "DELETE",
            path = "messages/$messageId",
            accessToken = accessToken,
            expectedCodes = setOf(HttpURLConnection.HTTP_NO_CONTENT),
        )
    }

    override suspend fun getMyProfile(accessToken: String?): ProfileDto {
        return request(method = "GET", path = "profile/me", accessToken = accessToken).toProfileDto()
    }

    override suspend fun getProfile(accessToken: String?, userId: String): ProfileDto {
        return request(method = "GET", path = "profile/$userId", accessToken = accessToken).toProfileDto()
    }

    override suspend fun updateProfile(
        accessToken: String?,
        name: String?,
        username: String?,
        bio: String?,
    ): ProfileDto {
        val body = JSONObject().apply {
            name?.let { put("name", it) }
            username?.let { put("username", it) }
            bio?.let { put("bio", it) }
        }
    }

```

```

        return request(method = "PATCH", path = "profile/me", accessToken = accessToken, body = body)
    }
    .toProfileDto()
}

```

```

override suspend fun uploadAvatar(accessToken: String?, pngBytes: ByteArray): String = withContext(Dispatchers.IO) {
    val boundary = "PosterBoundary-${UUID.randomUUID()}"
    val connection = URL(baseUrl.trimEnd('/') + "/profile/avatar")
        .openConnection() as HttpURLConnection

    try {
        connection.requestMethod = "POST"
        connection.connectTimeout = NetworkConfig.CONNECT_TIMEOUT_SECONDS.toInt() * 1000
        connection.readTimeout = NetworkConfig.READ_TIMEOUT_SECONDS.toInt() * 1000
        connection.doOutput = true
        connection.setRequestProperty("Accept", "application/json")
        connection.setRequestProperty("Content-Type", "multipart/form-data; boundary=$boundary")

        accessToken?.takeIf(String::isNotBlank)?.let {
            connection.setRequestProperty("Authorization", "Bearer $it")
        }

        connection.outputStream.use { output ->
            output.write("--$boundary\r\n".toByteArray())
            output.write(
                "Content-Disposition: form-data; name=\"avatar\"; filename=\"avatar.png\"\r\n"
                    .toByteArray()
            )
            output.write("Content-Type: image/png\r\n\r\n".toByteArray())
            output.write(pngBytes)
            output.write("\r\n--$boundary--\r\n".toByteArray())
        }

        val responseCode = connection.responseCode
        val responseText = connection.inputOrErrorText()
        if (responseCode != HttpURLConnection.HTTP_OK) {
            throw PosterApiException(responseCode, responseText.errorMessage())
        }
        JSONObject(responseText).getString("avatarUrl").toAbsoluteUrl()
    } finally {
        connection.disconnect()
    }
}

override suspend fun getSettings(accessToken: String?): SettingsDto {
    return request(method = "GET", path = "settings", accessToken = accessToken).toSettingsDto()
}

```

```

override suspend fun changeLanguage(accessToken: String?, language: String): SettingsDto {
    return request(
        method = "PATCH",
        path = "settings",
        accessToken = accessToken,
        body = jsonObject("language" to language),
    ).toSettingsDto()
}

override suspend fun saveMailAccessToken(accessToken: String?, settings: MailAccessSettings)
{
    request(
        method = "PUT",
        path = "settings/mail-access-token",
        accessToken = accessToken,
        body = JSONObject().apply {
            put("token", settings.token)
            settings.smtpHost?.let { put("smtpHost", it) }
            settings.smtpPort?.let { put("smtpPort", it) }
            settings.imapHost?.let { put("imapHost", it) }
            settings.imapPort?.let { put("imapPort", it) }
        },
        expectedCodes = setOf(HttpURLConnection.HTTP_NO_CONTENT),
    )
}

override suspend fun deleteMailAccessToken(accessToken: String?) {
    request(
        method = "DELETE",
        path = "settings/mail-access-token",
        accessToken = accessToken,
        expectedCodes = setOf(HttpURLConnection.HTTP_NO_CONTENT),
    )
}

override suspend fun hasMailAccessToken(accessToken: String?): Boolean {
    return request(
        method = "GET",
        path = "settings/mail-access-token/status",
        accessToken = accessToken,
    ).getBoolean("configured")
}

private suspend fun request(
    method: String,
    path: String,
    accessToken: String? = null,
    body: JSONObject? = null,
    expectedCodes: Set<Int> = setOf(HttpURLConnection.HTTP_OK),
): JSONObject = withContext(Dispatchers.IO) {
    val connection = URL(baseUrl.trimEnd('/') + "/" + path.trimStart('/'))

```

```

        .openConnection() as HttpURLConnection

    try {
        connection.requestMethod = method
        connection.connectTimeout = NetworkConfig.CONNECT_TIMEOUT_SECONDS.toInt() *
1000
        connection.readTimeout = NetworkConfig.READ_TIMEOUT_SECONDS.toInt() * 1000
        connection.setRequestProperty("Accept", "application/json")
        accessToken?.takeIf(String::isNotBlank)?.let {
            connection.setRequestProperty("Authorization", "Bearer $it")
        }

        if (body != null) {
            connection.doOutput = true
            connection.setRequestProperty("Content-Type", "application/json")
            OutputStreamWriter(connection.outputStream).use { writer ->
                writer.write(body.toString())
            }
        }

        val responseCode = connection.responseCode
        val responseText = connection.inputOrErrorText()
        if (responseCode !in expectedCodes) {
            throw PosterApiException(responseCode, responseText.errorMessage())
        }

        if (responseText.isBlank()) {
            JSONObject()
        } else {
            JSONObject(responseText)
        }
    } finally {
        connection.disconnect()
    }
}

private fun HttpURLConnection.inputOrErrorText(): String {
    val stream = if (responseCode in 200..299) inputStream else errorStream
    return stream?.bufferedReader()?.use { it.readText() }.orEmpty()
}

private fun String.errorMessage(): String {
    return runCatching {
        val error = JSONObject(this)
        error.optString("message").ifBlank { error.optString("code") }
    }.getOrDefault(this).ifBlank { "Network request failed" }
}

private fun authSession(json: JSONObject): RemoteAuthSessionDto {
    val profile = json.getJSONObject("profile")
    return RemoteAuthSessionDto(

```

```

        accessToken = json.getString("accessToken"),
        refreshToken = json.getString("refreshToken"),
        userId = profile.getString("id"),
    )
}

private fun JSONObject.toChatDto(): ChatDto {
    return ChatDto(
        id = getString("id"),
        title = getString("title"),
        initials = optString("initials", getString("title").initials()),
        lastMessage = optStringOrNull("lastMessage"),
        lastMessageTime = optStringOrNull("lastMessageTime"),
        unreadCount = optInt("unreadCount", 0),
        isOnline = optBoolean("isOnline", false),
    )
}

private fun JSONObject.toMessageDto(): MessageDto {
    val direction = optString("direction")
    return MessageDto(
        id = getLong("id").toString(),
        chatId = getLong("chatId").toString(),
        senderId = "",
        bodyText = optStringOrNull("bodyText"),
        createdAt = optString("createdAt"),
        isMine = direction.equals("OUTGOING", ignoreCase = true),
        status = optString("status", "SENT"),
        attachments = emptyList(),
    )
}

private fun JSONObject.toProfileDto(): ProfileDto {
    return ProfileDto(
        id = getString("id"),
        name = getString("name"),
        username = getString("username"),
        email = getString("email"),
        birthday = optStringOrNull("birthday"),
        bio = optStringOrNull("bio"),
        avatarUrl = optStringOrNull("avatarUrl")?.toAbsoluteUrl(),
        isOnline = optBoolean("isOnline", false),
    )
}

private fun JSONObject.toSettingsDto(): SettingsDto {
    return SettingsDto(
        language = optString("language", "en"),
        hasMailAccessToken = optBoolean("hasMailAccessToken", false),
        smtpHost = optStringOrNull("smtpHost"),
        smtpPort = optIntOrNull("smtpPort"),
    )
}

```

```

        imapHost = optStringOrNull("imapHost"),
        imapPort = optIntOrNull("imapPort"),
    )
}

private fun JSONObject.optStringOrNull(name: String): String? {
    return if (isNull(name)) null else optString(name).ifBlank { null }
}

private fun JSONObject.optIntOrNull(name: String): Int? {
    return if (isNull(name) || !has(name)) null else optInt(name)
}

private fun String.toAbsolutePath(): String {
    return if (startsWith("http://") || startsWith("https://")) {
        this
    } else {
        baseUrl.trimEnd('/') + "/" + trimStart('/')
    }
}

private fun String.initials(): String {
    return split(" ")
        .filter(String::isNotBlank)
        .take(2)
        .joinToString("") { it.first().uppercase() }
        .ifBlank { "P" }
}

private fun jsonObject(vararg entries: Pair<String, Any?>): JSONObject {
    return JSONObject().apply {
        entries.forEach { (key, value) ->
            put(key, value)
        }
    }
}

class PosterApiException(
    val statusCode: Int,
    override val message: String,
) : Exception(message)

```

Листинг 33 – Код файла `app/src/main/java/com/example/poster/data/remote/websocket/PosterSocketEvent.kt`

```
package com.example.poster.data.remote.websocket
```

```
enum class PosterSocketEventType {  
    NEW_MESSAGE,  
    MESSAGE_SENT,  
    MESSAGE_READ,  
    CHAT_UPDATED,  
    SYNC_STARTED,  
    SYNC_FINISHED,  
    ERROR,  
}
```

```
data class PosterSocketEventDto(  
    val type: PosterSocketEventType,  
    val payload: String,  
)
```

Листинг 34 – Код файла `app/src/main/java/com/example/poster/data/repository/AuthRepositoryImpl.kt`

```
package com.example.poster.data.repository

import android.util.Log
import com.example.poster.data.local.PosterLocalDataSource
import com.example.poster.data.mapper.toDomain
import com.example.poster.data.mapper.toLocal
import com.example.poster.data.remote.PosterRemoteDataSource
import com.example.poster.domain.model.AuthSession
import com.example.poster.domain.repository.AuthRepository

class AuthRepositoryImpl(
    private val remoteDataSource: PosterRemoteDataSource,
    private val localDataSource: PosterLocalDataSource,
) : AuthRepository {
    override suspend fun signIn(email: String, password: String): Result<Unit> {
        Log.d(TAG, "signIn requested: email=$email")
        return runCatching {
            val session = remoteDataSource.login(email, password).toDomain()
            localDataSource.saveAuthSession(session.toLocal())
            Log.d(TAG, "signIn succeeded: userId=${session.userId}")
            Unit
        }.onFailure { error ->
            Log.e(TAG, "signIn failed: email=$email", error)
        }
    }

    override suspend fun signUp(
        name: String,
        email: String,
        password: String,
    ): Result<Unit> {
        Log.d(TAG, "signUp requested: email=$email")
        return runCatching {
            remoteDataSource.register(email, password, name)
            Log.d(TAG, "signUp succeeded: email=$email")
            Unit
        }.onFailure { error ->
            Log.e(TAG, "signUp failed: email=$email", error)
        }
    }

    override suspend fun verifyOtp(email: String, code: String): Result<Unit> {
        Log.d(TAG, "verifyOtp requested: email=$email")
        return runCatching {
            val session = remoteDataSource.verifyOtp(email, code).toDomain()
            localDataSource.saveAuthSession(session.toLocal())
            Log.d(TAG, "verifyOtp succeeded: userId=${session.userId}")
            Unit
        }
    }
}
```

```

        }.onFailure { error ->
            Log.e(TAG, "verifyOtp failed: email=$email", error)
        }
    }

    override suspend fun resendOtp(email: String): Result<Unit> {
        return Result.success(Unit)
    }

    override suspend fun logout(): Result<Unit> {
        Log.d(TAG, "logout requested")
        return runCatching {
            remoteDataSource.logout(localDataSource.getAuthSession()?.accessToken)
            localDataSource.clearAuthSession()
            Log.d(TAG, "logout succeeded")
            Unit
        }.onFailure { error ->
            Log.e(TAG, "logout failed", error)
        }
    }

    override suspend fun isAuthorized(): Boolean {
        return localDataSource.getAuthSession() != null
    }

    override suspend fun refreshToken(): Result<Unit> {
        Log.d(TAG, "refreshToken requested")
        return runCatching {
            val currentSession = localDataSource.getAuthSession()
                ?: throw IllegalStateException("No active session")
            val refreshedSession = remoteDataSource.refresh(currentSession.refreshToken).toDomain()
            localDataSource.saveAuthSession(refreshedSession.toLocal())
            Log.d(TAG, "refreshToken succeeded: userId=${refreshedSession.userId}")
            Unit
        }.onFailure { error ->
            Log.e(TAG, "refreshToken failed", error)
        }
    }

    override suspend fun getActiveSession(): AuthSession? {
        return localDataSource.getAuthSession()?.toDomain()
    }

    private companion object {
        private const val TAG = "AuthRepository"
    }
}

```

Листинг 35 – Код файла `app/src/main/java/com/example/poster/data/repository/ChatRepositoryImpl.kt`

```
package com.example.poster.data.repository

import android.util.Log
import com.example.poster.data.mapper.toDomain
import com.example.poster.data.mapper.toLocal
import com.example.poster.data.remote.PosterApiException
import com.example.poster.data.remote.PosterRemoteDataSource
import com.example.poster.domain.model.Chat
import com.example.poster.domain.repository.ChatRepository
import kotlinx.coroutines.flow.Flow
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.asStateFlow

class ChatRepositoryImpl(
    private val remoteDataSource: PosterRemoteDataSource,
    private val localDataSource: com.example.poster.data.local.PosterLocalDataSource,
) : ChatRepository {
    private val chatsFlow = MutableStateFlow<List<Chat>>(emptyList())

    override suspend fun getChats(): Result<List<Chat>> {
        Log.d(TAG, "getChats requested")
        return runCatching {
            val chats = authorized { remoteDataSource.getChats(it) }.map { it.toDomain() }
            chatsFlow.value = chats
            Log.d(TAG, "getChats succeeded: count=${chats.size}")
            chats
        }.onFailure { error ->
            Log.e(TAG, "getChats failed", error)
        }
    }

    override suspend fun searchChats(query: String): Result<List<Chat>> {
        Log.d(TAG, "searchChats requested: queryLength=${query.length}")
        val chats = if (chatsFlow.value.isEmpty()) {
            authorized { remoteDataSource.getChats(it) }.map { it.toDomain() }
        } else {
            chatsFlow.value
        }
        val normalizedQuery = query.trim()
        val filteredChats = if (normalizedQuery.isBlank()) {
            chats
        } else {
            chats.filter {
                it.title.contains(normalizedQuery, ignoreCase = true) ||
                it.lastMessage.contains(normalizedQuery, ignoreCase = true)
            }
        }
        return Result.success(filteredChats)
    }
}
```

```

}

override suspend fun syncChats(): Result<Unit> {
    Log.d(TAG, "syncChats requested")
    return runCatching {
        chatsFlow.value = authorized { remoteDataSource.getChats(it) }.map { it.toDomain() }
        Log.d(TAG, "syncChats succeeded: count=${chatsFlow.value.size}")
        Unit
    }.onFailure { error ->
        Log.e(TAG, "syncChats failed", error)
    }
}

override fun observeChats(): Flow<List<Chat>> {
    return chatsFlow.asStateFlow()
}

override suspend fun markChatAsRead(chatId: String): Result<Unit> {
    Log.d(TAG, "markChatAsRead requested: chatId=$chatId")
    return runCatching {
        authorized { remoteDataSource.markChatAsRead(it, chatId) }
        chatsFlow.value = chatsFlow.value.map { chat ->
            if (chat.id == chatId) {
                chat.copy(unreadCount = 0)
            } else {
                chat
            }
        }
        Log.d(TAG, "markChatAsRead succeeded: chatId=$chatId")
        Unit
    }.onFailure { error ->
        Log.e(TAG, "markChatAsRead failed: chatId=$chatId", error)
    }
}

override suspend fun deleteChat(chatId: String): Result<Unit> {
    chatsFlow.value = chatsFlow.value.filterNot { it.id == chatId }
    return Result.success(Unit)
}

private suspend fun accessToken(): String? {
    return localDataSource.getAuthSession()?.accessToken
}

private suspend fun <T> authorized(block: suspend (String?) -> T): T {
    return try {
        block(accessToken())
    } catch (error: PosterApiException) {
        if (error.statusCode != 401) throw error
        Log.d(TAG, "Access token expired, refreshing for chat request")
        val currentSession = localDataSource.getAuthSession() ?: throw error
    }
}

```

```
        val refreshedSession = remoteDataSource.refresh(currentSession.refreshToken).toDomain()
        localDataSource.saveAuthSession(refreshedSession.toLocal())
        block(refreshedSession.accessToken)
    }
}

private companion object {
    private const val TAG = "ChatRepository"
}
}
```

Листинг 36 – Код файла `app/src/main/java/com/example/poster/data/repository/FileRepositoryImpl.kt`

```
package com.example.poster.data.repository

import android.content.Context
import android.provider.OpenableColumns
import com.example.poster.domain.model.Attachment
import com.example.poster.domain.model.AttachmentType
import com.example.poster.domain.model.AttachmentUploadStatus
import com.example.poster.domain.repository.FileRepository

class FileRepositoryImpl(
    private val context: Context,
) : FileRepository {
    override suspend fun getAttachmentInfo(uri: String): Result<Attachment> {
        val parsedUri = android.net.Uri.parse(uri)
        val contentResolver = context.contentResolver
        val mimeType = contentResolver.getType(parsedUri).orEmpty().ifBlank {
            "application/octet-stream"
        }

        var fileName = parsedUri.lastPathSegment ?: "attachment"
        var sizeBytes = 0L

        contentResolver.query(parsedUri, null, null, null, null)?.use { cursor ->
            val nameIndex = cursor.getColumnIndex(OpenableColumns.DISPLAY_NAME)
            val sizeIndex = cursor.getColumnIndex(OpenableColumns.SIZE)
            if (cursor.moveToFirst()) {
                if (nameIndex >= 0) {
                    fileName = cursor.getString(nameIndex) ?: fileName
                }
                if (sizeIndex >= 0) {
                    sizeBytes = cursor.getLong(sizeIndex)
                }
            }
        }

        return Result.success(
            Attachment(
                id = "local-`${System.currentTimeMillis()}`,
                localUri = uri,
                remoteUrl = null,
                fileName = fileName,
                mimeType = mimeType,
                sizeBytes = sizeBytes,
                type = mimeType.toAttachmentType(),
                uploadStatus = AttachmentUploadStatus.LOCAL_ONLY,
            )
        )
    }
}
```

```

override suspend fun copyToAppCache(uri: String): Result<Attachment> {
    return getAttachmentInfo(uri)
}

override suspend fun saveToDownloads(attachment: Attachment): Result<Unit> {
    return Result.success(Unit)
}

override suspend fun openAttachment(attachment: Attachment): Result<Unit> {
    return Result.success(Unit)
}
}

private fun String.toAttachmentType(): AttachmentType {
    return when {
        startsWith("image/") -> AttachmentType.IMAGE
        startsWith("video/") -> AttachmentType.VIDEO
        startsWith("audio/") -> AttachmentType.AUDIO
        contains("pdf") || contains("document") || contains("text") -> AttachmentType.DOCUMENT
        else -> AttachmentType.UNKNOWN
    }
}

```

Листинг 37 – Код файла app/src/main/java/com/example/poster/data/repository/MessageRepositoryImpl.kt

```
package com.example.poster.data.repository

import android.util.Log
import com.example.poster.data.mapper.toDomain
import com.example.poster.data.mapper.toLocal
import com.example.poster.data.remote.PosterApiException
import com.example.poster.data.remote.PosterRemoteDataSource
import com.example.poster.domain.model.Attachment
import com.example.poster.domain.model.Message
import com.example.poster.domain.repository.MessageRepository
import kotlinx.coroutines.flow.Flow
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.asStateFlow

class MessageRepositoryImpl(
    private val remoteDataSource: PosterRemoteDataSource,
    private val localDataSource: com.example.poster.data.local.PosterLocalDataSource,
) : MessageRepository {
    private val messagesByChat = mutableMapOf<String, MutableStateFlow<List<Message>>>()

    override suspend fun getMessages(chatId: String): Result<List<Message>> {
        Log.d(TAG, "getMessages requested: chatId=$chatId")
        return runCatching {
            val messages = remoteDataSource
                .let { authorized { token -> it.getMessages(token, chatId) } }
                .map { it.toDomain() }
            getOrCreateFlow(chatId).value = messages
            Log.d(TAG, "getMessages succeeded: chatId=$chatId, count=${messages.size}")
            messages
        }.onFailure { error ->
            Log.e(TAG, "getMessages failed: chatId=$chatId", error)
        }
    }

    override fun observeMessages(chatId: String): Flow<List<Message>> {
        return getOrCreateFlow(chatId).asStateFlow()
    }

    override suspend fun sendMessage(chatId: String, text: String, attachments: List<Attachment>):
        Result<Message> {
        Log.d(TAG, "sendMessage requested: chatId=$chatId, textLength=${text.length}, attachments
        =${attachments.size}")
        return runCatching {
            if (attachments.isNotEmpty()) {
                throw UnsupportedOperationException("Message attachments are not supported by the d
                ocumented API")
            }
        }
    }
}
```

```

val createdMessage = remoteDataSource
    .let { authorized { token -> it.sendMessage(token, chatId, text) } }
    .toDomain()

val currentMessages = getOrCreateFlow(chatId).value
getOrCreateFlow(chatId).value = currentMessages + createdMessage
Log.d(TAG, "sendMessage succeeded: chatId=$chatId, messageId=${createdMessage.id}")
createdMessage
}.onFailure { error ->
    Log.e(TAG, "sendMessage failed: chatId=$chatId", error)
}
}

override suspend fun uploadAttachment(chatId: String, attachment: Attachment): Result<Attachment> {
    return Result.success(attachment)
}

override suspend fun downloadAttachment(attachmentId: String): Result<Attachment> {
    val attachment = messagesByChat.values
        .flatMap { flow -> flow.value }
        .flatMap { message -> message.attachments }
        .firstOrNull { attachment -> attachment.id == attachmentId }
        ?: return Result.failure(IllegalArgumentException("Attachment not found"))

    return Result.success(attachment)
}

override suspend fun markMessageAsRead(messageId: String): Result<Unit> {
    Log.d(TAG, "markMessageAsRead requested: messageId=$messageId")
    return runCatching {
        authorized { remoteDataSource.markMessageAsRead(it, messageId) }
        Log.d(TAG, "markMessageAsRead succeeded: messageId=$messageId")
        Unit
    }.onFailure { error ->
        Log.e(TAG, "markMessageAsRead failed: messageId=$messageId", error)
    }
}

override suspend fun deleteMessage(messageId: String): Result<Unit> {
    Log.d(TAG, "deleteMessage requested: messageId=$messageId")
    return runCatching {
        authorized { remoteDataSource.deleteMessage(it, messageId) }
        messagesByChat.forEach { (_, flow) ->
            flow.value = flow.value.filterNot { it.id == messageId }
        }
        Log.d(TAG, "deleteMessage succeeded: messageId=$messageId")
        Unit
    }.onFailure { error ->
        Log.e(TAG, "deleteMessage failed: messageId=$messageId", error)
    }
}

```

```

    }

    private fun getOrCreateFlow(chatId: String): MutableStateFlow<List<Message>> {
        return messagesByChat.getOrPut(chatId) {
            MutableStateFlow(emptyList())
        }
    }

    private suspend fun accessToken(): String? {
        return localDataSource.getAuthSession()?.accessToken
    }

    private suspend fun <T> authorized(block: suspend (String?) -> T): T {
        return try {
            block(accessToken())
        } catch (error: PosterApiException) {
            if (error.statusCode != 401) throw error
            Log.d(TAG, "Access token expired, refreshing for message request")
            val currentSession = localDataSource.getAuthSession() ?: throw error
            val refreshedSession = remoteDataSource.refresh(currentSession.refreshToken).toDomain()
            localDataSource.saveAuthSession(refreshedSession.toLocal())
            block(refreshedSession.accessToken)
        }
    }

    private companion object {
        private const val TAG = "MessageRepository"
    }
}

```

Листинг 38 – Код файла `app/src/main/java/com/example/poster/data/repository/MockAuthRepository.kt`

```
package com.example.poster.data.repository

import com.example.poster.domain.model.AuthSession
import com.example.poster.domain.repository.AuthRepository

class MockAuthRepository : AuthRepository {
    private var session: AuthSession? = null

    override suspend fun signIn(email: String, password: String): Result<Unit> {
        session = createSession(email)
        return Result.success(Unit)
    }

    override suspend fun signUp(name: String, email: String, password: String): Result<Unit> {
        session = createSession(email)
        return Result.success(Unit)
    }

    override suspend fun verifyOtp(email: String, code: String): Result<Unit> {
        session = createSession(email)
        return Result.success(Unit)
    }

    override suspend fun resendOtp(email: String): Result<Unit> {
        return Result.success(Unit)
    }

    override suspend fun logout(): Result<Unit> {
        session = null
        return Result.success(Unit)
    }

    override suspend fun isAuthorized(): Boolean {
        return session != null
    }

    override suspend fun refreshToken(): Result<Unit> {
        val currentSession = session
        ?: return Result.failure(IllegalStateException("No active session"))
        session = currentSession.copy(accessToken = "mock-access-refreshed")
        return Result.success(Unit)
    }

    override suspend fun getActiveSession(): AuthSession? {
        return session
    }

    private fun createSession(email: String): AuthSession {
```

```
    return AuthSession(  
        accessToken = "mock-access-token",  
        refreshToken = "mock-refresh-token",  
        userId = "user-#{email.hashCode()}",  
    )  
}
```

Листинг 39 – Код файла app/src/main/java/com/example/poster/data/repository/MockChatRepository.kt

```
package com.example.poster.data.repository

import com.example.poster.domain.model.Chat
import com.example.poster.domain.repository.ChatRepository
import kotlinx.coroutines.flow.Flow
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.asStateFlow

class MockChatRepository : ChatRepository {
    private val chatsFlow = MutableStateFlow(mockChats())

    override suspend fun getChats(): Result<List<Chat>> {
        return Result.success(chatsFlow.value)
    }

    override suspend fun searchChats(query: String): Result<List<Chat>> {
        val normalizedQuery = query.trim()
        val chats = if (normalizedQuery.isBlank()) {
            chatsFlow.value
        } else {
            chatsFlow.value.filter {
                it.title.contains(normalizedQuery, ignoreCase = true) ||
                it.lastMessage.contains(normalizedQuery, ignoreCase = true)
            }
        }
        return Result.success(chats)
    }

    override suspend fun syncChats(): Result<Unit> {
        chatsFlow.value = mockChats()
        return Result.success(Unit)
    }

    override fun observeChats(): Flow<List<Chat>> {
        return chatsFlow.asStateFlow()
    }

    override suspend fun markChatAsRead(chatId: String): Result<Unit> {
        chatsFlow.value = chatsFlow.value.map { chat ->
            if (chat.id == chatId) chat.copy(unreadCount = 0) else chat
        }
        return Result.success(Unit)
    }

    override suspend fun deleteChat(chatId: String): Result<Unit> {
        chatsFlow.value = chatsFlow.value.filterNot { it.id == chatId }
        return Result.success(Unit)
    }
}
```

```

private fun mockChats(): List<Chat> {
    return listOf(
        Chat(
            id = "1",
            title = "Alice Johnson",
            initials = "AJ",
            lastMessage = "See you tomorrow!",
            lastMessageTime = "10:30 AM",
            unreadCount = 2,
        ),
        Chat(
            id = "2",
            title = "Bob Smith",
            initials = "BS",
            lastMessage = "Thanks for the update",
            lastMessageTime = "Yesterday",
            unreadCount = 0,
        ),
        Chat(
            id = "3",
            title = "Carol White",
            initials = "CW",
            lastMessage = "Meeting at 3 PM?",
            lastMessageTime = "Yesterday",
            unreadCount = 1,
        ),
        Chat(
            id = "4",
            title = "David Brown",
            initials = "DB",
            lastMessage = "Perfect, sounds good!",
            lastMessageTime = "2 days ago",
            unreadCount = 0,
        ),
        Chat(
            id = "5",
            title = "Emma Davis",
            initials = "ED",
            lastMessage = "I'll send the files",
            lastMessageTime = "3 days ago",
            unreadCount = 0,
        ),
        Chat(
            id = "6",
            title = "Frank King",
            initials = "FK",
            lastMessage = "SMTP sync completed",
            lastMessageTime = "Monday",
            unreadCount = 0,
        ),
    ),
}

```

```

Chat(
    id = "7",
    title = "Grace Lee",
    initials = "GL",
    lastMessage = "Can we review the IMAP logs?",
    lastMessageTime = "Sunday",
    unreadCount = 4,
),
Chat(
    id = "8",
    title = "Henry Miller",
    initials = "HM",
    lastMessage = "Token settings look correct",
    lastMessageTime = "Saturday",
    unreadCount = 0,
),
Chat(
    id = "9",
    title = "Ivy Nelson",
    initials = "IN",
    lastMessage = "New message arrived via mail",
    lastMessageTime = "Friday",
    unreadCount = 3,
),
Chat(
    id = "10",
    title = "Jack Owens",
    initials = "JO",
    lastMessage = "I'll check the server response",
    lastMessageTime = "Friday",
    unreadCount = 0,
),
Chat(
    id = "11",
    title = "Kate Parker",
    initials = "KP",
    lastMessage = "Thanks, it works now",
    lastMessageTime = "Thursday",
    unreadCount = 0,
),
Chat(
    id = "12",
    title = "Liam Reed",
    initials = "LR",
    lastMessage = "Let's deploy the Ktor endpoint",
    lastMessageTime = "Wednesday",
    unreadCount = 1,
),
    )
}
}

```

Листинг 40 – Код файла app/src/main/java/com/example/poster/data/repository/MockMessageRepository.kt

```
package com.example.poster.data.repository

import com.example.poster.domain.model.Attachment
import com.example.poster.domain.model.AttachmentType
import com.example.poster.domain.model.AttachmentUploadStatus
import com.example.poster.domain.model.Message
import com.example.poster.domain.model.MessageStatus
import com.example.poster.domain.repository.MessageRepository
import kotlinx.coroutines.flow.Flow
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.asStateFlow

class MockMessageRepository : MessageRepository {
    private val messagesByChat = mutableMapOf<String, MutableStateFlow<List<Message>>>()

    override suspend fun getMessages(chatId: String): Result<List<Message>> {
        val flow = getOrCreateFlow(chatId)
        return Result.success(flow.value)
    }

    override fun observeMessages(chatId: String): Flow<List<Message>> {
        return getOrCreateFlow(chatId).asStateFlow()
    }

    override suspend fun sendMessage(
        chatId: String,
        text: String,
        attachments: List<Attachment>,
    ): Result<Message> {
        val message = Message(
            id = System.currentTimeMillis().toString(),
            chatId = chatId,
            text = text,
            time = "Now",
            isMine = true,
            status = MessageStatus.SENT,
            attachments = attachments,
        )
        val flow = getOrCreateFlow(chatId)
        flow.value = flow.value + message
        return Result.success(message)
    }

    override suspend fun uploadAttachment(chatId: String, attachment: Attachment): Result<Attachment> {
        return Result.success(attachment)
    }
}
```

```

override suspend fun downloadAttachment(attachmentId: String): Result<Attachment> {
    val attachment = messagesByChat.values
        .flatMap { flow -> flow.value }
        .flatMap { message -> message.attachments }
        .firstOrNull { attachment -> attachment.id == attachmentId }
        ?: return Result.failure(IllegalArgumentException("Attachment not found"))

    return Result.success(attachment)
}

override suspend fun markMessageAsRead(messageId: String): Result<Unit> {
    return Result.success(Unit)
}

override suspend fun deleteMessage(messageId: String): Result<Unit> {
    messagesByChat.forEach { (_, flow) ->
        flow.value = flow.value.filterNot { it.id == messageId }
    }
    return Result.success(Unit)
}

private fun getOrCreateFlow(chatId: String): MutableStateFlow<List<Message>> {
    return messagesByChat.getOrPut(chatId) {
        MutableStateFlow(mockMessages(chatId))
    }
}

private fun mockMessages(chatId: String): List<Message> {
    return listOf(
        Message(
            id = "$chatId-1",
            chatId = chatId,
            text = "Hey! How are you?",
            time = "10:00 AM",
            isMine = false,
            status = MessageStatus.READ,
        ),
        Message(
            id = "$chatId-2",
            chatId = chatId,
            text = "I'm good, thanks! Just finished the project.",
            time = "10:05 AM",
            isMine = true,
            status = MessageStatus.READ,
        ),
        Message(
            id = "$chatId-3",
            chatId = chatId,
            text = "That's great! Can you send me the files?",
            time = "10:06 AM",
            isMine = false,

```

```

        status = MessageStatus.READ,
    ),
    Message(
        id = "$chatId-4",
        chatId = chatId,
        text = "Sure, I'll send them in a few minutes.",
        time = "10:08 AM",
        isMine = true,
        status = MessageStatus.READ,
    ),
    Message(
        id = "$chatId-5",
        chatId = chatId,
        text = "Thanks! Looking forward to it.",
        time = "10:10 AM",
        isMine = false,
        status = MessageStatus.READ,
    ),
    Message(
        id = "$chatId-6",
        chatId = chatId,
        text = "",
        time = "4:17 PM",
        isMine = true,
        status = MessageStatus.SENT,
        attachments = listOf(
            Attachment(
                id = "$chatId-file-1",
                localUri = null,
                remoteUrl = null,
                type = AttachmentType.DOCUMENT,
                fileName = "RSUBD_PR7_Savvin_SS_IKBO_63_23.pdf",
                mimeType = "application/pdf",
                sizeBytes = 1_017_958L,
                uploadStatus = AttachmentUploadStatus.UPLOADED,
            ),
        ),
    ),
)
}
}

```

Листинг 41 – Код файла `app/src/main/java/com/example/poster/data/repository/MockProfileRepository.kt`

```
package com.example.poster.data.repository

import com.example.poster.domain.model.Profile
import com.example.poster.domain.repository.ProfileRepository

class MockProfileRepository : ProfileRepository {
    private var myProfile = Profile(
        id = "user-1",
        name = "Your Name",
        username = "@your.username",
        email = "your.email@example.com",
        bio = "Hey there! I'm using Poster.",
        avatarUrl = null,
        isOnline = true,
    )

    override suspend fun getMyProfile(): Result<Profile> {
        return Result.success(myProfile)
    }

    override suspend fun getUserProfile(userId: String): Result<Profile> {
        return Result.success(
            Profile(
                id = userId,
                name = "Alice Johnson",
                username = "@alice.johnson",
                email = "alice@example.com",
                bio = "Product designer passionate about creating beautiful and functional user experience
s.",
                avatarUrl = null,
                isOnline = true,
            )
        )
    }

    override suspend fun updateName(name: String): Result<Profile> {
        myProfile = myProfile.copy(name = name)
        return Result.success(myProfile)
    }

    override suspend fun updateUsername(username: String): Result<Profile> {
        myProfile = myProfile.copy(username = username)
        return Result.success(myProfile)
    }

    override suspend fun updateBio(bio: String): Result<Profile> {
        myProfile = myProfile.copy(bio = bio)
        return Result.success(myProfile)
    }
}
```

```
}  
  
override suspend fun updateAvatar(uri: String): Result<Profile> {  
    myProfile = myProfile.copy(avatarUrl = uri)  
    return Result.success(myProfile)  
}  
}
```

Листинг 42 – Код файла `app/src/main/java/com/example/poster/data/repository/MockSettingsRepository.kt`

```
package com.example.poster.data.repository

import com.example.poster.domain.model.UserSettings
import com.example.poster.domain.model.MailAccessSettings
import com.example.poster.domain.repository.SettingsRepository

class MockSettingsRepository : SettingsRepository {
    private var hasMailAccessToken = false
    private var language = "English"

    override suspend fun getSettings(): Result<UserSettings> {
        return Result.success(
            UserSettings(
                name = "Your Name",
                username = "@your.username",
                email = "your.email@example.com",
                birthday = "January 1, 2000",
                bio = "Hey there! I'm using Poster.",
                language = language,
                hasMailAccessToken = hasMailAccessToken,
            )
        )
    }

    override suspend fun saveMailAccessToken(settings: MailAccessSettings): Result<Unit> {
        hasMailAccessToken = settings.token.isNotBlank()
        return Result.success(Unit)
    }

    override suspend fun hasMailAccessToken(): Boolean {
        return hasMailAccessToken
    }

    override suspend fun deleteMailAccessToken(): Result<Unit> {
        hasMailAccessToken = false
        return Result.success(Unit)
    }

    override suspend fun changeLanguage(languageCode: String): Result<Unit> {
        language = languageCode
        return Result.success(Unit)
    }
}
```

Листинг 43 – Код файла app/src/main/java/com/example/poster/data/repository/ProfileRepositoryImpl.kt

```
package com.example.poster.data.repository

import android.content.Context
import android.net.Uri
import android.util.Log
import com.example.poster.data.mapper.toDomain
import com.example.poster.data.mapper.toLocal
import com.example.poster.data.remote.PosterApiException
import com.example.poster.data.remote.PosterRemoteDataSource
import com.example.poster.domain.model.Profile
import com.example.poster.domain.repository.ProfileRepository

class ProfileRepositoryImpl(
    private val remoteDataSource: PosterRemoteDataSource,
    private val localDataSource: com.example.poster.data.local.PosterLocalDataSource,
    private val context: Context? = null,
) : ProfileRepository {
    private var myProfile: Profile? = null

    override suspend fun getMyProfile(): Result<Profile> {
        Log.d(TAG, "getMyProfile requested")
        return runCatching {
            val profile = myProfile ?: authorized { remoteDataSource.getMyProfile(it) }.toDomain()
            myProfile = profile
            Log.d(TAG, "getMyProfile succeeded: id=${profile.id}")
            profile
        }.onFailure { error ->
            Log.e(TAG, "getMyProfile failed", error)
        }
    }

    override suspend fun getUserProfile(userId: String): Result<Profile> {
        Log.d(TAG, "getUserProfile requested: userId=$userId")
        return runCatching {
            authorized { remoteDataSource.getProfile(it, userId) }.toDomain()
        }.onFailure { error ->
            Log.e(TAG, "getUserProfile failed: userId=$userId", error)
        }
    }

    override suspend fun updateName(name: String): Result<Profile> {
        return updateMyProfile { it.copy(name = name) }
    }

    override suspend fun updateUsername(username: String): Result<Profile> {
        return updateMyProfile { it.copy(username = username) }
    }
}
```

```

override suspend fun updateBio(bio: String): Result<Profile> {
    return updateMyProfile { it.copy(bio = bio) }
}

override suspend fun updateAvatar(uri: String): Result<Profile> {
    Log.d(TAG, "updateAvatar requested")
    return runCatching {
        val appContext = context
            ?: throw IllegalStateException("Application context is required to upload avatar")
        val avatarUri = Uri.parse(uri)
        val contentType = appContext.contentResolver.getType(avatarUri)
        if (contentType != "image/png") {
            throw IllegalArgumentException("Avatar must be PNG")
        }
        val bytes = appContext.contentResolver.openInputStream(avatarUri)?.use { it.readBytes() }
            ?: throw IllegalArgumentException("Cannot read avatar file")
        val avatarUrl = authorized { remoteDataSource.uploadAvatar(it, bytes) }
        val currentProfile = getMyProfile().getOrThrow()
        val updatedProfile = currentProfile.copy(avatarUrl = avatarUrl)
        myProfile = updatedProfile
        Log.d(TAG, "updateAvatar succeeded: id=${updatedProfile.id}")
        updatedProfile
    }.onFailure { error ->
        Log.e(TAG, "updateAvatar failed", error)
    }
}

private suspend fun updateMyProfile(transform: (Profile) -> Profile): Result<Profile> = runCatching {
    val currentProfile = getMyProfile().getOrThrow()
    val desiredProfile = transform(currentProfile)
    Log.d(TAG, "updateMyProfile requested: id=${currentProfile.id}")
    val updatedProfile = authorized { token -> remoteDataSource.updateProfile(
        accessToken = token,
        name = desiredProfile.name.takeIf { it != currentProfile.name },
        username = desiredProfile.username.takeIf { it != currentProfile.username },
        bio = desiredProfile.bio.takeIf { it != currentProfile.bio },
    ) }.toDomain()
    myProfile = updatedProfile
    Log.d(TAG, "updateMyProfile succeeded: id=${updatedProfile.id}")
    updatedProfile
}.onFailure { error ->
    Log.e(TAG, "updateMyProfile failed", error)
}

private suspend fun accessToken(): String? {
    return localDataSource.getAuthSession()?.accessToken
}

private suspend fun <T> authorized(block: suspend (String?) -> T): T {
    return try {

```

```

        block(accessToken())
    } catch (error: PosterApiException) {
        if (error.statusCode != 401) throw error
        Log.d(TAG, "Access token expired, refreshing for profile request")
        val currentSession = localDataSource.getAuthSession() ?: throw error
        val refreshedSession = remoteDataSource.refresh(currentSession.refreshToken).toDomain()
        localDataSource.saveAuthSession(refreshedSession.toLocal())
        block(refreshedSession.accessToken)
    }
}

private companion object {
    private const val TAG = "ProfileRepository"
}
}

```

Листинг 44 – Код файла `app/src/main/java/com/example/poster/data/repository/SettingsRepositoryImpl.kt`

```
package com.example.poster.data.repository

import android.util.Log
import com.example.poster.data.local.PosterLocalDataSource
import com.example.poster.data.mapper.toDomain
import com.example.poster.data.mapper.toLocal
import com.example.poster.data.remote.PosterApiException
import com.example.poster.data.remote.PosterRemoteDataSource
import com.example.poster.domain.model.MailAccessSettings
import com.example.poster.domain.model.UserSettings
import com.example.poster.domain.repository.SettingsRepository

class SettingsRepositoryImpl(
    private val remoteDataSource: PosterRemoteDataSource,
    private val localDataSource: PosterLocalDataSource,
) : SettingsRepository {
    override suspend fun getSettings(): Result<UserSettings> {
        Log.d(TAG, "getSettings requested")
        return runCatching {
            authorized { token ->
                remoteDataSource.getSettings(token).toDomain(
                    profile = remoteDataSource.getMyProfile(token),
                )
            }
        }.onSuccess { settings ->
            Log.d(TAG, "getSettings succeeded: hasMailAccessToken=${settings.hasMailAccessToken}")
        }.onFailure { error ->
            Log.e(TAG, "getSettings failed", error)
        }
    }

    override suspend fun saveMailAccessToken(settings: MailAccessSettings): Result<Unit> {
        Log.d(TAG, "saveMailAccessToken requested")
        return runCatching {
            authorized { remoteDataSource.saveMailAccessToken(it, settings) }
        }.onSuccess {
            Log.d(TAG, "saveMailAccessToken succeeded")
        }.onFailure { error ->
            Log.e(TAG, "saveMailAccessToken failed", error)
        }
    }

    override suspend fun hasMailAccessToken(): Boolean {
        Log.d(TAG, "hasMailAccessToken requested")
        return runCatching {
            authorized { remoteDataSource.hasMailAccessToken(it) }
        }.onSuccess { hasToken ->
            hasToken
        }
    }
}
```

```

        Log.d(TAG, "hasMailAccessToken succeeded: $hasToken")
    }.onFailure { error ->
        Log.e(TAG, "hasMailAccessToken failed", error)
    }.getOrDefault(false)
}

override suspend fun deleteMailAccessToken(): Result<Unit> {
    Log.d(TAG, "deleteMailAccessToken requested")
    return runCatching {
        authorized { remoteDataSource.deleteMailAccessToken(it) }
    }.onSuccess {
        Log.d(TAG, "deleteMailAccessToken succeeded")
    }.onFailure { error ->
        Log.e(TAG, "deleteMailAccessToken failed", error)
    }
}

override suspend fun changeLanguage(languageCode: String): Result<Unit> {
    Log.d(TAG, "changeLanguage requested: languageCode=$languageCode")
    return runCatching {
        authorized { remoteDataSource.changeLanguage(it, languageCode) }
    }.map { Unit }
        .onSuccess {
            Log.d(TAG, "changeLanguage succeeded: languageCode=$languageCode")
        }.onFailure { error ->
            Log.e(TAG, "changeLanguage failed: languageCode=$languageCode", error)
        }
}

private suspend fun accessToken(): String? {
    return localDataSource.getAuthSession()?.accessToken
}

private suspend fun <T> authorized(block: suspend (String?) -> T): T {
    return try {
        block(accessToken())
    } catch (error: PosterApiException) {
        if (error.statusCode != 401) throw error
        Log.d(TAG, "Access token expired, refreshing for settings request")
        val currentSession = localDataSource.getAuthSession() ?: throw error
        val refreshedSession = remoteDataSource.refresh(currentSession.refreshToken).toDomain()
        localDataSource.saveAuthSession(refreshedSession.toLocal())
        block(refreshedSession.accessToken)
    }
}

private companion object {
    private const val TAG = "SettingsRepository"
}
}

```

Листинг 45 – Код файла app/src/main/java/com/example/poster/di/AppModule.kt

```
package com.example.poster.di

import android.content.Context
import com.example.poster.data.local.InMemoryPosterLocalDataSource
import com.example.poster.data.local.PosterLocalDataSource
import com.example.poster.data.remote.HttpPosterRemoteDataSource
import com.example.poster.data.remote.PosterRemoteDataSource
import com.example.poster.data.repository.AuthRepositoryImpl
import com.example.poster.data.repository.ChatRepositoryImpl
import com.example.poster.data.repository.MessageRepositoryImpl
import com.example.poster.data.repository.ProfileRepositoryImpl
import com.example.poster.data.repository.SettingsRepositoryImpl
import com.example.poster.domain.repository.AuthRepository
import com.example.poster.domain.repository.ChatRepository
import com.example.poster.domain.repository.MessageRepository
import com.example.poster.domain.repository.ProfileRepository
import com.example.poster.domain.repository.SettingsRepository
import com.example.poster.domain.usecase.observechats.ObserveChatsUseCase
import com.example.poster.domain.usecase.app.GetStartDestinationUseCase
import com.example.poster.domain.usecase.auth.CheckAuthStateUseCase
import com.example.poster.domain.usecase.auth.LogoutUseCase
import com.example.poster.domain.usecase.auth.SignInUseCase
import com.example.poster.domain.usecase.auth.SignUpUseCase
import com.example.poster.domain.usecase.auth.VerifyOtpUseCase
import com.example.poster.domain.usecase.chat.GetChatsUseCase
import com.example.poster.domain.usecase.chat.SearchChatsUseCase
import com.example.poster.domain.usecase.message.GetMessagesUseCase
import com.example.poster.domain.usecase.message.ObserveMessagesUseCase
import com.example.poster.domain.usecase.message.SendMessageUseCase
import com.example.poster.domain.usecase.message.SendMessageWithAttachmentsUseCase
import com.example.poster.domain.usecase.profile.GetMyProfileUseCase
import com.example.poster.domain.usecase.profile.GetUserProfileUseCase
import com.example.poster.domain.usecase.profile.UpdateAvatarUseCase
import com.example.poster.domain.usecase.profile.UpdateBioUseCase
import com.example.poster.domain.usecase.profile.UpdateNameUseCase
import com.example.poster.domain.usecase.profile.UpdateUsernameUseCase
import com.example.poster.domain.usecase.settings.CheckMailAccessTokenUseCase
import com.example.poster.domain.usecase.settings.GetSettingsUseCase
import com.example.poster.domain.usecase.settings.SaveMailAccessTokenUseCase
import dagger.Module
import dagger.Provides
import dagger.hilt.InstallIn
import dagger.hilt.android.qualifiers.ApplicationContext
import dagger.hilt.components.SingletonComponent
import javax.inject.Singleton

@Module
@InstallIn(SingletonComponent::class)
object AppModule {
```

```
@Provides
@Singleton
fun provideLocalDataSource(): PosterLocalDataSource = InMemoryPosterLocalDataSource()
```

```
@Provides
@Singleton
fun provideRemoteDataSource(): PosterRemoteDataSource = HttpPosterRemoteDataSource()
```

```
@Provides
@Singleton
fun provideAuthRepository(
    remoteDataSource: PosterRemoteDataSource,
    localDataSource: PosterLocalDataSource,
): AuthRepository = AuthRepositoryImpl(remoteDataSource, localDataSource)
```

```
@Provides
@Singleton
fun provideChatRepository(
    remoteDataSource: PosterRemoteDataSource,
    localDataSource: PosterLocalDataSource,
): ChatRepository = ChatRepositoryImpl(remoteDataSource, localDataSource)
```

```
@Provides
@Singleton
fun provideMessageRepository(
    remoteDataSource: PosterRemoteDataSource,
    localDataSource: PosterLocalDataSource,
): MessageRepository = MessageRepositoryImpl(remoteDataSource, localDataSource)
```

```
@Provides
@Singleton
fun provideProfileRepository(
    remoteDataSource: PosterRemoteDataSource,
    localDataSource: PosterLocalDataSource,
    @ApplicationContext context: Context,
): ProfileRepository = ProfileRepositoryImpl(remoteDataSource, localDataSource, context)
```

```
@Provides
@Singleton
fun provideSettingsRepository(
    remoteDataSource: PosterRemoteDataSource,
    localDataSource: PosterLocalDataSource,
): SettingsRepository = SettingsRepositoryImpl(remoteDataSource, localDataSource)
```

```
@Provides
fun provideObserveChatsUseCase(repository: ChatRepository): ObserveChatsUseCase =
    ObserveChatsUseCase(repository)
```

```
@Provides
fun provideSignInUseCase(repository: AuthRepository): SignInUseCase = SignInUseCase(repository)
```

@Provides

```
fun provideSignUpUseCase(repository: AuthRepository): SignUpUseCase = SignUpUseCase(repository)
```

@Provides

```
fun provideCheckAuthStateUseCase(repository: AuthRepository): CheckAuthStateUseCase =  
    CheckAuthStateUseCase(repository)
```

@Provides

```
fun provideLogoutUseCase(repository: AuthRepository): LogoutUseCase = LogoutUseCase(repository)
```

@Provides

```
fun provideVerifyOtpUseCase(repository: AuthRepository): VerifyOtpUseCase =  
    VerifyOtpUseCase(repository)
```

@Provides

```
fun provideGetStartDestinationUseCase(repository: AuthRepository): GetStartDestinationUseCase =  
    GetStartDestinationUseCase(repository)
```

@Provides

```
fun provideGetChatsUseCase(repository: ChatRepository): GetChatsUseCase = GetChatsUseCase(repository)
```

@Provides

```
fun provideSearchChatsUseCase(repository: ChatRepository): SearchChatsUseCase =  
    SearchChatsUseCase(repository)
```

@Provides

```
fun provideGetMessagesUseCase(repository: MessageRepository): GetMessagesUseCase =  
    GetMessagesUseCase(repository)
```

@Provides

```
fun provideSendMessageUseCase(repository: MessageRepository): SendMessageUseCase =  
    SendMessageUseCase(repository)
```

@Provides

```
fun provideSendMessageWithAttachmentsUseCase(repository: MessageRepository): SendMessageWithAttachmentsUseCase =  
    SendMessageWithAttachmentsUseCase(repository)
```

@Provides

```
fun provideObserveMessagesUseCase(repository: MessageRepository): ObserveMessagesUseCase =  
    ObserveMessagesUseCase(repository)
```

@Provides

```
fun provideGetMyProfileUseCase(repository: ProfileRepository): GetMyProfileUseCase =  
    GetMyProfileUseCase(repository)
```

```

@Provides
fun provideGetUserProfileUseCase(repository: ProfileRepository): GetUserProfileUseCase =
    GetUserProfileUseCase(repository)

@Provides
fun provideUpdateUsernameUseCase(repository: ProfileRepository): UpdateUsernameUseCase
=
    UpdateUsernameUseCase(repository)

@Provides
fun provideUpdateNameUseCase(repository: ProfileRepository): UpdateNameUseCase =
    UpdateNameUseCase(repository)

@Provides
fun provideUpdateAvatarUseCase(repository: ProfileRepository): UpdateAvatarUseCase =
    UpdateAvatarUseCase(repository)

@Provides
fun provideUpdateBioUseCase(repository: ProfileRepository): UpdateBioUseCase =
    UpdateBioUseCase(repository)

@Provides
fun provideGetSettingsUseCase(repository: SettingsRepository): GetSettingsUseCase =
    GetSettingsUseCase(repository)

@Provides
fun provideSaveMailAccessTokenUseCase(repository: SettingsRepository): SaveMailAccessTok
enUseCase =
    SaveMailAccessTokenUseCase(repository)

@Provides
fun provideCheckMailAccessTokenUseCase(repository: SettingsRepository): CheckMailAccess
TokenUseCase =
    CheckMailAccessTokenUseCase(repository)
}

```

Листинг 46 – Код файла `app/src/main/java/com/example/poster/domain/model/Attachment.kt`

```
package com.example.poster.domain.model

data class Attachment(
    val id: String,
    val localUri: String?,
    val remoteUrl: String?,
    val type: AttachmentType,
    val fileName: String,
    val mimeType: String,
    val sizeBytes: Long,
    val uploadStatus: AttachmentUploadStatus,
)

enum class AttachmentType {
    IMAGE,
    VIDEO,
    AUDIO,
    DOCUMENT,
    UNKNOWN,
}

enum class AttachmentUploadStatus {
    LOCAL_ONLY,
    UPLOADING,
    UPLOADED,
    FAILED,
}
```

Листинг 47 – Код файла `app/src/main/java/com/example/poster/domain/model/AuthSession.kt`

```
package com.example.poster.domain.model
```

```
data class AuthSession(  
    val accessToken: String,  
    val refreshToken: String,  
    val userId: String,  
)
```

Листинг 48 – Код файла app/src/main/java/com/example/poster/domain/model/Chat.kt

```
package com.example.poster.domain.model
```

```
data class Chat(  
    val id: String,  
    val title: String,  
    val initials: String,  
    val lastMessage: String,  
    val lastMessageTime: String,  
    val unreadCount: Int,  
)
```

Листинг 49 – Код файла `app/src/main/java/com/example/poster/domain/model/MailAccessSettings.kt`

```
package com.example.poster.domain.model
```

```
data class MailAccessSettings(  
    val token: String,  
    val smtpHost: String? = null,  
    val smtpPort: Int? = null,  
    val imapHost: String? = null,  
    val imapPort: Int? = null,  
)
```

Листинг 50 – Код файла `app/src/main/java/com/example/poster/domain/model/Message.kt`

```
package com.example.poster.domain.model

data class Message(
    val id: String,
    val chatId: String,
    val text: String,
    val time: String,
    val isMine: Boolean,
    val status: MessageStatus,
    val attachments: List<Attachment> = emptyList(),
)
```

Листинг 51 – Код файла `app/src/main/java/com/example/poster/domain/model/MessageStatus.kt`

```
package com.example.poster.domain.model
```

```
enum class MessageStatus {  
    SENDING,  
    SENT,  
    DELIVERED,  
    READ,  
    FAILED,  
}
```

Листинг 52 – Код файла app/src/main/java/com/example/poster/domain/model/Profile.kt

```
package com.example.poster.domain.model
```

```
data class Profile(  
    val id: String,  
    val name: String,  
    val username: String,  
    val email: String,  
    val bio: String,  
    val avatarUrl: String?,  
    val isOnline: Boolean,  
)
```

Листинг 53 – Код файла `app/src/main/java/com/example/poster/domain/model/StartDestination.kt`

```
package com.example.poster.domain.model

enum class StartDestination {
    AUTH,
    CHATS,
}
```

Листинг 54 – Код файла `app/src/main/java/com/example/poster/domain/model/User.kt`

```
package com.example.poster.domain.model
```

```
data class User(  
    val id: String,  
    val username: String,  
    val displayName: String,  
)
```

Листинг 55 – Код файла `app/src/main/java/com/example/poster/domain/model/UserSettings.kt`

```
package com.example.poster.domain.model
```

```
data class UserSettings(  
    val name: String,  
    val username: String,  
    val email: String,  
    val birthday: String,  
    val bio: String,  
    val language: String,  
    val hasMailAccessToken: Boolean,  
    val smtpHost: String? = null,  
    val smtpPort: Int? = null,  
    val imapHost: String? = null,  
    val imapPort: Int? = null,  
)
```

Листинг 56 – Код файла `app/src/main/java/com/example/poster/domain/repository/AuthRepository.kt`

```
package com.example.poster.domain.repository

import com.example.poster.domain.model.AuthSession

interface AuthRepository {
    suspend fun signIn(email: String, password: String): Result<Unit>
    suspend fun signUp(name: String, email: String, password: String): Result<Unit>
    suspend fun verifyOtp(email: String, code: String): Result<Unit>
    suspend fun resendOtp(email: String): Result<Unit>
    suspend fun logout(): Result<Unit>
    suspend fun isAuthorized(): Boolean
    suspend fun refreshToken(): Result<Unit>

    suspend fun getActiveSession(): AuthSession?
}
```

Листинг 57 – Код файла `app/src/main/java/com/example/poster/domain/repository/ChatRepository.kt`

```
package com.example.poster.domain.repository

import com.example.poster.domain.model.Chat
import kotlinx.coroutines.flow.Flow

interface ChatRepository {
    suspend fun getChats(): Result<List<Chat>>
    suspend fun searchChats(query: String): Result<List<Chat>>
    suspend fun syncChats(): Result<Unit>
    fun observeChats(): Flow<List<Chat>>
    suspend fun markChatAsRead(chatId: String): Result<Unit>
    suspend fun deleteChat(chatId: String): Result<Unit>
}
```

Листинг 58 – Код файла `app/src/main/java/com/example/poster/domain/repository/FileRepository.kt`

```
package com.example.poster.domain.repository

import com.example.poster.domain.model.Attachment

interface FileRepository {
    suspend fun getAttachmentInfo(uri: String): Result<Attachment>
    suspend fun copyToAppCache(uri: String): Result<Attachment>
    suspend fun saveToDownloads(attachment: Attachment): Result<Unit>
    suspend fun openAttachment(attachment: Attachment): Result<Unit>
}
```

Листинг 59 – Код файла app/src/main/java/com/example/poster/domain/repository/MessageRepository.kt

```
package com.example.poster.domain.repository

import com.example.poster.domain.model.Attachment
import com.example.poster.domain.model.Message
import kotlinx.coroutines.flow.Flow

interface MessageRepository {
    suspend fun getMessages(chatId: String): Result<List<Message>>
    fun observeMessages(chatId: String): Flow<List<Message>>
    suspend fun sendMessage(chatId: String, text: String, attachments: List<Attachment> = emptyList()): Result<Message>
    suspend fun uploadAttachment(chatId: String, attachment: Attachment): Result<Attachment>
    suspend fun downloadAttachment(attachmentId: String): Result<Attachment>
    suspend fun markMessageAsRead(messageId: String): Result<Unit>
    suspend fun deleteMessage(messageId: String): Result<Unit>
}
```

Листинг 60 – Код файла `app/src/main/java/com/example/poster/domain/repository/ProfileRepository.kt`

```
package com.example.poster.domain.repository

import com.example.poster.domain.model.Profile

interface ProfileRepository {
    suspend fun getMyProfile(): Result<Profile>
    suspend fun getUserProfile(userId: String): Result<Profile>
    suspend fun updateName(name: String): Result<Profile>
    suspend fun updateUsername(username: String): Result<Profile>
    suspend fun updateBio(bio: String): Result<Profile>
    suspend fun updateAvatar(uri: String): Result<Profile>
}
```

Листинг 61 – Код файла `app/src/main/java/com/example/poster/domain/repository/SettingsRepository.kt`

```
package com.example.poster.domain.repository

import com.example.poster.domain.model.UserSettings
import com.example.poster.domain.model.MailAccessSettings

interface SettingsRepository {
    suspend fun getSettings(): Result<UserSettings>
    suspend fun saveMailAccessToken(settings: MailAccessSettings): Result<Unit>
    suspend fun hasMailAccessToken(): Boolean
    suspend fun deleteMailAccessToken(): Result<Unit>
    suspend fun changeLanguage(languageCode: String): Result<Unit>
}
```

Листинг 62 – Код файла `app/src/main/java/com/example/poster/domain/usecase/app/GetStartDestinationUseCase.kt`

```
package com.example.poster.domain.usecase.app

import com.example.poster.domain.model.StartDestination
import com.example.poster.domain.repository.AuthRepository

class GetStartDestinationUseCase(
    private val authRepository: AuthRepository,
) {
    suspend operator fun invoke(): StartDestination {
        return if (authRepository.isAuthenticated()) {
            StartDestination.CHATS
        } else {
            StartDestination.AUTH
        }
    }
}
```

Листинг 63 – Код файла `app/src/main/java/com/example/poster/domain/usecase/auth/CheckAuthStateUseCase.kt`

```
package com.example.poster.domain.usecase.auth

import com.example.poster.domain.repository.AuthRepository

class CheckAuthStateUseCase(
    private val authRepository: AuthRepository,
) {
    suspend operator fun invoke(): Boolean {
        return authRepository.isAuthenticated()
    }
}
```

Листинг 64 – Код файла `app/src/main/java/com/example/poster/domain/usecase/auth/LogoutUseCase.kt`

```
package com.example.poster.domain.usecase.auth

import com.example.poster.domain.repository.AuthRepository

class LogoutUseCase(
    private val authRepository: AuthRepository,
) {
    suspend operator fun invoke(): Result<Unit> {
        return authRepository.logout()
    }
}
```

Листинг 65 – Код файла app/src/main/java/com/example/poster/domain/usecase/auth/SignInUseCase.kt

```
package com.example.poster.domain.usecase.auth

import com.example.poster.domain.repository.AuthRepository

class SignInUseCase(
    private val authRepository: AuthRepository,
) {
    suspend operator fun invoke(email: String, password: String): Result<Unit> {
        if (email.isBlank()) {
            return Result.failure(IllegalArgumentException("Email is empty"))
        }
        if (!email.contains("@")) {
            return Result.failure(IllegalArgumentException("Invalid email"))
        }
        if (password.isBlank()) {
            return Result.failure(IllegalArgumentException("Password is empty"))
        }

        return authRepository.signIn(email, password)
    }
}
```

Листинг 66 – Код файла app/src/main/java/com/example/poster/domain/usecase/auth/SignUpUseCase.kt

```
package com.example.poster.domain.usecase.auth

import com.example.poster.domain.repository.AuthRepository

class SignUpUseCase(
    private val authRepository: AuthRepository,
) {
    suspend operator fun invoke(name: String, email: String, password: String): Result<Unit> {
        if (name.isBlank()) {
            return Result.failure(IllegalArgumentException("Name is empty"))
        }
        if (email.isBlank()) {
            return Result.failure(IllegalArgumentException("Email is empty"))
        }
        if (!email.contains("@")) {
            return Result.failure(IllegalArgumentException("Invalid email"))
        }
        if (password.isBlank()) {
            return Result.failure(IllegalArgumentException("Password is empty"))
        }

        return authRepository.signUp(name, email, password)
    }
}
```

Листинг 67 – Код файла `app/src/main/java/com/example/poster/domain/usecase/auth/VerifyOtpUseCase.kt`

```
package com.example.poster.domain.usecase.auth

import com.example.poster.domain.repository.AuthRepository

class VerifyOtpUseCase(
    private val authRepository: AuthRepository,
) {
    suspend operator fun invoke(email: String, code: String): Result<Unit> {
        if (email.isBlank()) {
            return Result.failure(IllegalArgumentException("Email is empty"))
        }
        if (code.length != 6 || code.any { !it.isDigit() }) {
            return Result.failure(IllegalArgumentException("Invalid OTP code"))
        }

        return authRepository.verifyOtp(email, code)
    }
}
```

Листинг 68 – Код файла `app/src/main/java/com/example/poster/domain/usecase/chat/GetChatsUseCase.kt`

```
package com.example.poster.domain.usecase.chat

import com.example.poster.domain.model.Chat
import com.example.poster.domain.repository.ChatRepository

class GetChatsUseCase(
    private val chatRepository: ChatRepository,
) {
    suspend operator fun invoke(): Result<List<Chat>> {
        return chatRepository.getChats()
    }
}
```

Листинг 69 – Код файла `app/src/main/java/com/example/poster/domain/usecase/chat/SearchChatsUseCase.kt`

```
package com.example.poster.domain.usecase.chat

import com.example.poster.domain.model.Chat
import com.example.poster.domain.repository.ChatRepository

class SearchChatsUseCase(
    private val chatRepository: ChatRepository,
) {
    suspend operator fun invoke(query: String): Result<List<Chat>> {
        return chatRepository.searchChats(query)
    }
}
```

Листинг 70 – Код файла `app/src/main/java/com/example/poster/domain/usecase/file/GetAttachmentInfoUseCase.kt`

```
package com.example.poster.domain.usecase.file

import com.example.poster.domain.model.Attachment
import com.example.poster.domain.repository.FileRepository

class GetAttachmentInfoUseCase(
    private val fileRepository: FileRepository,
) {
    suspend operator fun invoke(uri: String): Result<Attachment> {
        if (uri.isBlank()) {
            return Result.failure(IllegalArgumentException("Uri is empty"))
        }

        return fileRepository.getAttachmentInfo(uri)
    }
}
```

Листинг 71 – Код файла `app/src/main/java/com/example/poster/domain/usecase/file/OpenAttachmentUseCase.kt`

```
package com.example.poster.domain.usecase.file

import com.example.poster.domain.model.Attachment
import com.example.poster.domain.repository.FileRepository

class OpenAttachmentUseCase(
    private val fileRepository: FileRepository,
) {
    suspend operator fun invoke(attachment: Attachment): Result<Unit> {
        return fileRepository.openAttachment(attachment)
    }
}
```

Листинг 72 – Код файла `app/src/main/java/com/example/poster/domain/usecase/file/SaveAttachmentToDownloadsUseCase.kt`

```
package com.example.poster.domain.usecase.file

import com.example.poster.domain.model.Attachment
import com.example.poster.domain.repository.FileRepository

class SaveAttachmentToDownloadsUseCase(
    private val fileRepository: FileRepository,
) {
    suspend operator fun invoke(attachment: Attachment): Result<Unit> {
        return fileRepository.saveToDownloads(attachment)
    }
}
```

Листинг 73 – Код файла `app/src/main/java/com/example/poster/domain/usecase/message/DownloadAttachmentUseCase.kt`

```
package com.example.poster.domain.usecase.message

import com.example.poster.domain.model.Attachment
import com.example.poster.domain.repository.MessageRepository

class DownloadAttachmentUseCase(
    private val messageRepository: MessageRepository,
) {
    suspend operator fun invoke(attachmentId: String): Result<Attachment> {
        if (attachmentId.isBlank()) {
            return Result.failure(IllegalArgumentException("Attachment id is empty"))
        }

        return messageRepository.downloadAttachment(attachmentId)
    }
}
```

Листинг 74 – Код файла `app/src/main/java/com/example/poster/domain/usecase/message/GetMessagesUseCase.kt`

```
package com.example.poster.domain.usecase.message

import com.example.poster.domain.model.Message
import com.example.poster.domain.repository.MessageRepository

class GetMessagesUseCase(
    private val messageRepository: MessageRepository,
) {
    suspend operator fun invoke(chatId: String): Result<List<Message>> {
        if (chatId.isBlank()) {
            return Result.failure(IllegalArgumentException("Chat id is empty"))
        }

        return messageRepository.getMessages(chatId)
    }
}
```

Листинг 75 – Код файла `app/src/main/java/com/example/poster/domain/usecase/message/ObserveMessagesUseCase.kt`

```
package com.example.poster.domain.usecase.message

import com.example.poster.domain.model.Message
import com.example.poster.domain.repository.MessageRepository
import kotlinx.coroutines.flow.Flow

class ObserveMessagesUseCase(
    private val messageRepository: MessageRepository,
) {
    operator fun invoke(chatId: String): Flow<List<Message>> {
        return messageRepository.observeMessages(chatId)
    }
}
```

Листинг 76 – Код файла `app/src/main/java/com/example/poster/domain/usecase/message/SendMessageUseCase.kt`

```
package com.example.poster.domain.usecase.message

import com.example.poster.domain.model.Message
import com.example.poster.domain.repository.MessageRepository

class SendMessageUseCase(
    private val messageRepository: MessageRepository,
) {
    suspend operator fun invoke(chatId: String, text: String): Result<Message> {
        if (chatId.isBlank()) {
            return Result.failure(IllegalArgumentException("Chat id is empty"))
        }
        if (text.isBlank()) {
            return Result.failure(IllegalArgumentException("Message is empty"))
        }

        return messageRepository.sendMessage(chatId, text.trim())
    }
}
```

Листинг 77 – Код файла app/src/main/java/com/example/poster/domain/usecase/message/SendMessageWithAttachmentsUseCase.kt

```
package com.example.poster.domain.usecase.message

import com.example.poster.domain.model.Attachment
import com.example.poster.domain.model.Message
import com.example.poster.domain.repository.MessageRepository

class SendMessageWithAttachmentsUseCase(
    private val messageRepository: MessageRepository,
) {
    suspend operator fun invoke(
        chatId: String,
        text: String,
        attachments: List<Attachment>,
    ): Result<Message> {
        if (chatId.isBlank()) {
            return Result.failure(IllegalArgumentException("Chat id is empty"))
        }
        if (text.isBlank() && attachments.isEmpty()) {
            return Result.failure(IllegalArgumentException("Message cannot be empty"))
        }

        return messageRepository.sendMessage(
            chatId = chatId,
            text = text.trim(),
            attachments = attachments,
        )
    }
}
```

Листинг 78 – Код файла app/src/main/java/com/example/poster/domain/usecase/message/UploadAttachmentUseCase.kt

```
package com.example.poster.domain.usecase.message

import com.example.poster.domain.model.Attachment
import com.example.poster.domain.repository.MessageRepository

class UploadAttachmentUseCase(
    private val messageRepository: MessageRepository,
) {
    suspend operator fun invoke(chatId: String, attachment: Attachment): Result<Attachment> {
        if (chatId.isBlank()) {
            return Result.failure(IllegalArgumentException("Chat id is empty"))
        }

        return messageRepository.uploadAttachment(chatId, attachment)
    }
}
```

Листинг 79 – Код файла `app/src/main/java/com/example/poster/domain/usecase/PosterUseCases.kt`

```
package com.example.poster.domain.usecase

import com.example.poster.domain.model.Attachment
import com.example.poster.domain.model.AuthSession
import com.example.poster.domain.model.Chat
import com.example.poster.domain.model.Message
import com.example.poster.domain.model.Profile
import com.example.poster.domain.repository.AuthRepository
import com.example.poster.domain.repository.ChatRepository
import com.example.poster.domain.repository.MessageRepository
import com.example.poster.domain.repository.ProfileRepository
import kotlinx.coroutines.flow.Flow

class LoginUseCase(
    private val authRepository: AuthRepository,
) {
    suspend operator fun invoke(identifier: String, password: String): AuthSession {
        authRepository.signIn(identifier, password).getOrThrow()
        return authRepository.getActiveSession()
            ?: throw IllegalStateException("No active session after login")
    }
}

class RegisterUseCase(
    private val authRepository: AuthRepository,
) {
    suspend operator fun invoke(identifier: String, password: String, displayName: String): AuthSession {
        authRepository.signUp(displayName, identifier, password).getOrThrow()
        return authRepository.getActiveSession()
            ?: throw IllegalStateException("No active session after registration")
    }
}

class VerifyOtpUseCase(
    private val authRepository: AuthRepository,
) {
    suspend operator fun invoke(code: String): AuthSession {
        authRepository.verifyOtp(email = "", code = code).getOrThrow()
        return authRepository.getActiveSession()
            ?: throw IllegalStateException("No active session after OTP verification")
    }
}

class ObserveChatsUseCase(
    private val chatRepository: ChatRepository,
) {
    operator fun invoke(): Flow<List<Chat>> {
```

```

        return chatRepository.observeChats()
    }
}

class ObserveMessagesUseCase(
    private val messageRepository: MessageRepository,
) {
    operator fun invoke(chatId: String): Flow<List<Message>> {
        return messageRepository.observeMessages(chatId)
    }
}

class SendMessageUseCase(
    private val messageRepository: MessageRepository,
) {
    suspend operator fun invoke(chatId: String, text: String, attachments: List<Attachment> = emptyList()) {
        messageRepository.sendMessage(chatId, text, attachments).getOrThrow()
    }
}

class GetProfileUseCase(
    private val profileRepository: ProfileRepository,
) {
    suspend operator fun invoke(userId: String): Profile {
        return profileRepository.getUserProfile(userId).getOrThrow()
    }
}

class UpdateProfileUseCase(
    private val profileRepository: ProfileRepository,
) {
    suspend operator fun invoke(profile: Profile): Profile {
        return profileRepository.updateBio(profile.bio).getOrThrow()
    }
}

```

Листинг 80 – Код файла `app/src/main/java/com/example/poster/domain/usecase/profile/GetMyProfileUseCase.kt`

```
package com.example.poster.domain.usecase.profile

import com.example.poster.domain.model.Profile
import com.example.poster.domain.repository.ProfileRepository

class GetMyProfileUseCase(
    private val profileRepository: ProfileRepository,
) {
    suspend operator fun invoke(): Result<Profile> {
        return profileRepository.getMyProfile()
    }
}
```

Листинг 81 – Код файла app/src/main/java/com/example/poster/domain/usecase/profile/GetUserProfileUseCase.kt

```
package com.example.poster.domain.usecase.profile

import com.example.poster.domain.model.Profile
import com.example.poster.domain.repository.ProfileRepository

class GetUserProfileUseCase(
    private val profileRepository: ProfileRepository,
) {
    suspend operator fun invoke(userId: String): Result<Profile> {
        if (userId.isBlank()) {
            return Result.failure(IllegalArgumentException("User id is empty"))
        }

        return profileRepository.getUserProfile(userId)
    }
}
```

Листинг 82 – Код файла app/src/main/java/com/example/poster/domain/usecase/profile/UpdateAvatarUseCase.kt

```
package com.example.poster.domain.usecase.profile

import com.example.poster.domain.model.Profile
import com.example.poster.domain.repository.ProfileRepository

class UpdateAvatarUseCase(
    private val profileRepository: ProfileRepository,
) {
    suspend operator fun invoke(uri: String): Result<Profile> {
        if (uri.isBlank()) {
            return Result.failure(IllegalArgumentException("Avatar uri is empty"))
        }

        return profileRepository.updateAvatar(uri)
    }
}
```

Листинг 83 – Код файла `app/src/main/java/com/example/poster/domain/usecase/profile/UpdateBioUseCase.kt`

```
package com.example.poster.domain.usecase.profile

import com.example.poster.domain.model.Profile
import com.example.poster.domain.repository.ProfileRepository

class UpdateBioUseCase(
    private val profileRepository: ProfileRepository,
) {
    suspend operator fun invoke(bio: String): Result<Profile> {
        return profileRepository.updateBio(bio)
    }
}
```

Листинг 84 – Код файла `app/src/main/java/com/example/poster/domain/usecase/profile/UpdateNameUseCase.kt`

```
package com.example.poster.domain.usecase.profile

import com.example.poster.domain.model.Profile
import com.example.poster.domain.repository.ProfileRepository

class UpdateNameUseCase(
    private val profileRepository: ProfileRepository,
) {
    suspend operator fun invoke(name: String): Result<Profile> {
        if (name.isBlank()) {
            return Result.failure(IllegalArgumentException("Name is empty"))
        }

        return profileRepository.updateName(name.trim())
    }
}
```

Листинг 85 – Код файла `app/src/main/java/com/example/poster/domain/usecase/profile/UpdateUsernameUseCase.kt`

```
package com.example.poster.domain.usecase.profile

import com.example.poster.domain.model.Profile
import com.example.poster.domain.repository.ProfileRepository

class UpdateUsernameUseCase(
    private val profileRepository: ProfileRepository,
) {
    suspend operator fun invoke(username: String): Result<Profile> {
        if (username.isBlank()) {
            return Result.failure(IllegalArgumentException("Username is empty"))
        }

        return profileRepository.updateUsername(username)
    }
}
```

Листинг 86 – Код файла `app/src/main/java/com/example/poster/domain/usecase/settings/CheckMailAccessTokenUseCase.kt`

```
package com.example.poster.domain.usecase.settings

import com.example.poster.domain.repository.SettingsRepository

class CheckMailAccessTokenUseCase(
    private val settingsRepository: SettingsRepository,
) {
    suspend operator fun invoke(): Boolean {
        return settingsRepository.hasMailAccessToken()
    }
}
```

Листинг 87 – Код файла `app/src/main/java/com/example/poster/domain/usecase/settings/GetSettingsUseCase.kt`

```
package com.example.poster.domain.usecase.settings

import com.example.poster.domain.model.UserSettings
import com.example.poster.domain.repository.SettingsRepository

class GetSettingsUseCase(
    private val settingsRepository: SettingsRepository,
) {
    suspend operator fun invoke(): Result<UserSettings> {
        return settingsRepository.getSettings()
    }
}
```

Листинг 88 – Код файла app/src/main/java/com/example/poster/domain/usecase/settings/SaveMailAccessTokenUseCase.kt

```
package com.example.poster.domain.usecase.settings

import com.example.poster.domain.model.MailAccessSettings
import com.example.poster.domain.repository.SettingsRepository

class SaveMailAccessTokenUseCase(
    private val settingsRepository: SettingsRepository,
) {
    suspend operator fun invoke(settings: MailAccessSettings): Result<Unit> {
        if (settings.token.isBlank()) {
            return Result.failure(IllegalArgumentException("Access token is empty"))
        }
        if (settings.smtpPort != null && settings.smtpPort !in 1..65535) {
            return Result.failure(IllegalArgumentException("SMTP port is invalid"))
        }
        if (settings.imapPort != null && settings.imapPort !in 1..65535) {
            return Result.failure(IllegalArgumentException("IMAP port is invalid"))
        }

        return settingsRepository.saveMailAccessToken(
            settings.copy(
                token = settings.token.trim(),
                smtpHost = settings.smtpHost?.trim()?.ifBlank { null },
                imapHost = settings.imapHost?.trim()?.ifBlank { null },
            )
        )
    }
}
```

Листинг 89 – Код файла app/src/main/java/com/example/poster/MainActivity.kt

```
package com.example.poster

import android.content.Context
import android.content.Intent
import android.net.Uri
import android.os.Bundle
import android.provider.OpenableColumns
import android.util.Log
import androidx.activity.compose.rememberLauncherForActivityResult
import androidx.activity.result.contract.ActivityResultContracts
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.compose.animation.AnimatedContent
import androidx.compose.animation.ExperimentalAnimationApi
import androidx.compose.animation.core.CubicBezierEasing
import androidx.compose.animation.core.tween
import androidx.compose.animation.fadeIn
import androidx.compose.animation.fadeOut
import androidx.compose.animation.scaleIn
import androidx.compose.animation.scaleOut
import androidx.compose.animation.togetherWith
import androidx.compose.foundation.background
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.runtime.CompositionLocalProvider
import androidx.compose.runtime.LaunchedEffect
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.rememberCoroutineScope
import androidx.compose.runtime.saveable.rememberSaveable
import androidx.compose.runtime.setValue
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.platform.LocalDensity
import androidx.compose.ui.unit.Density
import com.example.poster.domain.model.Attachment
import com.example.poster.domain.model.AttachmentType
import com.example.poster.domain.model.AttachmentUploadStatus
import com.example.poster.domain.model.Profile
import com.example.poster.domain.repository.AuthRepository
import com.example.poster.domain.repository.SettingsRepository
import com.example.poster.domain.usecase.ObserveChatsUseCase
import com.example.poster.domain.usecase.auth.SignInUseCase
import com.example.poster.domain.usecase.auth.SignUpUseCase
import com.example.poster.domain.usecase.auth.VerifyOtpUseCase
import com.example.poster.domain.usecase.chat.GetChatsUseCase
import com.example.poster.domain.usecase.message.GetMessagesUseCase
import com.example.poster.domain.usecase.message.ObserveMessagesUseCase
```

```

import com.example.poster.domain.usecase.message.SendMessageUseCase
import com.example.poster.domain.usecase.message.SendMessageWithAttachmentsUseCase
import com.example.poster.domain.usecase.profile.GetMyProfileUseCase
import com.example.poster.domain.usecase.profile.UpdateAvatarUseCase
import com.example.poster.domain.usecase.profile.UpdateBioUseCase
import com.example.poster.domain.usecase.profile.UpdateNameUseCase
import com.example.poster.domain.usecase.profile.UpdateUsernameUseCase
import com.example.poster.domain.usecase.settings.CheckMailAccessTokenUseCase
import com.example.poster.domain.usecase.settings.GetSettingsUseCase
import com.example.poster.domain.usecase.settings.SaveMailAccessTokenUseCase
import com.example.poster.presentation.auth.otp.VerifyEmailScreen
import com.example.poster.presentation.auth.register.RegisterScreen
import com.example.poster.presentation.auth.signin.SignInScreen
import com.example.poster.presentation.chats.ChatListScreen
import com.example.poster.presentation.chats.ChatPreviewUi
import com.example.poster.presentation.chats.toChatPreviewUi
import com.example.poster.presentation.messages.MessageContentType
import com.example.poster.presentation.messages.MessageUi
import com.example.poster.presentation.messages.MessagesScreen
import com.example.poster.presentation.messages.toMessageUi
import com.example.poster.presentation.profile.MyProfileScreen
import com.example.poster.presentation.profile.ProfileFileUi
import com.example.poster.presentation.profile.ProfileScreen
import com.example.poster.presentation.settings.AccessTokenGuideScreen
import com.example.poster.presentation.settings.AccessTokenSettingsScreen
import com.example.poster.presentation.settings.LanguageSettingsScreen
import com.example.poster.presentation.settings.PrivacySettingsScreen
import com.example.poster.presentation.settings.PrivacySettingsUi
import com.example.poster.presentation.settings.SettingsScreen
import com.example.poster.ui.theme.PosterTheme
import dagger.hilt.android.AndroidEntryPoint
import kotlinx.coroutines.flow.collect
import kotlinx.coroutines.launch
import javax.inject.Inject

```

```

@AndroidEntryPoint
class MainActivity : ComponentActivity() {
    @Inject
    lateinit var authRepository: AuthRepository

    @Inject
    lateinit var settingsRepository: SettingsRepository

    @Inject
    lateinit var observeChatsUseCase: ObserveChatsUseCase

    @Inject
    lateinit var signInUseCase: SignInUseCase

    @Inject
    lateinit var signUpUseCase: SignUpUseCase

```

```
@Inject
lateinit var verifyOtpUseCase: VerifyOtpUseCase
```

```
@Inject
lateinit var getChatsUseCase: GetChatsUseCase
```

```
@Inject
lateinit var getMessagesUseCase: GetMessagesUseCase
```

```
@Inject
lateinit var observeMessagesUseCase: ObserveMessagesUseCase
```

```
@Inject
lateinit var sendMessageUseCase: SendMessageUseCase
```

```
@Inject
lateinit var sendMessageWithAttachmentsUseCase: SendMessageWithAttachmentsUseCase
```

```
@Inject
lateinit var getMyProfileUseCase: GetMyProfileUseCase
```

```
@Inject
lateinit var updateAvatarUseCase: UpdateAvatarUseCase
```

```
@Inject
lateinit var updateNameUseCase: UpdateNameUseCase
```

```
@Inject
lateinit var updateUsernameUseCase: UpdateUsernameUseCase
```

```
@Inject
lateinit var updateBioUseCase: UpdateBioUseCase
```

```
@Inject
lateinit var getSettingsUseCase: GetSettingsUseCase
```

```
@Inject
lateinit var saveMailAccessTokenUseCase: SaveMailAccessTokenUseCase
```

```
@Inject
lateinit var checkMailAccessTokenUseCase: CheckMailAccessTokenUseCase
```

```
companion object {
    private const val TAG = "PosterMainActivity"
    private const val UI_SCALE = 0.85f
}
```

```
@OptIn(ExperimentalAnimationApi::class)
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
```

```
Log.d(TAG, "onCreate: app started")
enableEdgeToEdge()
```

```
setContent {
    PosterTheme {
        val baseDensity = LocalDensity.current

        CompositionLocalProvider(
            LocalDensity provides Density(
                density = baseDensity.density * UI_SCALE,
                fontScale = baseDensity.fontScale,
            )
        ) {
            /*
             * PLACEHOLDER AUTH LOGIC
             *
             * Сейчас всегда false.
             * Поэтому приложение по дефолту открывает Sign In / Sign Up.
             *
             * Потом здесь будет проверка:
             * - есть ли JWT access token
             * - не истёк ли токен
             * - нужно ли отправлять пользователя на Login/Register
             */
            val isAuthorized = false
            val scope = rememberCoroutineScope()

            var currentScreen by rememberSaveable {
                mutableStateOf(
                    if (isAuthorized) {
                        AppScreen.CHATS
                    } else {
                        AppScreen.SIGN_IN
                    }
                )
            }

            var verificationEmail by rememberSaveable {
                mutableStateOf("user@mail.ru")
            }

            var selectedChat by remember {
                mutableStateOf<ChatPreviewUi?>(null)
            }

            var chats by remember {
                mutableStateOf<List<ChatPreviewUi>>(emptyList())
            }

            var messages by remember {
                mutableStateOf<List<MessageUi>>(emptyList())
            }

            var myProfile by remember {
                mutableStateOf<Profile?>(null)
            }
        }
    }
}
```

```

    }
    var accessTokenBackScreen by rememberSaveable {
        mutableStateOf(AppScreen.CHATS)
    }
    var myProfileBackScreen by rememberSaveable {
        mutableStateOf(AppScreen.SETTINGS)
    }
    var hasMailAccessToken by rememberSaveable {
        mutableStateOf(false)
    }
    var settingsEmail by rememberSaveable {
        mutableStateOf("your.email@example.com")
    }
    var settingsBirthday by rememberSaveable {
        mutableStateOf("January 1, 2000")
    }
    var settingsLanguage by rememberSaveable {
        mutableStateOf("English")
    }
    var smtpHost by rememberSaveable {
        mutableStateOf("")
    }
    var smtpPort by rememberSaveable {
        mutableStateOf("")
    }
    var imapHost by rememberSaveable {
        mutableStateOf("")
    }
    var imapPort by rememberSaveable {
        mutableStateOf("")
    }
    var showOnlineStatus by rememberSaveable {
        mutableStateOf(true)
    }
    var sendReadReceipts by rememberSaveable {
        mutableStateOf(true)
    }
    var confirmBeforeOpeningFiles by rememberSaveable {
        mutableStateOf(true)
    }
    val avatarPickerLauncher = rememberLauncherForActivityResult(
        contract = ActivityResultContracts.OpenDocument(),
    ) { uri ->
        uri ?: return@rememberLauncherForActivityResult

        runCatching {
            contentResolver.takePersistableUriPermission(
                uri,
                Intent.FLAG_GRANT_READ_URI_PERMISSION,
            )
        }
    }

```

```

scope.launch {
    updateAvatarUseCase(uri.toString())
        .onSuccess { updatedProfile ->
            myProfile = updatedProfile
        }
        .onFailure { error ->
            Log.e(TAG, "Failed to update avatar", error)
        }
    }
}

LaunchedEffect(Unit) {
    Log.d(TAG, "Compose content loaded")
    Log.d(TAG, "Auth placeholder check started")
    Log.d(TAG, "isAuthorized = $isAuthorized")

    observeChatsUseCase().collect { loadedChats ->
        chats = loadedChats.map { chat -> chat.toChatPreviewUi() }
    }
}

LaunchedEffect(Unit) {
    hasMailAccessToken = checkMailAccessTokenUseCase()

    getMyProfileUseCase()
        .onSuccess { profile ->
            myProfile = profile
            if (settingsEmail == "your.email@example.com") {
                settingsEmail = profile.email
            }
        }
        .onFailure { error ->
            Log.e(TAG, "Failed to load my profile", error)
        }
    }

    getSettingsUseCase()
        .onSuccess { settings ->
            hasMailAccessToken = settings.hasMailAccessToken
            settingsLanguage = settings.language
            smtpHost = settings.smtpHost.orEmpty()
            smtpPort = settings.smtpPort?.toString().orEmpty()
            imapHost = settings.imapHost.orEmpty()
            imapPort = settings.imapPort?.toString().orEmpty()
        }
        .onFailure { error ->
            Log.e(TAG, "Failed to load settings", error)
        }
    }
}

LaunchedEffect(currentScreen) {

```

```

        Log.d(TAG, "Current screen = $currentScreen")
    }

    LaunchedEffect(selectedChat?.id) {
        val chatId = selectedChat?.id ?: return@LaunchedEffect
        messages = emptyList()
        getMessagesUseCase(chatId)
            .onFailure { error ->
                Log.e(TAG, "Failed to load messages", error)
            }

        observeMessagesUseCase(chatId).collect { loadedMessages ->
            messages = loadedMessages.map { message -> message.toMessageUi() }
        }
    }

    AnimatedContent(
        targetState = currentScreen,
        modifier = Modifier
            .fillMaxSize()
            .background(Color(0xFF030306)),
        transitionSpec = {
            val enterTransition = scaleIn(
                initialScale = 0.94f,
                animationSpec = tween(
                    durationMillis = ScreenEnterDurationMillis,
                    easing = ScreenEnterEasing,
                ),
            ) + fadeIn(
                initialAlpha = 0.28f,
                animationSpec = tween(
                    durationMillis = ScreenEnterDurationMillis,
                    easing = ScreenEnterEasing,
                ),
            )
            val exitTransition = scaleOut(
                targetScale = 1.04f,
                animationSpec = tween(
                    durationMillis = ScreenExitDurationMillis,
                    easing = ScreenExitEasing,
                ),
            ) + fadeOut(
                animationSpec = tween(
                    durationMillis = ScreenExitDurationMillis,
                    easing = ScreenExitEasing,
                ),
            )

            enterTransition.togetherWith(exitTransition)
        },
        label = "posterScreenTransition",
    )

```

```

) { screen ->
when (screen) {
  AppScreen.SIGN_IN -> {
    SignInScreen(
      onSignInClick = { email, password ->
        Log.d(TAG, "Sign In clicked: email=$email")
        scope.launch {
          signInUseCase(email, password)
            .onSuccess {
              getChatsUseCase()
              getMyProfileUseCase()
                .onSuccess { profile -> myProfile = profile }
              hasMailAccessToken = checkMailAccessTokenUseCase()
              currentScreen = AppScreen.CHATS
            }
            .onFailure { error ->
              Log.e(TAG, "Sign in failed", error)
            }
          }
        },
      onSignUpClick = {
        Log.d(TAG, "Navigate to Sign Up")
        currentScreen = AppScreen.SIGN_UP
      },
    )
  }

  AppScreen.SIGN_UP -> {
    RegisterScreen(
      onContinueClick = { name, email, password ->
        Log.d(TAG, "Sign Up clicked: name=$name, email=$email")
        scope.launch {
          signUpUseCase(name, email, password)
            .onSuccess {
              verificationEmail = email
              Log.d(TAG, "Opening VerifyEmailScreen for $verificationEmail")
              currentScreen = AppScreen.VERIFY_EMAIL
            }
            .onFailure { error ->
              Log.e(TAG, "Sign up failed", error)
            }
          }
        },
      onSignInClick = {
        Log.d(TAG, "Navigate to Sign In")
        currentScreen = AppScreen.SIGN_IN
      },
    )
  }

  AppScreen.VERIFY_EMAIL -> {

```

```

VerifyEmailScreen(
  email = verificationEmail,
  onVerifyClick = { code ->
    Log.d(TAG, "Verify code clicked: codeLength=${code.length}")
    scope.launch {
      verifyOtpUseCase(verificationEmail, code)
      .onSuccess {
        getChatsUseCase()
        getMyProfileUseCase()
        .onSuccess { profile -> myProfile = profile }
        hasMailAccessToken = checkMailAccessTokenUseCase()
        currentScreen = AppScreen.CHATS
      }
      .onFailure { error ->
        Log.e(TAG, "OTP verification failed", error)
      }
    }
  },
  onResendClick = {
    Log.d(TAG, "Resend code clicked for $verificationEmail")
    scope.launch {
      authRepository.resendOtp(verificationEmail)
      .onFailure { error ->
        Log.e(TAG, "Resend OTP failed", error)
      }
    }
  },
)
}

AppScreen.CHATS -> {
  ChatListScreen(
    chats = chats,
    isSetupRequired = !hasMailAccessToken,
    profileAvatarUrl = myProfile?.avatarUrl,
    hasMailAccessToken = hasMailAccessToken,
    onChatClick = { chat ->
      if (hasMailAccessToken) {
        Log.d(TAG, "Chat clicked: id=${chat.id}, name=${chat.name}")
        selectedChat = chat
        currentScreen = AppScreen.MESSAGES
      } else {
        Log.d(TAG, "Chat blocked, mail access token is missing")
      }
    },
    onSetupTokenClick = {
      Log.d(TAG, "Setup token clicked")
      accessTokenBackScreen = AppScreen.CHATS
      currentScreen = AppScreen.ACCESS_TOKEN_SETTINGS
    },
    onSettingsClick = {

```

```

        Log.d(TAG, "Settings clicked")
        currentScreen = AppScreen.SETTINGS
    },
    onProfileClick = {
        Log.d(TAG, "Open MyProfile from Chats")
        myProfileBackScreen = AppScreen.CHATS
        currentScreen = AppScreen.MY_PROFILE
    },
)
}

```

```

AppScreen.SETTINGS -> {
    val profile = myProfile

```

```

SettingsScreen(
    name = profile?.name ?: "Your Name",
    username = profile?.username ?: "@your.username",
    email = settingsEmail,
    birthday = settingsBirthday,
    bio = profile?.bio ?: "Hey there! I'm using Poster.",
    language = settingsLanguage,
    accessTokenStatus = if (hasMailAccessToken) {
        "Configured"
    } else {
        "Not configured"
    },
    onBackClick = {
        Log.d(TAG, "Back from Settings")
        currentScreen = AppScreen.CHATS
    },
    onAccessTokenClick = {
        Log.d(TAG, "Open Access Token Settings")
        accessTokenBackScreen = AppScreen.SETTINGS
        currentScreen = AppScreen.ACCESS_TOKEN_SETTINGS
    },
    onPrivacyClick = {
        Log.d(TAG, "Privacy & Security clicked")
        currentScreen = AppScreen.PRIVACY_SETTINGS
    },
    onLanguageClick = {
        Log.d(TAG, "Language clicked")
        currentScreen = AppScreen.LANGUAGE_SETTINGS
    },
    onProfilePhotoClick = {
        Log.d(TAG, "Profile photo clicked")
        avatarPickerLauncher.launch(arrayOf("image/*"))
    },
    onAccountValueSave = { itemId, value ->
        Log.d(TAG, "Save setting $itemId inline: $value")
        when (EditableSetting.fromId(itemId)) {
            EditableSetting.NAME -> {

```

```

        scope.launch {
            updateNameUseCase(value)
                .onSuccess { updatedProfile ->
                    myProfile = updatedProfile
                }
                .onFailure { error ->
                    Log.e(TAG, "Failed to update name", error)
                }
        }
    }

    EditableSetting.USERNAME -> {
        val username = value.withUsernamePrefix()
        scope.launch {
            updateUsernameUseCase(username)
                .onSuccess { updatedProfile ->
                    myProfile = updatedProfile
                }
                .onFailure { error ->
                    Log.e(TAG, "Failed to update username", error)
                }
        }
    }

    EditableSetting.EMAIL -> {
        settingsEmail = value
    }

    EditableSetting.BIRTHDAY -> {
        settingsBirthday = value
    }

    EditableSetting.BIO -> {
        scope.launch {
            updateBioUseCase(value)
                .onSuccess { updatedProfile ->
                    myProfile = updatedProfile
                }
                .onFailure { error ->
                    Log.e(TAG, "Failed to update bio", error)
                }
        }
    }

    null -> Unit
}
},
)
}

AppScreen.LANGUAGE_SETTINGS -> {

```

```

LanguageSettingsScreen(
    currentLanguage = settingsLanguage,
    onBackClick = {
        currentScreen = AppScreen.SETTINGS
    },
    onSaveLanguageClick = { language ->
        Log.d(TAG, "Language saved: $language")
        settingsLanguage = language
        scope.launch {
            settingsRepository.changeLanguage(language)
                .onFailure { error ->
                    Log.e(TAG, "Failed to save language", error)
                }
        }
        currentScreen = AppScreen.SETTINGS
    },
)
}

```

```

AppScreen.PRIVACY_SETTINGS -> {
    PrivacySettingsScreen(
        settings = PrivacySettingsUi(
            showOnlineStatus = showOnlineStatus,
            sendReadReceipts = sendReadReceipts,
            confirmBeforeOpeningFiles = confirmBeforeOpeningFiles,
        ),
        onBackClick = {
            currentScreen = AppScreen.SETTINGS
        },
        onSavePrivacyClick = { settings ->
            Log.d(TAG, "Privacy settings saved: $settings")
            showOnlineStatus = settings.showOnlineStatus
            sendReadReceipts = settings.sendReadReceipts
            confirmBeforeOpeningFiles = settings.confirmBeforeOpeningFiles
            currentScreen = AppScreen.SETTINGS
        },
    )
}

```

```

AppScreen.MY_PROFILE -> {
    val profile = myProfile

```

```

MyProfileScreen(
    name = profile?.name ?: "Your Name",
    username = profile?.username ?: "@your.username",
    bio = profile?.bio ?: "Hey there! I'm using Poster.",
    avatarUrl = profile?.avatarUrl,
    onBackClick = {
        Log.d(TAG, "Back from MyProfile")
        currentScreen = myProfileBackScreen
    },
)

```

```

onAvatarClick = {
    Log.d(TAG, "Avatar clicked from MyProfile")
    avatarPickerLauncher.launch(arrayOf("image/*"))
},
onNameChange = { name ->
    Log.d(TAG, "Name changed placeholder: $name")
    scope.launch {
        updateNameUseCase(name)
        .onSuccess { updatedProfile ->
            myProfile = updatedProfile
        }
        .onFailure { error ->
            Log.e(TAG, "Failed to update name", error)
        }
    }
},
onUsernameChange = { username ->
    Log.d(TAG, "Username changed placeholder: $username")
    scope.launch {
        updateUsernameUseCase(username)
        .onSuccess { updatedProfile ->
            myProfile = updatedProfile
        }
        .onFailure { error ->
            Log.e(TAG, "Failed to update username", error)
        }
    }
},
onBioChange = { bio ->
    Log.d(TAG, "Bio changed placeholder, length=${bio.length}")
    scope.launch {
        updateBioUseCase(bio)
        .onSuccess { updatedProfile ->
            myProfile = updatedProfile
        }
        .onFailure { error ->
            Log.e(TAG, "Failed to update bio", error)
        }
    }
},
)
}

AppScreen.ACCESS_TOKEN_SETTINGS -> {
    AccessTokenSettingsScreen(
        initialToken = "",
        initialSmtpHost = smtpHost,
        initialSmtpPort = smtpPort,
        initialImapHost = imapHost,
        initialImapPort = imapPort,
        onBackClick = {

```

```

        Log.d(TAG, "Back from AccessTokenSettings")
        currentScreen = accessTokenBackScreen
    },
    onSaveTokenClick = { mailSettings ->
        Log.d(TAG, "Access token save requested, length=${mailSettings.token.length}

    })

    scope.launch {
        saveMailAccessTokenUseCase(mailSettings)
            .onSuccess {
                hasMailAccessToken = true
                smtpHost = mailSettings.smtpHost.orEmpty()
                smtpPort = mailSettings.smtpPort?.toString().orEmpty()
                imapHost = mailSettings.imapHost.orEmpty()
                imapPort = mailSettings.imapPort?.toString().orEmpty()
                currentScreen = accessTokenBackScreen
            }
            .onFailure { error ->
                Log.e(TAG, "Failed to save access token", error)
            }
        }
    },
    onHowToGetTokenClick = {
        Log.d(TAG, "How to get access token clicked")
        currentScreen = AppScreen.ACCESS_TOKEN_HELP
    },
)
}

AppScreen.ACCESS_TOKEN_HELP -> {
    AccessTokenGuideScreen(
        onBackClick = {
            currentScreen = AppScreen.ACCESS_TOKEN_SETTINGS
        },
    )
}

AppScreen.MESSAGES -> {
    val chat = selectedChat ?: chats.firstOrNull()

    if (chat == null) {
        LaunchedEffect(Unit) {
            currentScreen = AppScreen.CHATS
        }
    } else {
        MessagesScreen(
            contactName = chat.name,
            contactInitials = chat.initials,
            messages = messages,
            canSendMessages = hasMailAccessToken,
            onBackClick = {
                Log.d(TAG, "Back from Messages")
            }
        )
    }
}

```

```

        currentScreen = AppScreen.CHATS
    },
    onProfileClick = {
        Log.d(TAG, "Open Profile from Messages")
        currentScreen = AppScreen.PROFILE
    },
    onMoreClick = {
        Log.d(TAG, "More clicked in Messages")
    },
    onSendClick = { text ->
        if (hasMailAccessToken) {
            Log.d(TAG, "Send message requested: textLength=${text.length}")
            scope.launch {
                sendMessageUseCase(chat.id, text)
                .onFailure { error ->
                    Log.e(TAG, "Failed to send message", error)
                }
            }
        } else {
            Log.d(TAG, "Send message blocked, mail access token is missing")
        }
    },
    onFilePicked = { uri ->
        Log.d(TAG, "File picked: $uri")
        scope.launch {
            val attachment = uri.toAttachment(
                context = this@MainActivity,
                type = AttachmentType.DOCUMENT,
                fallbackName = "Selected_File.pdf",
                fallbackMimeType = "application/pdf",
            )

            sendMessageWithAttachmentsUseCase(
                chatId = chat.id,
                text = "",
                attachments = listOf(attachment),
            ).onFailure { error ->
                Log.e(TAG, "Failed to send file attachment", error)
            }
        }
    },
    onImagePicked = { uri ->
        Log.d(TAG, "Image picked: $uri")
        scope.launch {
            val attachment = uri.toAttachment(
                context = this@MainActivity,
                type = AttachmentType.IMAGE,
                fallbackName = "image.jpg",
                fallbackMimeType = "image/jpeg",
            )

```

```

        sendMessageWithAttachmentsUseCase(
            chatId = chat.id,
            text = "",
            attachments = listOf(attachment),
        ).onFailure { error ->
            Log.e(TAG, "Failed to send image attachment", error)
        }
    },
)
}
}

AppScreen.PROFILE -> {
    val chat = selectedChat ?: chats.firstOrNull()

    if (chat == null) {
        LaunchedEffect(Unit) {
            currentScreen = AppScreen.CHATS
        }
    } else {
        ProfileScreen(
            initials = chat.initials,
            name = chat.name,
            username = "@${chat.name.lowercase().replace(" ", ".")}",
            bio = "Product designer passionate about creating beautiful and functional user
experiences.",
            attachments = messages.toProfileMediaAttachments(),
            files = messages.toProfileFiles(),
            onBackClick = {
                Log.d(TAG, "Back from Profile")
                currentScreen = AppScreen.MESSAGES
            },
        )
    }
}
}
}
}
}

override fun onStart() {
    super.onStart()
    Log.d(TAG, "onStart")
}

override fun onResume() {
    super.onResume()
    Log.d(TAG, "onResume")
}

```

```

    }

    override fun onPause() {
        super.onPause()
        Log.d(TAG, "onPause")
    }

    override fun onStop() {
        super.onStop()
        Log.d(TAG, "onStop")
    }

    override fun onDestroy() {
        super.onDestroy()
        Log.d(TAG, "onDestroy")
    }
}

private enum class AppScreen {
    SIGN_IN,
    SIGN_UP,
    VERIFY_EMAIL,
    CHATS,
    SETTINGS,
    LANGUAGE_SETTINGS,
    PRIVACY_SETTINGS,
    MY_PROFILE,
    ACCESS_TOKEN_SETTINGS,
    ACCESS_TOKEN_HELP,
    MESSAGES,
    PROFILE,
}

private const val ScreenEnterDurationMillis = 360
private const val ScreenExitDurationMillis = 280

private val ScreenEnterEasing = CubicBezierEasing(0.2f, 0f, 0f, 1f)
private val ScreenExitEasing = CubicBezierEasing(0.4f, 0f, 1f, 1f)

private enum class EditableSetting(
    val id: String,
) {
    NAME(
        id = "name",
    ),
    USERNAME(
        id = "username",
    ),
    EMAIL(
        id = "email",
    ),
}

```

```

    BIRTHDAY(
        id = "birthday",
    ),
    BIO(
        id = "bio",
    );

    companion object {
        fun fromId(id: String): EditableSetting? {
            return entries.firstOrNull { it.id == id }
        }
    }
}

private fun String.withUsernamePrefix(): String {
    val username = trim()
    return if (username.startsWith("@")) {
        username
    } else {
        "@$username"
    }
}

private fun List<MessageUi>.toProfileFiles(): List<ProfileFileUi> {
    return mapNotNull { message ->
        val attachment = message.attachment ?: return@mapNotNull null
        if (message.type != MessageContentType.FILE) {
            return@mapNotNull null
        }

        ProfileFileUi(
            id = attachment.id,
            name = attachment.fileName,
            size = attachment.fileSize,
            date = message.time,
        )
    }
}

private fun List<MessageUi>.toProfileMediaAttachments(): List<Attachment> {
    return mapNotNull { message ->
        val attachment = message.attachment ?: return@mapNotNull null
        if (message.type != MessageContentType.IMAGE) {
            return@mapNotNull null
        }

        Attachment(
            id = attachment.id,
            localUri = attachment.localUri,
            remoteUrl = attachment.remoteUrl,
            type = AttachmentType.IMAGE,
        )
    }
}

```

```

        fileName = attachment.fileName,
        mimeType = attachment.mimeType,
        sizeBytes = attachment.sizeBytes,
        uploadStatus = AttachmentUploadStatus.UPLOADED,
    )
}
}

private fun Uri.toAttachment(
    context: Context,
    type: AttachmentType,
    fallbackName: String,
    fallbackMimeType: String,
): Attachment {
    val contentResolver = context.contentResolver
    var fileName = fallbackName
    var sizeBytes = 0L
    val mimeType = contentResolver.getType(this).orEmpty().ifBlank {
        fallbackMimeType
    }

    contentResolver.query(this, null, null, null, null)?.use { cursor ->
        val nameIndex = cursor.getColumnIndex(OpenableColumns.DISPLAY_NAME)
        val sizeIndex = cursor.getColumnIndex(OpenableColumns.SIZE)
        if (cursor.moveToFirst()) {
            if (nameIndex >= 0) {
                fileName = cursor.getString(nameIndex) ?: fileName
            }
            if (sizeIndex >= 0) {
                sizeBytes = cursor.getLong(sizeIndex)
            }
        }
    }

    return Attachment(
        id = "local-#{System.currentTimeMillis()}",
        localUri = toString(),
        remoteUrl = null,
        type = type,
        fileName = fileName,
        mimeType = mimeType,
        sizeBytes = sizeBytes,
        uploadStatus = AttachmentUploadStatus.LOCAL_ONLY,
    )
}

```

Листинг 90 – Код файла `app/src/main/java/com/example/poster/PosterApplication.kt`

```
package com.example.poster

import android.app.Application
import android.util.Log
import dagger.hilt.android.HiltAndroidApp

@HiltAndroidApp
class PosterApplication : Application() {
    override fun onCreate() {
        super.onCreate()
        Log.d(TAG, "Application created")
    }

    private companion object {
        private const val TAG = "PosterApplication"
    }
}
```

Листинг 91 – Код файла `app/src/main/java/com/example/poster/presentation/auth/components/AuthComponents.kt`

```
package com.example.poster.presentation.auth.components

import androidx.compose.animation.animateColorAsState
import androidx.compose.animation.core.CubicBezierEasing
import androidx.compose.animation.core.animateDpAsState
import androidx.compose.animation.core.animateFloatAsState
import androidx.compose.animation.core.tween
import androidx.compose.foundation.background
import androidx.compose.foundation.border
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.BoxScope
import androidx.compose.foundation.layout.PaddingValues
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.layout.width
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.foundation.text.BasicTextField
import androidx.compose.foundation.text.KeyboardOptions
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.outlined.Email
import androidx.compose.material.icons.outlined.Lock
import androidx.compose.material.icons.outlined.Person
import androidx.compose.material3.Button
import androidx.compose.material3.ButtonDefaults
import androidx.compose.material3.Icon
import androidx.compose.material3.Text
import androidx.compose.material3.TextButton
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.alpha
import androidx.compose.ui.draw.scale
import androidx.compose.ui.draw.shadow
import androidx.compose.ui.graphics.Brush
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.SolidColor
import androidx.compose.ui.graphics.vector.ImageVector
import androidx.compose.ui.text.TextStyle
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.input.KeyboardType
import androidx.compose.ui.text.input.PasswordVisualTransformation
```

```
import androidx.compose.ui.text.input.VisualTransformation
import androidx.compose.ui.text.style.TextAlign
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
```

```
val PosterBackground = Color(0xFF030306)
val PosterSurface = Color(0xFF1B1A31)
val PosterStroke = Color(0xFF302B68)
val PosterPrimary = Color(0xFF625BFF)
val PosterPrimaryDark = Color(0xFF4538C8)
val PosterTextPrimary = Color(0xFFFF6F6FF)
val PosterTextSecondary = Color(0xFFA2A3B8)
val PosterTextMuted = Color(0xFF7B7D92)
val PosterIcon = Color(0xFF7F8BFF)
```

```
private val AuthMotionEasing = CubicBezierEasing(0.2f, 0f, 0f, 1f)
```

```
@Composable
fun PosterAuthBackground(
    modifier: Modifier = Modifier,
    content: @Composable BoxScope.() -> Unit,
) {
    Box(
        modifier = modifier
            .fillMaxSize()
            .background(
                Brush.radialGradient(
                    colors = listOf(
                        Color(0xFF10102A),
                        PosterBackground,
                        Color(0xFF070414),
                    ),
                    radius = 1250f,
                )
            ),
        content = content,
    )
}
```

```
@Composable
fun PosterLogoBlock(
    showSubtitle: Boolean,
    modifier: Modifier = Modifier,
) {
    androidx.compose.foundation.layout.Column(
        modifier = modifier,
        horizontalAlignment = Alignment.CenterHorizontally,
    ) {
        Text(
            text = "Poster",
            color = PosterPrimary,

```

```

        fontSize = 44.sp,
        fontWeight = FontWeight.SemiBold,
        letterSpacing = (-1).sp,
    )

    if (showSubtitle) {
        Spacer(modifier = Modifier.height(6.dp))
        Text(
            text = "Secure messaging via SMTP/IMAP",
            color = PosterTextSecondary,
            fontSize = 15.sp,
        )
    }
}

@Composable
fun PosterAuthInput(
    value: String,
    onChange: (String) -> Unit,
    hint: String,
    icon: ImageVector,
    modifier: Modifier = Modifier,
    keyboardType: KeyboardType = KeyboardType.Text,
    isPassword: Boolean = false,
) {
    Row(
        modifier = modifier
            .height(58.dp)
            .fillMaxWidth()
            .background(
                color = PosterSurface,
                shape = RoundedCornerShape(13.dp),
            )
            .border(
                width = 1.dp,
                color = PosterStroke,
                shape = RoundedCornerShape(13.dp),
            )
            .padding(horizontal = 18.dp),
        verticalAlignment = Alignment.CenterVertically,
    ) {
        Icon(
            imageVector = icon,
            contentDescription = null,
            tint = PosterIcon,
            modifier = Modifier.size(21.dp),
        )

        Spacer(modifier = Modifier.width(14.dp))
    }
}

```

```

BasicTextField(
    value = value,
    onValueChange = onValueChange,
    singleLine = true,
    cursorBrush = SolidColor(PosterPrimary),
    textStyle = TextStyle(
        color = PosterTextPrimary,
        fontSize = 15.sp,
    ),
    keyboardOptions = KeyboardOptions(keyboardType = keyboardType),
    visualTransformation = if (isPassword) {
        PasswordVisualTransformation()
    } else {
        VisualTransformation.None
    },
    decorationBox = { innerTextField ->
        Box(
            modifier = Modifier.fillMaxWidth(),
            contentAlignment = Alignment.CenterStart,
        ) {
            if (value.isEmpty()) {
                Text(
                    text = hint,
                    color = PosterTextMuted,
                    fontSize = 15.sp,
                )
            }
            innerTextField()
        }
    },
    modifier = Modifier.fillMaxWidth(),
)
}

```

```

@Composable
fun PosterPrimaryButton(
    text: String,
    onClick: () -> Unit,
    modifier: Modifier = Modifier,
    enabled: Boolean = true,
) {
    Button(
        onClick = onClick,
        enabled = enabled,
        shape = RoundedCornerShape(13.dp),
        colors = ButtonDefaults.buttonColors(
            containerColor = Color.Transparent,
            disabledContainerColor = Color.Transparent,
        ),
        contentPadding = PaddingValues(),
    )
}

```

```

modifier = modifier
    .height(56.dp)
    .fillMaxWidth()
    .alpha(if (enabled) 1f else 0.6f)
    .shadow(
        elevation = 28.dp,
        shape = RoundedCornerShape(13.dp),
        spotColor = PosterPrimary.copy(alpha = 0.45f),
    )
    .background(
        brush = Brush.horizontalGradient(
            colors = listOf(PosterPrimary, PosterPrimaryDark),
        ),
        shape = RoundedCornerShape(13.dp),
    ),
) {
    Text(
        text = text,
        color = Color.White,
        fontSize = 16.sp,
        fontWeight = FontWeight.Bold,
    )
}
}

```

```

@Composable
fun PosterBottomAuthText(
    normalText: String,
    actionText: String,
    onActionClick: () -> Unit,
    modifier: Modifier = Modifier,
) {
    Row(
        modifier = modifier,
        horizontalArrangement = Arrangement.Center,
        verticalAlignment = Alignment.CenterVertically,
    ) {
        Text(
            text = normalText,
            color = PosterTextMuted,
            fontSize = 15.sp,
        )
        TextButton(
            onClick = onActionClick,
            contentPadding = PaddingValues(horizontal = 4.dp),
        ) {
            Text(
                text = actionText,
                color = PosterIcon,
                fontSize = 15.sp,
                fontWeight = FontWeight.Bold,
            )
        }
    }
}

```

```

    }
  }
}

```

```

@Composable
fun PosterOtpCell(
    value: String,
    isActive: Boolean,
    modifier: Modifier = Modifier,
) {
    val cellScale by animateFloatAsState(
        targetValue = when {
            isActive -> 1.025f
            value.isNotEmpty() -> 1.015f
            else -> 1f
        },
        animationSpec = tween(
            durationMillis = 220,
            easing = AuthMotionEasing,
        ),
        label = "otpCellScale",
    )
    val cellElevation by animateDpAsState(
        targetValue = if (isActive || value.isNotEmpty()) 18.dp else 0.dp,
        animationSpec = tween(
            durationMillis = 240,
            easing = AuthMotionEasing,
        ),
        label = "otpCellElevation",
    )
    val borderColor by animateColorAsState(
        targetValue = when {
            isActive -> PosterPrimary
            value.isNotEmpty() -> PosterPrimary.copy(alpha = 0.72f)
            else -> PosterStroke
        },
        animationSpec = tween(
            durationMillis = 220,
            easing = AuthMotionEasing,
        ),
        label = "otpBorderColor",
    )
    val glowColor by animateColorAsState(
        targetValue = when {
            isActive -> PosterPrimary.copy(alpha = 0.32f)
            value.isNotEmpty() -> PosterPrimary.copy(alpha = 0.18f)
            else -> Color.Transparent
        },
        animationSpec = tween(
            durationMillis = 240,

```

```

        easing = AuthMotionEasing,
    ),
    label = "otpGlowColor",
)

Box(
    modifier = modifier
        .shadow(
            elevation = cellElevation,
            shape = RoundedCornerShape(13.dp),
            spotColor = glowColor,
        )
        .scale(cellScale)
        .background(
            color = PosterSurface,
            shape = RoundedCornerShape(13.dp),
        )
        .border(
            width = 2.dp,
            color = borderColor,
            shape = RoundedCornerShape(13.dp),
        ),
    contentAlignment = Alignment.Center,
) {
    Text(
        text = value,
        color = PosterTextPrimary,
        fontSize = 22.sp,
        fontWeight = FontWeight.SemiBold,
        textAlign = TextAlign.Center,
    )
}

object PosterAuthIcons {
    val Name = Icons.Outlined.Person
    val Email = Icons.Outlined.Email
    val Password = Icons.Outlined.Lock
}

```

Листинг 92 – Код файла `app/src/main/java/com/example/poster/presentation/auth/login/LoginScreen.kt`

```
package com.example.poster.presentation.auth.login

import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import com.example.poster.presentation.common.FeatureStubScreen

@Composable
fun LoginScreen(modifier: Modifier = Modifier) {
    FeatureStubScreen(
        title = "Login",
        description = "Entry point for sign in and session bootstrap.",
        checkpoints = listOf(
            "Collect identifier and password",
            "Trigger LoginUseCase from a future ViewModel",
            "Redirect to chats after successful authentication",
        ),
        modifier = modifier,
    )
}
```

Листинг 93 – Код файла app/src/main/java/com/example/poster/presentation/auth/otp/OtpScreen.kt

```
package com.example.poster.presentation.auth.otp

import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier

@Composable
fun OtpScreen(modifier: Modifier = Modifier) {
    VerifyEmailScreen(
        email = "user@mail.ru",
        modifier = modifier,
    )
}
```

Листинг 94 – Код файла `app/src/main/java/com/example/poster/presentation/auth/otp/VerifyEmailScreen.kt`

```
package com.example.poster.presentation.auth.otp

import androidx.compose.foundation.clickable
import androidx.compose.foundation.interaction.MutableInteractionSource
import androidx.compose.animation.core.CubicBezierEasing
import androidx.compose.animation.core.animateDpAsState
import androidx.compose.animation.core.tween
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.BoxWithConstraints
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.WindowInsets
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.ime
import androidx.compose.foundation.layout.imePadding
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.layout.widthIn
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.text.BasicTextField
import androidx.compose.foundation.verticalScroll
import androidx.compose.foundation.text.KeyboardOptions
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.saveable.rememberSaveable
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.focus.FocusRequester
import androidx.compose.ui.focus.focusRequester
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.SolidColor
import androidx.compose.ui.platform.LocalDensity
import androidx.compose.ui.platform.LocalFocusManager
import androidx.compose.ui.platform.LocalSoftwareKeyboardController
import androidx.compose.ui.text.TextStyle
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.input.KeyboardType
import androidx.compose.ui.text.style.TextAlign
import androidx.compose.ui.tooling.preview.Preview
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import androidx.compose.material3.Text
import com.example.poster.presentation.auth.components.PostAuthBackground
```

```
import com.example.poster.presentation.auth.components.PosterBottomAuthText
import com.example.poster.presentation.auth.components.PosterLogoBlock
import com.example.poster.presentation.auth.components.PosterPrimary
import com.example.poster.presentation.auth.components.PosterOtpCell
import com.example.poster.presentation.auth.components.PosterPrimaryButton
import com.example.poster.presentation.auth.components.PosterTextPrimary
import com.example.poster.presentation.auth.components.PosterTextSecondary
import com.example.poster.ui.theme.PosterTheme
```

```
private val VerifyMotionEasing = CubicBezierEasing(0.2f, 0f, 0f, 1f)
```

```
@Composable
fun VerifyEmailScreen(
    email: String,
    onVerifyClick: (code: String) -> Unit = {},
    onResendClick: () -> Unit = {},
    modifier: Modifier = Modifier,
) {
    var code by rememberSaveable { mutableStateOf("") }
    val focusManager = LocalFocusManager.current
    val keyboardController = LocalSoftwareKeyboardController.current
    val focusRequester = remember { FocusRequester() }
    val interactionSource = remember { MutableInteractionSource() }
    val scrollState = rememberScrollState()
    val density = LocalDensity.current
    val isKeyboardVisible = WindowInsets.ime.getBottom(density) > 0
    val topPadding by animateDpAsState(
        targetValue = if (isKeyboardVisible) 48.dp else 92.dp,
        animationSpec = tween(durationMillis = 300, easing = VerifyMotionEasing),
        label = "verifyTopPadding",
    )
    val logoBottomSpacing by animateDpAsState(
        targetValue = if (isKeyboardVisible) 28.dp else 48.dp,
        animationSpec = tween(durationMillis = 300, easing = VerifyMotionEasing),
        label = "verifyLogoSpacing",
    )
    val otpTopSpacing by animateDpAsState(
        targetValue = if (isKeyboardVisible) 24.dp else 34.dp,
        animationSpec = tween(durationMillis = 260, easing = VerifyMotionEasing),
        label = "verifyOtpTopSpacing",
    )
    val buttonTopSpacing by animateDpAsState(
        targetValue = if (isKeyboardVisible) 24.dp else 34.dp,
        animationSpec = tween(durationMillis = 260, easing = VerifyMotionEasing),
        label = "verifyButtonTopSpacing",
    )

    PosterAuthBackground(modifier = modifier) {
        Column(
            modifier = Modifier
                .fillMaxWidth()
        )
    }
}
```

```

        .widthIn(max = 448.dp)
        .align(Alignment.TopCenter)
        .verticalScroll(scrollState)
        .imePadding()
        .padding(
            start = 28.dp,
            end = 28.dp,
            top = topPadding,
            bottom = 32.dp,
        ),
        horizontalAlignment = Alignment.CenterHorizontally,
    ) {
        PosterLogoBlock(showSubtitle = false)

        Spacer(modifier = Modifier.height(logoBottomSpacing))

        Text(
            text = "Verify Your Email",
            color = PosterTextPrimary,
            fontSize = 24.sp,
            fontWeight = FontWeight.Bold,
        )

        Spacer(modifier = Modifier.height(10.dp))

        Text(
            text = "Enter the 6-digit code sent to",
            color = PosterTextSecondary,
            fontSize = 15.sp,
            textAlign = TextAlign.Center,
        )

        Spacer(modifier = Modifier.height(4.dp))

        Text(
            text = email,
            color = PosterTextPrimary,
            fontSize = 15.sp,
            textAlign = TextAlign.Center,
        )

        Spacer(modifier = Modifier.height(otpTopSpacing))

        BoxWithConstraints(
            modifier = Modifier
                .fillMaxWidth()
                .clickable(
                    interactionSource = interactionSource,
                    indication = null,
                ) {
                focusRequester.requestFocus()
            }
        )
    }
}

```

```

        keyboardController?.show()
    },
) {
    val cellSpacing = 10.dp
    val cellSize = ((maxWidth - cellSpacing * 5) / 6)
        .coerceAtMost(56.dp)
        .coerceAtLeast(42.dp)
    val activeIndex = code.length.coerceAtMost(5)

    BasicTextField(
        value = code,
        onValueChange = { newValue ->
            val filtered = newValue.filter(Char::isDigit).take(6)
            code = filtered

            if (filtered.length == 6) {
                focusManager.clearFocus()
                keyboardController?.hide()
            }
        },
        keyboardOptions = KeyboardOptions(
            keyboardType = KeyboardType.NumberPassword,
        ),
        singleLine = true,
        cursorBrush = SolidColor(PosterPrimary),
        textStyle = TextStyle(
            color = Color.Transparent,
            fontSize = 1.sp,
        ),
        modifier = Modifier
            .fillMaxWidth()
            .height(cellSize)
            .focusRequester(focusRequester),
    )

    androidx.compose.foundation.layout.Row(
        modifier = Modifier.fillMaxWidth(),
        horizontalArrangement = Arrangement.spacedBy(
            space = cellSpacing,
            alignment = Alignment.CenterHorizontally,
        ),
        verticalAlignment = Alignment.CenterVertically,
    ) {
        repeat(6) { index ->
            PosterOtpCell(
                value = code.getOrNull(index)?.toString().orEmpty(),
                isActive = index == activeIndex && code.length < 6,
                modifier = Modifier.size(cellSize),
            )
        }
    }
}

```

```

    }

    Spacer(modifier = Modifier.height(buttonTopSpacing))

    PosterPrimaryButton(
        text = "Verify",
        enabled = code.length == 6,
        onClick = { onVerifyClick(code) },
    )

    Spacer(modifier = Modifier.height(22.dp))

    PosterBottomAuthText(
        normalText = "Didn't receive the code?",
        actionText = "Resend",
        onActionClick = onResendClick,
    )
}
}
}

@Preview(showBackground = true, widthDp = 390, heightDp = 844)
@Composable
private fun VerifyEmailScreenPreview() {
    PosterTheme {
        VerifyEmailScreen(email = "user@mail.ru")
    }
}

```

Листинг 95 – Код файла app/src/main/java/com/example/poster/presentation/auth/register/RegisterScreen.kt

```
package com.example.poster.presentation.auth.register

import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.widthIn
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.saveable.rememberSaveable
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.input.KeyboardType
import androidx.compose.ui.tooling.preview.Preview
import androidx.compose.ui.unit.dp
import com.example.poster.presentation.auth.components.PosterAuthBackground
import com.example.poster.presentation.auth.components.PosterAuthIcons
import com.example.poster.presentation.auth.components.PosterAuthInput
import com.example.poster.presentation.auth.components.PosterBottomAuthText
import com.example.poster.presentation.auth.components.PosterLogoBlock
import com.example.poster.presentation.auth.components.PosterPrimaryButton
import com.example.poster.ui.theme.PosterTheme

@Composable
fun RegisterScreen(
    onContinueClick: (name: String, email: String, password: String) -> Unit = { _, _, _ -> },
    onSignInClick: () -> Unit = {},
    modifier: Modifier = Modifier,
) {
    var name by rememberSaveable { mutableStateOf("") }
    var email by rememberSaveable { mutableStateOf("") }
    var password by rememberSaveable { mutableStateOf("") }

    PosterAuthBackground(modifier = modifier) {
        Column(
            modifier = Modifier
                .fillMaxWidth()
                .widthIn(max = 448.dp)
                .align(Alignment.Center)
                .padding(horizontal = 28.dp),
            horizontalAlignment = Alignment.CenterHorizontally,
        ) {
            PosterLogoBlock(showSubtitle = true)

            Spacer(modifier = Modifier.height(48.dp))
        }
    }
}
```

```

        PosterAuthInput(
            value = name,
            onValueChange = { name = it },
            hint = "Name",
            icon = PosterAuthIcons.Name,
        )

        Spacer(modifier = Modifier.height(16.dp))

        PosterAuthInput(
            value = email,
            onValueChange = { email = it },
            hint = "Email",
            icon = PosterAuthIcons.Email,
            keyboardType = KeyboardType.Email,
        )

        Spacer(modifier = Modifier.height(16.dp))

        PosterAuthInput(
            value = password,
            onValueChange = { password = it },
            hint = "Password",
            icon = PosterAuthIcons.Password,
            keyboardType = KeyboardType.Password,
            isPassword = true,
        )

        Spacer(modifier = Modifier.height(40.dp))

        PosterPrimaryButton(
            text = "Continue",
            onClick = { onContinueClick(name, email, password) },
        )

        Spacer(modifier = Modifier.height(20.dp))

        PosterBottomAuthText(
            normalText = "Already have an account?",
            actionText = "Sign In",
            onActionClick = onSignInClick,
        )
    }
}

@Preview(showBackground = true, widthDp = 390, heightDp = 844)
@Composable
private fun RegisterScreenPreview() {
    PosterTheme {

```

```
        RegisterScreen()  
    }  
}
```

Листинг 96 – Код файла `app/src/main/java/com/example/poster/presentation/auth/signin/SignInScreen.kt`

```
package com.example.poster.presentation.auth.signin

import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.widthIn
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.saveable.rememberSaveable
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.input.KeyboardType
import androidx.compose.ui.tooling.preview.Preview
import androidx.compose.ui.unit.dp
import com.example.poster.presentation.auth.components.PosterAuthBackground
import com.example.poster.presentation.auth.components.PosterAuthIcons
import com.example.poster.presentation.auth.components.PosterAuthInput
import com.example.poster.presentation.auth.components.PosterBottomAuthText
import com.example.poster.presentation.auth.components.PosterLogoBlock
import com.example.poster.presentation.auth.components.PosterPrimaryButton
import com.example.poster.ui.theme.PosterTheme

@Composable
fun SignInScreen(
    onSignInClick: (email: String, password: String) -> Unit = { _, _ -> },
    onSignUpClick: () -> Unit = {},
    modifier: Modifier = Modifier,
) {
    var email by rememberSaveable { mutableStateOf("") }
    var password by rememberSaveable { mutableStateOf("") }

    PosterAuthBackground(modifier = modifier) {
        Column(
            modifier = Modifier
                .fillMaxWidth()
                .widthIn(max = 448.dp)
                .align(Alignment.Center)
                .padding(horizontal = 28.dp),
            horizontalAlignment = Alignment.CenterHorizontally,
        ) {
            PosterLogoBlock(showSubtitle = true)

            Spacer(modifier = Modifier.height(48.dp))
        }
    }
}
```

```

        PosterAuthInput(
            value = email,
            onValueChange = { email = it },
            hint = "Email",
            icon = PosterAuthIcons.Email,
            keyboardType = KeyboardType.Email,
        )

        Spacer(modifier = Modifier.height(16.dp))

        PosterAuthInput(
            value = password,
            onValueChange = { password = it },
            hint = "Password",
            icon = PosterAuthIcons.Password,
            keyboardType = KeyboardType.Password,
            isPassword = true,
        )

        Spacer(modifier = Modifier.height(40.dp))

        PosterPrimaryButton(
            text = "Sign In",
            onClick = { onSignInClick(email, password) },
        )

        Spacer(modifier = Modifier.height(20.dp))

        PosterBottomAuthText(
            normalText = "Don't have an account?",
            actionText = "Sign Up",
            onActionClick = onSignUpClick,
        )
    }
}

@Preview(showBackground = true, widthDp = 390, heightDp = 844)
@Composable
private fun SignInScreenPreview() {
    PosterTheme {
        SignInScreen()
    }
}

```

Листинг 97 – Код файла app/src/main/java/com/example/poster/presentation/chats/ChatListScreen.kt

```
package com.example.poster.presentation.chats

import androidx.compose.animation.AnimatedVisibility
import androidx.compose.animation.expandVertically
import androidx.compose.animation.fadeIn
import androidx.compose.animation.fadeOut
import androidx.compose.animation.shrinkVertically
import androidx.compose.animation.slideInVertically
import androidx.compose.animation.slideOutVertically
import androidx.compose.animation.core.FastOutSlowInEasing
import androidx.compose.animation.core.LinearEasing
import androidx.compose.animation.core.RepeatMode
import androidx.compose.animation.core.animateFloat
import androidx.compose.animation.core.infiniteRepeatable
import androidx.compose.animation.core.rememberInfiniteTransition
import androidx.compose.animation.core.tween
import androidx.compose.foundation.background
import androidx.compose.foundation.border
import androidx.compose.foundation.clickable
import androidx.compose.foundation.gestures.awaitFirstDown
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.PaddingValues
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.navigationBarsPadding
import androidx.compose.foundation.layout.offset
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.layout.statusBarsPadding
import androidx.compose.foundation.layout.width
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.foundation.lazy.rememberLazyListState
import androidx.compose.foundation.shape.CircleShape
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.foundation.text.BasicTextField
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.outlined.Close
import androidx.compose.material.icons.outlined.Email
import androidx.compose.material.icons.outlined.Info
import androidx.compose.material.icons.outlined.Person
import androidx.compose.material.icons.outlined.Search
import androidx.compose.material.icons.outlined.Settings
```

```
import androidx.compose.material3.Icon
import androidx.compose.material3.IconButton
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.LaunchedEffect
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.rememberCoroutineScope
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.blur
import androidx.compose.ui.draw.clip
import androidx.compose.ui.draw.shadow
import androidx.compose.ui.focus.FocusRequester
import androidx.compose.ui.focus.focusRequester
import androidx.compose.ui.graphics.Brush
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.SolidColor
import androidx.compose.ui.input.pointer.pointerInput
import androidx.compose.ui.layout.ContentScale
import androidx.compose.ui.platform.LocalFocusManager
import androidx.compose.ui.text.TextStyle
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.style.TextOverflow
import androidx.compose.ui.tooling.preview.Preview
import androidx.compose.ui.unit.Dp
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import coil.compose.AsyncImage
import com.example.poster.domain.model.Chat
import kotlinx.coroutines.delay
import kotlinx.coroutines.launch
```

```
private val PosterBackground = Color(0xFF030306)
private val PosterTopBar = Color(0xFF101014)
private val PosterSurface = Color(0xFF1B1A31)
private val PosterStroke = Color(0xFF302B68)
private val PosterPrimary = Color(0xFF625BFF)
private val PosterPrimaryDark = Color(0xFF493CCB)
private val PosterTextPrimary = Color(0xFFFF7F7FF)
private val PosterTextSecondary = Color(0xFFA5A6BA)
private val PosterTextMuted = Color(0xFF7B7D92)
private val PosterDivider = Color(0xFF11111A)
private val PosterIcon = Color(0xFF8B93FF)
```

```
data class ChatPreviewUi(
    val id: String,
    val initials: String,
    val name: String,
```

```

        val lastMessage: String,
        val time: String,
        val unreadCount: Int = 0,
    )

    fun Chat.toChatPreviewUi(): ChatPreviewUi {
        return ChatPreviewUi(
            id = id,
            initials = initials,
            name = title,
            lastMessage = lastMessage,
            time = lastMessageTime,
            unreadCount = unreadCount,
        )
    }
}

```

```

@Composable
fun ChatListScreen(
    chats: List<ChatPreviewUi>,
    isSetupRequired: Boolean,
    profileAvatarUrl: String? = null,
    hasMailAccessToken: Boolean = !isSetupRequired,
    onChatClick: (ChatPreviewUi) -> Unit,
    onSetupTokenClick: () -> Unit,
    onSettingsClick: () -> Unit,
    onProfileClick: () -> Unit = {},
) {
    var searchQuery by remember { mutableStateOf("") }
    var isSearchVisible by remember { mutableStateOf(false) }
    var shouldFocusSearchOnShow by remember { mutableStateOf(false) }
    var isScrollingToTopForSearch by remember { mutableStateOf(false) }
    val listState = rememberLazyListState()
    val coroutineScope = rememberCoroutineScope()
    val searchFocusRequester = remember { FocusRequester() }
    val focusManager = LocalFocusManager.current

    fun showSearch(requestFocus: Boolean) {
        shouldFocusSearchOnShow = requestFocus
        if (!requestFocus) {
            focusManager.clearFocus()
        }
        isSearchVisible = true
    }

    fun openSearchFromIcon() {
        if (hasMailAccessToken) {
            isScrollingToTopForSearch = true
        }
        showSearch(requestFocus = true)
        if (hasMailAccessToken) {
            coroutineScope.launch {

```

```

        try {
            listState.animateScrollToItem(0)
        } finally {
            isScrollingToTopForSearch = false
        }
    }
}

fun dismissSearch() {
    searchQuery = ""
    isSearchVisible = false
    shouldFocusSearchOnShow = false
    isScrollingToTopForSearch = false
    focusManager.clearFocus()
}

val filteredChats = remember(searchQuery, chats) {
    if (searchQuery.isBlank()) {
        chats
    } else {
        chats.filter {
            it.name.contains(searchQuery, ignoreCase = true) ||
            it.lastMessage.contains(searchQuery, ignoreCase = true)
        }
    }
}

val isListAtTop = listState.firstVisibleItemIndex == 0 &&
    listState.firstVisibleItemScrollOffset == 0

LaunchedEffect(isSearchVisible, shouldFocusSearchOnShow) {
    if (isSearchVisible && shouldFocusSearchOnShow) {
        delay(80)
        searchFocusRequester.requestFocus()
    }
}

LaunchedEffect(isSearchVisible, isListAtTop, isScrollingToTopForSearch) {
    if (isSearchVisible && !isListAtTop && !isScrollingToTopForSearch) {
        dismissSearch()
    }
}

Box(
    modifier = Modifier
        .fillMaxSize()
        .background(
            Brush.radialGradient(
                colors = listOf(
                    Color(0xFF08051A),

```

```

        PosterBackground,
        Color(0xFF030306),
    ),
    radius = 1100f,
)
)
.pointerInput(isSearchVisible, hasMailAccessToken, isListAtTop) {
    awaitPointerEventScope {
        while (true) {
            val down = awaitFirstDown(requireUnconsumed = false)
            val startY = down.position.y
            var searchStateChangedByGesture = false

            do {
                val event = awaitPointerEvent()
                val change = event.changes.firstOrNull { it.id == down.id }
                val dragDistance = change?.position?.y?.minus(startY) ?: 0f

                if (
                    change != null &&
                    change.pressed &&
                    !isSearchVisible &&
                    hasMailAccessToken &&
                    isListAtTop &&
                    dragDistance > 72f
                ) {
                    showSearch(requestFocus = false)
                    searchStateChangedByGesture = true
                }
            } while (event.changes.any { it.pressed } && !searchStateChangedByGesture)
        }
    },
) {
    Column(
        modifier = Modifier.fillMaxSize(),
    ) {
        ChatTopBar(
            searchQuery = searchQuery,
            onSearchQueryChange = { searchQuery = it },
            isSearchVisible = isSearchVisible,
            searchFocusRequester = searchFocusRequester,
            onSearchClick = ::openSearchFromIcon,
            onSearchDismiss = ::dismissSearch,
            onSettingsClick = onSettingsClick,
            onProfileClick = onProfileClick,
            profileAvatarUrl = profileAvatarUrl,
        )

        if (hasMailAccessToken) {
            LazyColumn(

```

```

        state = listState,
        modifier = Modifier
            .weight(1f)
            .fillMaxWidth()
            .navigationBarsPadding(),
        contentPadding = PaddingValues(bottom = 24.dp),
    ) {
        items(
            items = filteredChats,
            key = { it.id },
        ) { chat ->
            ChatPreviewItem(
                chat = chat,
                enabled = true,
                onClick = { onChatClick(chat) },
            )
        }
    }
} else {
    MailAccessEmptyContent(
        isSetupRequired = isSetupRequired,
        onSetupTokenClick = onSetupTokenClick,
        modifier = Modifier
            .weight(1f)
            .fillMaxWidth()
            .navigationBarsPadding(),
    )
}
}
}
}
}
}

```

```

@Composable
private fun MailAccessEmptyContent(
    isSetupRequired: Boolean,
    onSetupTokenClick: () -> Unit,
    modifier: Modifier = Modifier,
) {
    Column(modifier = modifier) {
        if (isSetupRequired) {
            SetupTokenAlert(
                onSetupTokenClick = onSetupTokenClick,
                modifier = Modifier.padding(
                    start = 16.dp,
                    end = 16.dp,
                    top = 16.dp,
                    bottom = 12.dp,
                ),
            ),
        }
    }
}

```

```

MailAccessLockedNotice(
    modifier = Modifier.padding(
        start = 16.dp,
        end = 16.dp,
    ),
)

Box(
    modifier = Modifier
        .fillMaxWidth()
        .weight(1f),
    contentAlignment = Alignment.Center,
) {
    BlockedMailEmptyIcon(
        modifier = Modifier.padding(bottom = 42.dp),
    )
}
}
}

@Composable
private fun BlockedMailEmptyIcon(
    modifier: Modifier = Modifier,
) {
    val infiniteTransition = rememberInfiniteTransition(label = "BlockedMailEmptyIcon")
    val translationY by infiniteTransition.animateFloat(
        initialValue = -8f,
        targetValue = 8f,
        animationSpec = infiniteRepeatable(
            animation = tween(durationMillis = 2000, easing = FastOutSlowInEasing),
            repeatMode = RepeatMode.Reverse,
        ),
        label = "MailLevitation",
    )
    val glowScale by infiniteTransition.animateFloat(
        initialValue = 0.82f,
        targetValue = 1.18f,
        animationSpec = infiniteRepeatable(
            animation = tween(durationMillis = 2000, easing = LinearEasing),
            repeatMode = RepeatMode.Reverse,
        ),
        label = "MailGlowPulse",
    )

    Box(
        modifier = modifier
            .size(220.dp)
            .offset(y = translationY.dp),
        contentAlignment = Alignment.Center,
    ) {
        Box(

```

```

modifier = Modifier
    .size(128.dp * glowScale)
    .blur(30.dp)
    .background(
        brush = Brush.radialGradient(
            colors = listOf(
                PosterPrimary.copy(alpha = 0.42f),
                Color(0xFF00D2FF).copy(alpha = 0.14f),
                Color.Transparent,
            ),
        ),
        shape = CircleShape,
    ),
)

Box(
    modifier = Modifier
        .size(154.dp)
        .clip(CircleShape)
        .background(
            brush = Brush.verticalGradient(
                colors = listOf(
                    Color.White.copy(alpha = 0.075f),
                    PosterPrimary.copy(alpha = 0.08f),
                    Color.White.copy(alpha = 0.018f),
                ),
            ),
        )
        .border(
            width = 1.dp,
            color = PosterPrimary.copy(alpha = 0.34f),
            shape = CircleShape,
        ),
    contentAlignment = Alignment.Center,
) {
    Box(
        modifier = Modifier.size(82.dp),
        contentAlignment = Alignment.Center,
    ) {
        Icon(
            imageVector = Icons.Outlined.Email,
            contentDescription = "Mail access token is missing",
            tint = PosterPrimary.copy(alpha = 0.94f),
            modifier = Modifier.fillMaxSize(),
        )
    }
}
}
}
}

```

@Composable

```

private fun MailAccessLockedNotice(
    modifier: Modifier = Modifier,
) {
    Column(
        modifier = modifier
        .fillMaxWidth()
        .background(
            color = Color(0xFF100F1D),
            shape = RoundedCornerShape(14.dp),
        )
        .border(
            width = 1.dp,
            color = PosterStroke.copy(alpha = 0.75f),
            shape = RoundedCornerShape(14.dp),
        )
        .padding(horizontal = 18.dp, vertical = 16.dp),
    ) {
        Text(
            text = "Чаты пока грустят",
            color = PosterTextPrimary,
            fontSize = 16.sp,
            fontWeight = FontWeight.Bold,
        )

        Spacer(modifier = Modifier.height(7.dp))

        Text(
            text = "Без mail access token нельзя читать чаты и отправлять сообщения. Настрой токен, и Poster снова оживет.",
            color = PosterTextSecondary,
            fontSize = 14.sp,
            lineHeight = 20.sp,
        )
    }
}

```

```

@Composable
private fun ChatTopBar(
    searchQuery: String,
    onSearchQueryChange: (String) -> Unit,
    isSearchVisible: Boolean,
    searchFocusRequester: FocusRequester,
    onSearchClick: () -> Unit,
    onSearchDismiss: () -> Unit,
    onSettingsClick: () -> Unit,
    onProfileClick: () -> Unit,
    profileAvatarUrl: String?,
) {
    Column(
        modifier = Modifier
        .fillMaxWidth()

```

```

        .background(PosterTopBar)
        .statusBarsPadding()
        .padding(start = 16.dp, end = 14.dp, top = 16.dp, bottom = 12.dp),
    ) {
        Row(
            modifier = Modifier
                .fillMaxWidth()
                .height(40.dp),
            verticalAlignment = Alignment.CenterVertically,
        ) {
            AvatarCircle(
                text = "",
                size = 40.dp,
                showPersonIcon = true,
                imageUrl = profileAvatarUrl,
                modifier = Modifier.clickable { onProfileClick() },
            )

            Spacer(modifier = Modifier.width(12.dp))

            Text(
                text = "Poster",
                color = PosterTextPrimary,
                fontSize = 20.sp,
                fontWeight = FontWeight.Bold,
                modifier = Modifier.weight(1f),
            )

            IconButton(
                onClick = onSearchClick,
                modifier = Modifier
                    .size(40.dp),
            ) {
                Icon(
                    imageVector = Icons.Outlined.Search,
                    contentDescription = "Search",
                    tint = if (isSearchVisible) PosterIcon else PosterTextSecondary,
                    modifier = Modifier.size(24.dp),
                )
            }

            IconButton(
                onClick = onSettingsClick,
                modifier = Modifier.size(40.dp),
            ) {
                Icon(
                    imageVector = Icons.Outlined.Settings,
                    contentDescription = "Settings",
                    tint = PosterTextSecondary,
                    modifier = Modifier.size(24.dp),
                )
            }
        }
    }

```

```

    }
}

AnimatedVisibility(
    visible = isSearchVisible,
    enter = slideInVertically(
        animationSpec = tween(durationMillis = 220),
        initialOffsetY = { -it / 2 },
    ) + expandVertically(
        animationSpec = tween(durationMillis = 220),
        expandFrom = Alignment.Top,
    ) + fadeIn(animationSpec = tween(durationMillis = 160)),
    exit = shrinkVertically(
        animationSpec = tween(durationMillis = 180),
        shrinkTowards = Alignment.Top,
    ) + slideOutVertically(
        animationSpec = tween(durationMillis = 180),
        targetOffsetY = { -it / 2 },
    ) + fadeOut(animationSpec = tween(durationMillis = 120)),
) {
    SearchField(
        value = searchQuery,
        onValueChange = onSearchQueryChange,
        onDismiss = onSearchDismiss,
        focusRequester = searchFocusRequester,
        modifier = Modifier.padding(top = 12.dp),
    )
}
}
}
}

```

```

@Composable
private fun SearchField(
    value: String,
    onValueChange: (String) -> Unit,
    onDismiss: () -> Unit,
    focusRequester: FocusRequester,
    modifier: Modifier = Modifier,
) {
    Row(
        modifier = modifier
            .fillMaxWidth()
            .height(46.dp)
            .shadow(
                elevation = 18.dp,
                shape = RoundedCornerShape(16.dp),
            )
        .background(
            color = PosterSurface.copy(alpha = 0.96f),
            shape = RoundedCornerShape(16.dp),
        )
    )
}

```

```

        .border(
            width = 1.dp,
            color = PosterStroke.copy(alpha = 0.75f),
            shape = RoundedCornerShape(16.dp),
        )
        .padding(horizontal = 12.dp),
verticalAlignment = Alignment.CenterVertically,
) {
    Icon(
        imageVector = Icons.Outlined.Search,
        contentDescription = null,
        tint = PosterTextMuted,
        modifier = Modifier.size(22.dp),
    )
}

```

```

Spacer(modifier = Modifier.width(10.dp))

```

```

BasicTextField(
    value = value,
    onValueChange = onValueChange,
    singleLine = true,
    cursorBrush = SolidColor(PosterPrimary),
    textStyle = TextStyle(
        color = PosterTextPrimary,
        fontSize = 16.sp,
    ),
    decorationBox = { innerTextField ->
        Box(
            modifier = Modifier.fillMaxWidth(),
            contentAlignment = Alignment.CenterStart,
        ) {
            if (value.isEmpty()) {
                Text(
                    text = "Search messages",
                    color = PosterTextMuted,
                    fontSize = 16.sp,
                )
            }

            innerTextField()
        }
    },
    modifier = Modifier
        .weight(1f)
        .focusRequester(focusRequester),
)

```

```

IconButton(
    onClick = {
        if (value.isBlank()) {
            onDismiss()
        }
    }
)

```

```

        } else {
            onChange("")
        }
    },
    modifier = Modifier.size(32.dp),
) {
    Icon(
        imageVector = Icons.Outlined.Close,
        contentDescription = "Close search",
        tint = PosterTextMuted,
        modifier = Modifier.size(20.dp),
    )
}
}
}

```

```

@Composable
private fun SetupTokenAlert(
    onSetupTokenClick: () -> Unit,
    modifier: Modifier = Modifier,
) {
    Row(
        modifier = modifier
            .height(74.dp)
            .fillMaxWidth()
            .background(
                color = Color(0xFF15142D),
                shape = RoundedCornerShape(14.dp),
            )
            .border(
                width = 1.dp,
                color = PosterPrimary.copy(alpha = 0.65f),
                shape = RoundedCornerShape(14.dp),
            )
            .padding(start = 16.dp, end = 16.dp),
        verticalAlignment = Alignment.CenterVertically,
    ) {
        Icon(
            imageVector = Icons.Outlined.Info,
            contentDescription = null,
            tint = PosterIcon,
            modifier = Modifier.size(22.dp),
        )

        Spacer(modifier = Modifier.width(12.dp))

        Text(
            text = "Setup required to receive messages",
            color = PosterTextPrimary,
            fontSize = 16.sp,
            modifier = Modifier.weight(1f),
        )
    }
}

```

```

        maxLines = 1,
        overflow = TextOverflow.Ellipsis,
    )

    Spacer(modifier = Modifier.width(12.dp))

    Box(
        modifier = Modifier
            .height(44.dp)
            .shadow(
                elevation = 14.dp,
                shape = RoundedCornerShape(10.dp),
            )
            .background(
                brush = Brush.horizontalGradient(
                    listOf(PosterPrimary, PosterPrimaryDark),
                ),
                shape = RoundedCornerShape(10.dp),
            )
            .clickable { onSetupTokenClick() }
            .padding(horizontal = 16.dp),
        contentAlignment = Alignment.Center,
    ) {
        Text(
            text = "Setup Token",
            color = Color.White,
            fontSize = 15.sp,
            fontWeight = FontWeight.Bold,
        )
    }
}

```

```

@Composable
private fun ChatPreviewItem(
    chat: ChatPreviewUi,
    enabled: Boolean = true,
    onClick: () -> Unit,
) {
    Row(
        modifier = Modifier
            .fillMaxWidth()
            .height(84.dp)
            .clickable(enabled = enabled) { onClick() }
            .padding(horizontal = 16.dp),
        verticalAlignment = Alignment.CenterVertically,
    ) {
        AvatarCircle(
            text = chat.initials,
            size = 48.dp,
        )
    }
}

```

```
Spacer(modifier = Modifier.width(12.dp))
```

```
Column(  
    modifier = Modifier  
        .weight(1f)  
        .padding(end = 8.dp),  
) {  
    Text(  
        text = chat.name,  
        color = if (enabled) PosterTextPrimary else PosterTextMuted,  
        fontSize = 18.sp,  
        fontWeight = FontWeight.Bold,  
        maxLines = 1,  
        overflow = TextOverflow.Ellipsis,  
    )  
}
```

```
Spacer(modifier = Modifier.height(6.dp))
```

```
Text(  
    text = if (enabled) {  
        chat.lastMessage  
    } else {  
        "Недоступно без mail access token"  
    },  
    color = if (enabled) PosterTextSecondary else PosterTextMuted,  
    fontSize = 14.sp,  
    maxLines = 1,  
    overflow = TextOverflow.Ellipsis,  
)  
}
```

```
Column(  
    horizontalAlignment = Alignment.End,  
    verticalArrangement = Arrangement.Center,  
) {  
    Text(  
        text = if (enabled) chat.time else "--",  
        color = PosterTextMuted,  
        fontSize = 13.sp,  
    )  
}
```

```
if (enabled && chat.unreadCount > 0) {  
    Spacer(modifier = Modifier.height(14.dp))
```

```
Box(  
    modifier = Modifier  
        .size(20.dp)  
        .background(  
            color = PosterPrimary,  
            shape = CircleShape,
```

```

        ),
        contentAlignment = Alignment.Center,
    ) {
        Text(
            text = chat.unreadCount.toString(),
            color = Color.White,
            fontSize = 12.sp,
            fontWeight = FontWeight.Bold,
        )
    }
}
}
}

Box(
    modifier = Modifier
        .fillMaxWidth()
        .height(1.dp)
        .background(PosterDivider.copy(alpha = 0.75f)),
)
}

```

```

@Composable
private fun AvatarCircle(
    text: String,
    size: Dp,
    showPersonIcon: Boolean = false,
    imageUrl: String? = null,
    modifier: Modifier = Modifier,
) {
    Box(
        modifier = modifier
            .size(size)
            .background(
                brush = Brush.linearGradient(
                    colors = listOf(
                        PosterPrimary,
                        Color(0xFF3D39B8),
                    ),
                ),
                shape = CircleShape,
            ),
        contentAlignment = Alignment.Center,
    ) {
        if (imageUrl != null) {
            AsyncImage(
                model = imageUrl,
                contentDescription = null,
                modifier = Modifier
                    .fillMaxSize()
                    .clip(CircleShape),
            )
        }
    }
}

```

```

        contentScale = ContentScale.Crop,
    )
} else if (showPersonIcon) {
    Icon(
        imageVector = Icons.Outlined.Person,
        contentDescription = null,
        tint = Color.White,
        modifier = Modifier.size(22.dp),
    )
} else {
    Text(
        text = text,
        color = Color.White,
        fontSize = 15.sp,
        fontWeight = FontWeight.Medium,
    )
}
}
}

```

```

@Preview(showBackground = true, widthDp = 393, heightDp = 852)
@Composable
private fun ChatListScreenPreview() {
    ChatListScreen(
        chats = emptyList(),
        isSetupRequired = true,
        hasMailAccessToken = false,
        onChatClick = {},
        onSetupTokenClick = {},
        onSettingsClick = {},
        onProfileClick = {},
    )
}

```

Листинг 98 – Код файла app/src/main/java/com/example/poster/presentation/chats/ChatsScreen.kt

```
package com.example.poster.presentation.chats

import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.fillMaxSize

@Composable
fun ChatsScreen(
    chats: List<ChatPreviewUi> = emptyList(),
    isSetupRequired: Boolean = true,
    hasMailAccessToken: Boolean = !isSetupRequired,
    profileAvatarUrl: String? = null,
    onChatClick: (ChatPreviewUi) -> Unit = {},
    onSetupTokenClick: () -> Unit = {},
    onSettingsClick: () -> Unit = {},
    onProfileClick: () -> Unit = {},
    modifier: Modifier = Modifier,
) {
    Box(
        modifier = modifier.fillMaxSize(),
    ) {
        ChatListScreen(
            chats = chats,
            isSetupRequired = isSetupRequired,
            hasMailAccessToken = hasMailAccessToken,
            profileAvatarUrl = profileAvatarUrl,
            onChatClick = onChatClick,
            onSetupTokenClick = onSetupTokenClick,
            onSettingsClick = onSettingsClick,
            onProfileClick = onProfileClick,
        )
    }
}
```

Листинг 99 – Код файла `app/src/main/java/com/example/poster/presentation/common/FeatureStubScreen.kt`

```
package com.example.poster.presentation.common

import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp

@Composable
fun FeatureStubScreen(
    title: String,
    description: String,
    checkpoints: List<String>,
    modifier: Modifier = Modifier,
) {
    Column(
        modifier = modifier
            .fillMaxSize()
            .padding(16.dp),
        verticalArrangement = Arrangement.spacedBy(16.dp),
    ) {
        Text(
            text = title,
            style = MaterialTheme.typography.headlineMedium,
        )

        Text(
            text = description,
            style = MaterialTheme.typography.bodyLarge,
        )

        Surface(
            modifier = Modifier.fillMaxWidth(),
            shape = MaterialTheme.shapes.large,
            tonalElevation = 2.dp,
        ) {
            Column(
                modifier = Modifier.padding(16.dp),
                verticalArrangement = Arrangement.spacedBy(8.dp),
            ) {
                checkpoints.forEach { checkpoint ->
                    Text(

```

```
        text = "• $checkpoint",
        style = MaterialTheme.typography.bodyMedium,
    )
}
}
}
```

Листинг 100 – Код файла `app/src/main/java/com/example/poster/presentation/messages/MessagesScreen.kt`

```
package com.example.poster.presentation.messages

import android.net.Uri
import androidx.activity.compose.rememberLauncherForActivityResult
import androidx.activity.result.contract.ActivityResultContracts
import androidx.compose.animation.core.Animatable
import androidx.compose.animation.core.Spring
import androidx.compose.animation.core.spring
import androidx.compose.animation.core.tween
import androidx.compose.foundation.Canvas
import androidx.compose.foundation.background
import androidx.compose.foundation.border
import androidx.compose.foundation.clickable
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.PaddingValues
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.heightIn
import androidx.compose.foundation.layout.imePadding
import androidx.compose.foundation.layout.navigationBarsPadding
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.layout.statusBarsPadding
import androidx.compose.foundation.layout.width
import androidx.compose.foundation.layout.widthIn
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.foundation.lazy.rememberLazyListState
import androidx.compose.foundation.shape.CircleShape
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.foundation.text.BasicTextField
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.outlined.ArrowBack
import androidx.compose.material.icons.outlined.AttachFile
import androidx.compose.material.icons.outlined.Description
import androidx.compose.material.icons.outlined.Download
import androidx.compose.material.icons.outlined.Image
import androidx.compose.material.icons.outlined.MoreVert
import androidx.compose.material.icons.outlined.Send
import androidx.compose.material3.Icon
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.LaunchedEffect
```

```
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.draw.shadow
import androidx.compose.ui.graphics.Brush
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.SolidColor
import androidx.compose.ui.graphics.TransformOrigin
import androidx.compose.ui.graphics.drawscope.Stroke
import androidx.compose.ui.graphics.graphicsLayer
import androidx.compose.ui.layout.ContentScale
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.text.TextStyle
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.style.TextOverflow
import androidx.compose.ui.tooling.preview.Preview
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import coil.compose.AsyncImage
import com.example.poster.core.media.RemoteImageConfig
import kotlinx.coroutines.launch
```

```
private val PosterBackground = Color(0xFF030306)
private val PosterTopBar = Color(0xFF101014)
private val PosterSurface = Color(0xFF1B1A31)
private val PosterStroke = Color(0xFF302B68)
private val PosterPrimary = Color(0xFF625BFF)
private val PosterPrimaryDark = Color(0xFF493CCB)
private val PosterTextPrimary = Color(0xFFFF7F7FF)
private val PosterTextSecondary = Color(0xFFA5A6BA)
private val PosterTextMuted = Color(0xFF7B7D92)
private val PosterDivider = Color(0xFF1B1A25)
private val PosterIcon = Color(0xFF7F8BFF)
```

```
@Composable
fun MessagesScreen(
    contactName: String = "Alice Johnson",
    contactInitials: String = "AJ",
    messages: List<MessageUi> = emptyList(),
    canSendMessages: Boolean = true,
    onBackClick: () -> Unit = {},
    onProfileClick: () -> Unit = {},
    onMoreClick: () -> Unit = {},
    onSendClick: (String) -> Unit = {},
    onFilePicked: (Uri) -> Unit = {},
    onImagePicked: (Uri) -> Unit = {},
) {
```

```

var input by remember { mutableStateOf("") }
val listState = rememberLazyListState()
val lastMessageId = messages.lastOrNull()?.id

val filePickerLauncher = rememberLauncherForActivityResult(
    contract = ActivityResultContracts.OpenDocument(),
) { uri ->
    uri ?: return@rememberLauncherForActivityResult

    onFilePicked(uri)
}

val imagePickerLauncher = rememberLauncherForActivityResult(
    contract = ActivityResultContracts.OpenDocument(),
) { uri ->
    uri ?: return@rememberLauncherForActivityResult

    onImagePicked(uri)
}

LaunchedEffect(lastMessageId) {
    if (messages.isNotEmpty()) {
        listState.animateScrollToItem(messages.lastIndex)
    }
}

Box(
    modifier = Modifier
        .fillMaxSize()
        .background(
            Brush.radialGradient(
                colors = listOf(
                    Color(0xFF08051A),
                    PosterBackground,
                    Color(0xFF030306),
                ),
                radius = 1200f,
            )
        ),
) {
    ChatWallpaper(modifier = Modifier.fillMaxSize())

    Column(modifier = Modifier.fillMaxSize()) {
        MessagesTopBar(
            contactName = contactName,
            contactInitials = contactInitials,
            onBackClick = onBackClick,
            onProfileClick = onProfileClick,
            onMoreClick = onMoreClick,
        )
    }
}

```

```

LazyColumn(
    state = listState,
    modifier = Modifier
        .weight(1f)
        .fillMaxWidth(),
    contentPadding = PaddingValues(
        start = 16.dp,
        end = 16.dp,
        top = 18.dp,
        bottom = 18.dp,
    ),
) {
    items(
        items = messages,
        key = { it.id },
    ) { message ->
        MessageBubble(message = message)
        Spacer(modifier = Modifier.height(18.dp))
    }
}

```

```

MessageInputBar(
    value = input,
    onValueChange = { input = it },
    enabled = canSendMessages,
    onAttachFileClick = {
        if (canSendMessages) {
            filePickerLauncher.launch(
                arrayOf(
                    "application/pdf",
                    "application/msword",
                    "application/vnd.openxmlformats-officedocument.wordprocessingml.document",
                    "text/plain",
                    "application/zip",
                ),
            )
        }
    },
    onAttachImageClick = {
        if (canSendMessages) {
            imagePickerLauncher.launch(arrayOf("image/*"))
        }
    },
    onSendClick = {
        val text = input.trim()
        if (canSendMessages && text.isNotEmpty()) {
            onSendClick(text)
            input = ""
        }
    },
),

```

```

    }
  }
}

```

```

@Composable
private fun ChatWallpaper(
    modifier: Modifier = Modifier,
) {
    Canvas(modifier = modifier) {
        val width = size.width
        val height = size.height
        val dotColor = Color(0xFF403A92).copy(alpha = 0.16f)
        val ringColor = Color(0xFF625BFF).copy(alpha = 0.08f)

        drawCircle(
            brush = Brush.radialGradient(
                colors = listOf(
                    Color(0xFF625BFF).copy(alpha = 0.22f),
                    Color.Transparent,
                ),
                center = androidx.compose.ui.geometry.Offset(width * 0.22f, height * 0.16f),
                radius = width * 0.7f,
            ),
            radius = width * 0.7f,
            center = androidx.compose.ui.geometry.Offset(width * 0.22f, height * 0.16f),
        )

        drawCircle(
            brush = Brush.radialGradient(
                colors = listOf(
                    Color(0xFF21146A).copy(alpha = 0.32f),
                    Color.Transparent,
                ),
                center = androidx.compose.ui.geometry.Offset(width * 0.92f, height * 0.78f),
                radius = width * 0.82f,
            ),
            radius = width * 0.82f,
            center = androidx.compose.ui.geometry.Offset(width * 0.92f, height * 0.78f),
        )

        val spacing = 42.dp.toPx()
        var y = spacing
        var row = 0
        while (y < height) {
            var x = if (row % 2 == 0) spacing * 0.5f else spacing
            while (x < width) {
                drawCircle(
                    color = dotColor,
                    radius = 1.35.dp.toPx(),
                    center = androidx.compose.ui.geometry.Offset(x, y),
                )
            }
            row++
            y += spacing
        }
    }
}

```

```

        )
        x += spacing
    }
    row += 1
    y += spacing
}

listOf(
    androidx.compose.ui.geometry.Offset(width * 0.78f, height * 0.22f) to 44.dp.toPx(),
    androidx.compose.ui.geometry.Offset(width * 0.18f, height * 0.72f) to 58.dp.toPx(),
    androidx.compose.ui.geometry.Offset(width * 0.68f, height * 0.58f) to 32.dp.toPx(),
).forEach { (center, radius) ->
    drawCircle(
        color = ringColor,
        radius = radius,
        center = center,
        style = Stroke(width = 1.1.dp.toPx()),
    )
}
}
}
}

```

```

@Composable
private fun MessagesTopBar(
    contactName: String,
    contactInitials: String,
    onBackClick: () -> Unit,
    onProfileClick: () -> Unit,
    onMoreClick: () -> Unit,
) {
    Column(
        modifier = Modifier
            .fillMaxWidth()
            .background(PosterTopBar)
            .statusBarsPadding()
            .padding(start = 12.dp, end = 10.dp, top = 8.dp, bottom = 10.dp),
    ) {
        Row(
            modifier = Modifier
                .fillMaxWidth()
                .heightIn(min = 52.dp),
            verticalAlignment = Alignment.CenterVertically,
        ) {
            Icon(
                imageVector = Icons.Outlined.ArrowBack,
                contentDescription = "Back",
                tint = PosterTextSecondary,
                modifier = Modifier
                    .size(24.dp)
                    .clickable { onBackClick() },
            )
        }
    }
}

```

```
Spacer(modifier = Modifier.width(10.dp))
```

```
Box(  
    modifier = Modifier  
        .size(34.dp)  
        .background(  
            brush = Brush.linearGradient(  
                listOf(PosterPrimary, PosterPrimaryDark),  
            ),  
            shape = CircleShape,  
        )  
        .clickable { onProfileClick() },  
    contentAlignment = Alignment.Center,  
) {  
    Text(  
        text = contactInitials,  
        color = Color.White,  
        fontSize = 12.sp,  
        fontWeight = FontWeight.SemiBold,  
    )  
}
```

```
Spacer(modifier = Modifier.width(10.dp))
```

```
Column(  
    modifier = Modifier  
        .weight(1f)  
        .clickable { onProfileClick() },  
) {  
    Text(  
        text = contactName,  
        color = PosterTextPrimary,  
        fontSize = 18.sp,  
        fontWeight = FontWeight.Bold,  
        maxLines = 1,  
        overflow = TextOverflow.Ellipsis,  
    )  
}
```

```
Spacer(modifier = Modifier.height(2.dp))
```

```
Text(  
    text = "Online",  
    color = PosterTextSecondary,  
    fontSize = 12.sp,  
)  
}
```

```
Icon(  
    imageVector = Icons.Outlined.MoreVert,  
    contentDescription = "More",  
)
```

```

        tint = PosterTextSecondary,
        modifier = Modifier
            .size(22.dp)
            .clickable { onMoreClick() },
    )
}

Box(
    modifier = Modifier
        .fillMaxWidth()
        .padding(top = 8.dp)
        .height(1.dp)
        .background(PosterDivider),
)
}
}

@Composable
private fun MessageBubble(
    message: MessageUi,
    modifier: Modifier = Modifier,
) {
    val alpha = remember(message.id) { Animatable(0f) }
    val scale = remember(message.id) { Animatable(0.94f) }

    LaunchedEffect(message.id) {
        launch {
            alpha.animateTo(
                targetValue = 1f,
                animationSpec = tween(durationMillis = 180),
            )
        }
        launch {
            scale.animateTo(
                targetValue = 1f,
                animationSpec = spring(
                    dampingRatio = Spring.DampingRatioMediumBouncy,
                    stiffness = Spring.StiffnessMediumLow,
                ),
            )
        }
    }
}

Row(
    modifier = modifier
        .fillMaxWidth()
        .graphicsLayer {
            this.alpha = alpha.value
            scaleX = scale.value
            scaleY = scale.value
            transformOrigin = TransformOrigin(

```

```

        pivotFractionX = if (message.isMine) 1f else 0f,
        pivotFractionY = 1f,
    )
    },
    horizontalArrangement = if (message.isMine) Arrangement.End else Arrangement.Start,
) {
    Box(
        modifier = Modifier.widthIn(
            max = if (message.type == MessageContentType.IMAGE) 190.dp else 420.dp,
        ),
    ) {
        when (message.type) {
            MessageContentType.TEXT -> TextMessageBubble(message = message)
            MessageContentType.FILE -> FileMessageBubble(message = message)
            MessageContentType.IMAGE -> ImageMessageBubble(message = message)
        }
    }
}

```

```

@Composable
private fun TextMessageBubble(
    message: MessageUi,
) {
    Column(
        modifier = Modifier
            .background(
                color = if (message.isMine) PosterPrimary else PosterSurface,
                shape = RoundedCornerShape(
                    topStart = 14.dp,
                    topEnd = 14.dp,
                    bottomStart = if (message.isMine) 14.dp else 5.dp,
                    bottomEnd = if (message.isMine) 5.dp else 14.dp,
                ),
            )
        .padding(start = 16.dp, end = 16.dp, top = 13.dp, bottom = 11.dp),
    ) {
        Text(
            text = message.text,
            color = PosterTextPrimary,
            fontSize = 16.sp,
            fontWeight = FontWeight.SemiBold,
            lineHeight = 21.sp,
        )

        Spacer(modifier = Modifier.height(10.dp))

        Text(
            text = message.time,
            color = if (message.isMine) Color(0xFFE0DFFF) else PosterTextMuted,
            fontSize = 12.sp,

```

```

    )
  }
}

```

@Composable

private fun FileMessageBubble(

message: MessageUi,

) {

val attachment = message.attachment ?: return

Column(

modifier = Modifier

.background(

color = if (message.isMine) PosterPrimary else PosterSurface,

shape = RoundedCornerShape(

topStart = 14.dp,

topEnd = 14.dp,

bottomStart = if (message.isMine) 14.dp else 5.dp,

bottomEnd = if (message.isMine) 5.dp else 14.dp,

),

)

.padding(14.dp),

) {

Row(

verticalAlignment = Alignment.CenterVertically,

modifier = Modifier.widthIn(min = 310.dp, max = 390.dp),

) {

Box(

modifier = Modifier

.size(48.dp)

.background(

color = if (message.isMine) {

Color.White.copy(alpha = 0.22f)

} else {

PosterPrimary.copy(alpha = 0.18f)

},

shape = RoundedCornerShape(10.dp),

),

contentAlignment = Alignment.Center,

) {

Icon(

imageVector = Icons.Outlined.Description,

contentDescription = null,

tint = Color.White,

modifier = Modifier.size(26.dp),

)

}

Spacer(modifier = Modifier.width(12.dp))

Column(modifier = Modifier.weight(1f)) {

```

        Text(
            text = attachment.fileName,
            color = Color.White,
            fontSize = 15.sp,
            fontWeight = FontWeight.Bold,
            maxLines = 1,
            overflow = TextOverflow.Ellipsis,
        )

        Spacer(modifier = Modifier.height(4.dp))

        Text(
            text = attachment.fileSize,
            color = if (message.isMine) Color(0xFFE0DFFF) else PosterTextMuted,
            fontSize = 12.sp,
        )
    }

    Spacer(modifier = Modifier.width(12.dp))

    Icon(
        imageVector = Icons.Outlined.Download,
        contentDescription = "Download",
        tint = Color.White,
        modifier = Modifier.size(22.dp),
    )
}

Spacer(modifier = Modifier.height(12.dp))

Text(
    text = message.time,
    color = if (message.isMine) Color(0xFFE0DFFF) else PosterTextMuted,
    fontSize = 12.sp,
)
}
}

@Composable
private fun ImageMessageBubble(
    message: MessageUi,
) {
    val attachment = message.attachment ?: return
    val context = LocalContext.current
    val imageModel = remember(context, attachment.localUri, attachment.remoteUri) {
        attachment.localUri ?: attachment.remoteUri?.let { remoteUri ->
            RemoteImageConfig.buildImageModel(
                context = context,
                remoteUri = remoteUri,
            )
        }
    }
}

```

```

    }
    val bubbleShape = RoundedCornerShape(
        topStart = 14.dp,
        topEnd = 14.dp,
        bottomStart = if (message.isMine) 14.dp else 5.dp,
        bottomEnd = if (message.isMine) 5.dp else 14.dp,
    )

    Column(
        modifier = Modifier
            .background(
                color = if (message.isMine) PosterPrimary else PosterSurface,
                shape = bubbleShape,
            )
            .clip(bubbleShape),
    ) {
        if (imageModel != null) {
            AsyncImage(
                model = imageModel,
                contentDescription = attachment.fileName,
                modifier = Modifier
                    .width(190.dp)
                    .height(255.dp),
                contentScale = ContentScale.Crop,
            )
        } else {
            Box(
                modifier = Modifier
                    .width(190.dp)
                    .height(255.dp)
                    .background(PosterSurface),
                contentAlignment = Alignment.Center,
            ) {
                Icon(
                    imageVector = Icons.Outlined.Image,
                    contentDescription = null,
                    tint = PosterTextMuted,
                    modifier = Modifier.size(36.dp),
                )
            }
        }
    }

    Box(
        modifier = Modifier
            .fillMaxWidth()
            .background(if (message.isMine) PosterPrimary else PosterSurface)
            .padding(horizontal = 14.dp, vertical = 10.dp),
    ) {
        Text(
            text = message.time,
            color = if (message.isMine) Color(0xFFE0DFFF) else PosterTextMuted,

```

```

        fontSize = 12.sp,
    )
}
}
}

```

@Composable

private fun MessageInputBar(

```

    value: String,
    onValueChange: (String) -> Unit,
    onAttachFileClick: () -> Unit,
    onAttachImageClick: () -> Unit,
    onSendClick: () -> Unit,
    enabled: Boolean = true,

```

) {

```

    val canSend = enabled && value.trim().isEmpty()
    var sendPulse by remember { mutableStateOf(0) }
    val sendScale = remember { Animatable(1f) }

```

```

    LaunchedEffect(sendPulse) {
        if (sendPulse == 0) {
            return@LaunchedEffect
        }
    }

```

```

    sendScale.snapTo(0.84f)
    sendScale.animateTo(
        targetValue = 1f,
        animationSpec = spring(
            dampingRatio = Spring.DampingRatioMediumBouncy,
            stiffness = Spring.StiffnessMedium,
        ),
    )
}

```

Column(

```

    modifier = Modifier
        .fillMaxWidth()
        .imePadding()
        .navigationBarPadding()
        .padding(start = 16.dp, end = 16.dp, top = 8.dp, bottom = 14.dp),

```

) {

```

    if (!enabled) {

```

```

        Text(

```

```

            text = "Сообщения нельзя отправлять без mail access token. Немного грустно, но так безопаснее.",

```

```

            color = PosterTextMuted,

```

```

            fontSize = 13.sp,

```

```

            lineHeight = 18.sp,

```

```

            modifier = Modifier.padding(bottom = 10.dp),

```

```

        )

```

```

    }

```

```

Row(
  modifier = Modifier
    .fillMaxWidth()
    .height(54.dp),
  verticalAlignment = Alignment.CenterVertically,
) {
  FloatingInputAction(
    imageVector = Icons.Outlined.AttachFile,
    contentDescription = "Attach file",
    enabled = enabled,
    onClick = onAttachFileClick,
  )

  Spacer(modifier = Modifier.width(8.dp))

  FloatingInputAction(
    imageVector = Icons.Outlined.Image,
    contentDescription = "Attach image",
    enabled = enabled,
    onClick = onAttachImageClick,
  )

  Spacer(modifier = Modifier.width(10.dp))

  BasicTextField(
    value = value,
    onValueChange = onValueChange,
    enabled = enabled,
    singleLine = true,
    cursorBrush = SolidColor(PosterPrimary),
    textStyle = TextStyle(
      color = if (enabled) PosterTextPrimary else PosterTextMuted,
      fontSize = 16.sp,
    ),
    decorationBox = { innerTextField ->
      Box(
        modifier = Modifier
          .fillMaxWidth()
          .height(50.dp)
          .shadow(
            elevation = 22.dp,
            shape = RoundedCornerShape(25.dp),
            spotColor = PosterPrimary.copy(alpha = 0.14f),
          )
          .background(
            color = PosterSurface.copy(alpha = 0.9f),
            shape = RoundedCornerShape(25.dp),
          )
          .border(
            width = 1.dp,

```

```

        color = PosterStroke.copy(alpha = 0.55f),
        shape = RoundedCornerShape(25.dp),
    )
    .padding(horizontal = 20.dp),
    contentAlignment = Alignment.CenterStart,
) {
    if (value.isEmpty()) {
        Text(
            text = if (enabled) "Type a message..." else "Token required...",
            color = PosterTextMuted,
            fontSize = 16.sp,
        )
    }

    innerTextField()
}
},
modifier = Modifier.weight(1f),
)

```

```

Spacer(modifier = Modifier.width(12.dp))

```

```

Box(
    modifier = Modifier
        .size(50.dp)
        .graphicsLayer {
            scaleX = sendScale.value
            scaleY = sendScale.value
        }
        .shadow(
            elevation = if (canSend) 24.dp else 0.dp,
            shape = CircleShape,
            spotColor = PosterPrimary.copy(alpha = 0.32f),
        )
        .background(
            brush = Brush.linearGradient(
                if (canSend) {
                    listOf(PosterPrimary, PosterPrimaryDark)
                } else {
                    listOf(Color(0xFF312A7A), Color(0xFF261D65))
                }
            ),
            shape = CircleShape,
        )
        .clickable(enabled = canSend) {
            sendPulse += 1
            onSendClick()
        },
    contentAlignment = Alignment.Center,
) {
    Icon(

```

```

        imageVector = Icons.Outlined.Send,
        contentDescription = "Send",
        tint = if (canSend) Color.White else PosterTextMuted,
        modifier = Modifier.size(23.dp),
    )
}
}
}
}

```

```

@Composable
private fun FloatingInputAction(
    imageVector: androidx.compose.ui.graphics.vector.ImageVector,
    contentDescription: String,
    enabled: Boolean,
    onClick: () -> Unit,
) {
    Box(
        modifier = Modifier
            .size(46.dp)
            .shadow(
                elevation = 18.dp,
                shape = CircleShape,
                spotColor = PosterPrimary.copy(alpha = 0.16f),
            )
            .background(
                color = PosterSurface.copy(alpha = 0.92f),
                shape = CircleShape,
            )
            .border(
                width = 1.dp,
                color = PosterStroke.copy(alpha = 0.55f),
                shape = CircleShape,
            )
            .clickable(enabled = enabled) { onClick() },
        contentAlignment = Alignment.Center,
    ) {
        Icon(
            imageVector = imageVector,
            contentDescription = contentDescription,
            tint = if (enabled) PosterIcon else PosterTextMuted,
            modifier = Modifier.size(22.dp),
        )
    }
}

```

```

@Preview(showBackground = true, widthDp = 393, heightDp = 852)
@Composable
private fun MessagesScreenPreview() {
    MessagesScreen(
        contactName = "Alice Johnson",
    )
}

```

```
        contactInitials = "AJ",
        messages = emptyList(),
        onBackClick = {},
        onProfileClick = {},
        onMoreClick = {},
        onSendClick = {},
    )
}
```

Листинг 101 – Код файла `app/src/main/java/com/example/poster/presentation/messages/MessageUi.kt`

```
package com.example.poster.presentation.messages

import com.example.poster.domain.model.Attachment
import com.example.poster.domain.model.AttachmentType
import com.example.poster.domain.model.Message
import java.util.Locale

enum class MessageContentType {
    TEXT,
    FILE,
    IMAGE,
}

data class MessageAttachmentUi(
    val id: String,
    val fileName: String,
    val fileSize: String,
    val mimeType: String,
    val sizeBytes: Long = 0L,
    val localUri: String? = null,
    val remoteUrl: String? = null,
)

data class MessageUi(
    val id: String,
    val text: String = "",
    val time: String,
    val isMine: Boolean,
    val type: MessageContentType = MessageContentType.TEXT,
    val attachment: MessageAttachmentUi? = null,
)

fun Message.toMessageUi(): MessageUi {
    val firstAttachment = attachments.firstOrNull()
    return MessageUi(
        id = id,
        text = text,
        time = time,
        isMine = isMine,
        type = when (firstAttachment?.type) {
            AttachmentType.IMAGE -> MessageContentType.IMAGE
            null -> MessageContentType.TEXT
            else -> MessageContentType.FILE
        },
        attachment = firstAttachment?.toMessageAttachmentUi(),
    )
}
```

```

fun Attachment.toMessageAttachmentUi(): MessageAttachmentUi {
    return MessageAttachmentUi(
        id = id,
        fileName = fileName,
        fileSize = sizeBytes.toReadableFileSize(),
        sizeBytes = sizeBytes,
        mimeType = mimeType,
        localUri = localUri,
        remoteUrl = remoteUrl,
    )
}

```

```

fun Long.toReadableFileSize(): String {
    if (this <= 0L) {
        return "0 B"
    }

    val units = listOf("B", "KB", "MB", "GB")
    var value = toDouble()
    var index = 0

    while (value >= 1024 && index < units.lastIndex) {
        value /= 1024
        index += 1
    }

    return String.format(Locale.US, "%.1f %s", value, units[index])
}

```

Листинг 102 – Код файла `app/src/main/java/com/example/poster/presentation/navigation/PosterApp.kt`

```
package com.example.poster.presentation.navigation

import androidx.compose.foundation.horizontalScroll
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.rememberScrollState
import androidx.compose.material3.ExperimentalMaterial3Api
import androidx.compose.material3.HorizontalDivider
import androidx.compose.material3.NavigationBar
import androidx.compose.material3.NavigationBarItem
import androidx.compose.material3.OutlinedButton
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Text
import androidx.compose.material3.TopAppBar
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.saveable.rememberSaveable
import androidx.compose.runtime.setValue
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp
import com.example.poster.presentation.auth.login.LoginScreen
import com.example.poster.presentation.auth.otp.OtpScreen
import com.example.poster.presentation.auth.register.RegisterScreen
import com.example.poster.presentation.chats.ChatsScreen
import com.example.poster.presentation.messages.MessagesScreen
import com.example.poster.presentation.profile.ProfileScreen
import com.example.poster.presentation.settings.SettingsScreen

private val authDestinations = listOf(
    PosterDestination.LOGIN,
    PosterDestination.REGISTER,
    PosterDestination.OTP,
)

private val mainDestinations = listOf(
    PosterDestination.CHATS,
    PosterDestination.MESSAGES,
    PosterDestination.PROFILE,
    PosterDestination.SETTINGS,
)

@Composable
```

```

@OptIn(ExperimentalMaterial3Api::class)
fun PosterApp() {
    var currentDestination by rememberSaveable {
        mutableStateOf(PosterDestination.CHATS)
    }

    Scaffold(
        topBar = {
            TopAppBar(
                title = {
                    Text(text = currentDestination.title)
                }
            )
        },
        bottomBar = {
            NavigationBar {
                mainDestinations.forEach { destination ->
                    NavigationBarItem(
                        selected = currentDestination == destination,
                        onClick = { currentDestination = destination },
                        icon = { Text(destination.title.take(1)) },
                        label = { Text(destination.title) },
                    )
                }
            }
        },
    ) { innerPadding ->
        Column(
            modifier = Modifier
                .fillMaxSize()
                .padding(innerPadding),
        ) {
            AuthShortcutRow(
                currentDestination = currentDestination,
                onDestinationSelected = { currentDestination = it },
            )

            HorizontalDivider()

            Box(
                modifier = Modifier.fillMaxSize(),
            ) {
                when (currentDestination) {
                    PosterDestination.LOGIN -> LoginScreen()
                    PosterDestination.REGISTER -> RegisterScreen()
                    PosterDestination.OTP -> OtpScreen()
                    PosterDestination.CHATS -> ChatsScreen()
                    PosterDestination.MESSAGES -> MessagesScreen()
                    PosterDestination.PROFILE -> ProfileScreen()
                    PosterDestination.SETTINGS -> SettingsScreen()
                }
            }
        }
    }
}

```

```

    }
  }
}

```

```

@Composable
private fun AuthShortcutRow(
    currentDestination: PosterDestination,
    onDestinationSelected: (PosterDestination) -> Unit,
) {
    Row(
        modifier = Modifier
            .fillMaxWidth()
            .horizontalScroll(rememberScrollState())
            .padding(horizontal = 16.dp, vertical = 12.dp),
        horizontalArrangement = Arrangement.spacedBy(8.dp),
    ) {
        authDestinations.forEach { destination ->
            OutlinedButton(
                onClick = { onDestinationSelected(destination) },
                enabled = currentDestination != destination,
            ) {
                Text(text = destination.title)
            }
        }
    }
}

```

Листинг 103 – Код файла `app/src/main/java/com/example/poster/presentation/navigation/PosterDestination.kt`

```
package com.example.poster.presentation.navigation
```

```
enum class PosterDestination(val title: String) {  
    LOGIN("Login"),  
    REGISTER("Register"),  
    OTP("OTP"),  
    CHATS("Chats"),  
    MESSAGES("Messages"),  
    PROFILE("Profile"),  
    SETTINGS("Settings"),  
}
```

Листинг 104 – Код файла `app/src/main/java/com/example/poster/presentation/profile/MyProfileScreen.kt`

```
package com.example.poster.presentation.profile

import androidx.compose.foundation.background
import androidx.compose.foundation.border
import androidx.compose.foundation.clickable
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.heightIn
import androidx.compose.foundation.layout.navigationBarsPadding
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.layout.statusBarsPadding
import androidx.compose.foundation.layout.width
import androidx.compose.foundation.shape.CircleShape
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.foundation.text.BasicTextField
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.outlined.ArrowBack
import androidx.compose.material.icons.outlined.CameraAlt
import androidx.compose.material.icons.outlined.Edit
import androidx.compose.material3.Icon
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.draw.shadow
import androidx.compose.ui.graphics.Brush
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.SolidColor
import androidx.compose.ui.layout.ContentScale
import androidx.compose.ui.text.TextStyle
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.style.TextAlign
import androidx.compose.ui.tooling.preview.Preview
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import coil.compose.AsyncImage
```

```

private val PosterBackground = Color(0xFF030306)
private val PosterTopBar = Color(0xFF101014)
private val PosterSurface = Color(0xFF1B1A31)
private val PosterStroke = Color(0xFF302B68)
private val PosterPrimary = Color(0xFF625BFF)
private val PosterPrimaryDark = Color(0xFF493CCB)
private val PosterTextPrimary = Color(0xFFFF7F7FF)
private val PosterTextSecondary = Color(0xFFA5A6BA)
private val PosterTextMuted = Color(0xFF7B7D92)
private val PosterDivider = Color(0xFF1B1A25)

```

@Composable

```

fun MyProfileScreen(
    name: String = "Your Name",
    username: String = "@your.username",
    bio: String = "Hey there! I'm using Poster.",
    avatarUrl: String? = null,
    onBackClick: () -> Unit = {},
    onAvatarClick: () -> Unit = {},
    onNameChange: (String) -> Unit = {},
    onUsernameChange: (String) -> Unit = {},
    onBioChange: (String) -> Unit = {},
) {
    var nameValue by remember(name) { mutableStateOf(name) }
    var usernameValue by remember(username) { mutableStateOf(username) }
    var bioValue by remember(bio) { mutableStateOf(bio) }

```

```

Column(
    modifier = Modifier
        .fillMaxSize()
        .background(
            Brush.radialGradient(
                colors = listOf(
                    Color(0xFF09051E),
                    PosterBackground,
                    Color(0xFF030306),
                ),
                radius = 1200f,
            )
        )
        .navigationBarsPadding(),
) {
    MyProfileTopBar(
        onBackClick = onBackClick,
    )

    Box(
        modifier = Modifier
            .fillMaxWidth()
            .height(195.dp),
    ) {

```

```

Box(
    modifier = Modifier
        .fillMaxWidth()
        .height(138.dp)
        .background(
            Brush.horizontalGradient(
                colors = listOf(
                    PosterPrimary,
                    PosterPrimaryDark,
                ),
            ),
        ),
)

Box(
    modifier = Modifier
        .size(116.dp)
        .align(Alignment.BottomCenter)
        .background(PosterBackground, CircleShape),
    contentAlignment = Alignment.Center,
) {
    Box(
        modifier = Modifier
            .size(106.dp)
            .background(
                brush = Brush.linearGradient(
                    colors = listOf(
                        PosterPrimary,
                        PosterPrimaryDark,
                    ),
                ),
                shape = CircleShape,
            ),
        contentAlignment = Alignment.Center,
    ) {
        if (avatarUrl != null) {
            AsyncImage(
                model = avatarUrl,
                contentDescription = "Profile avatar",
                modifier = Modifier
                    .fillMaxSize()
                    .clip(CircleShape)
                    .background(Color.Transparent, CircleShape),
                contentScale = ContentScale.Crop,
            )
        } else {
            Text(
                text = getInitials(nameValue),
                color = Color.White,
                fontSize = 32.sp,
                fontWeight = FontWeight.Medium,
            )
        }
    }
}

```

```

    )
  }
}

Box(
  modifier = Modifier
    .size(35.dp)
    .align(Alignment.BottomEnd)
    .shadow(
      elevation = 12.dp,
      shape = CircleShape,
      spotColor = PosterPrimary.copy(alpha = 0.45f),
    )
    .background(
      brush = Brush.linearGradient(
        colors = listOf(
          PosterPrimary,
          PosterPrimaryDark,
        ),
      ),
      shape = CircleShape,
    )
    .border(
      width = 3.dp,
      color = PosterBackground,
      shape = CircleShape,
    )
    .clickable { onAvatarClick() },
  contentAlignment = Alignment.Center,
) {
  Icon(
    imageVector = Icons.Outlined.CameraAlt,
    contentDescription = "Change avatar",
    tint = Color.White,
    modifier = Modifier.size(18.dp),
  )
}
}
}

```

```

Spacer(modifier = Modifier.height(18.dp))

```

```

Text(
  text = nameValue,
  color = PosterTextPrimary,
  fontSize = 23.sp,
  fontWeight = FontWeight.Bold,
  textAlign = TextAlign.Center,
  modifier = Modifier.fillMaxWidth(),
)

```

```
Spacer(modifier = Modifier.height(26.dp))
```

```
Column(  
    modifier = Modifier  
        .fillMaxWidth()  
        .padding(horizontal = 30.dp),  
) {  
    Text(  
        text = "Name",  
        color = PosterTextSecondary,  
        fontSize = 13.sp,  
        fontWeight = FontWeight.Bold,  
    )
```

```
Spacer(modifier = Modifier.height(8.dp))
```

```
EditableProfileField(  
    value = nameValue,  
    onValueChange = {  
        nameValue = it  
        onNameChange(it)  
    },  
)
```

```
Spacer(modifier = Modifier.height(18.dp))
```

```
Text(  
    text = "Username",  
    color = PosterTextSecondary,  
    fontSize = 13.sp,  
    fontWeight = FontWeight.Bold,  
)
```

```
Spacer(modifier = Modifier.height(8.dp))
```

```
EditableProfileField(  
    value = usernameValue,  
    onValueChange = {  
        usernameValue = it  
        onUsernameChange(it)  
    },  
)
```

```
Spacer(modifier = Modifier.height(18.dp))
```

```
Text(  
    text = "Bio",  
    color = PosterTextSecondary,  
    fontSize = 13.sp,  
    fontWeight = FontWeight.Bold,  
)
```

```

        Spacer(modifier = Modifier.height(8.dp))

        EditableProfileField(
            value = bioValue,
            onValueChange = {
                bioValue = it
                onBioChange(it)
            },
        )
    }
}

@Composable
private fun MyProfileTopBar(
    onBackClick: () -> Unit,
) {
    Row(
        modifier = Modifier
            .fillMaxWidth()
            .background(PosterTopBar)
            .statusBarsPadding()
            .height(56.dp)
            .border(width = 0.5.dp, color = PosterDivider)
            .padding(horizontal = 16.dp),
        verticalAlignment = Alignment.CenterVertically,
    ) {
        Icon(
            imageVector = Icons.Outlined.ArrowBack,
            contentDescription = "Back",
            tint = PosterTextSecondary,
            modifier = Modifier
                .size(28.dp)
                .clickable { onBackClick() },
        )

        Spacer(modifier = Modifier.width(12.dp))

        Text(
            text = "My Profile",
            color = PosterTextPrimary,
            fontSize = 20.sp,
            fontWeight = FontWeight.Bold,
        )
    }
}

```

```

@Composable
private fun EditableProfileField(
    value: String,

```

```

        onValueChange: (String) -> Unit,
    ) {
        Row(
            modifier = Modifier
                .fillMaxWidth()
                .heightIn(min = 56.dp)
                .background(
                    color = PosterSurface,
                    shape = RoundedCornerShape(12.dp),
                )
                .border(
                    width = 1.dp,
                    color = PosterStroke.copy(alpha = 0.75f),
                    shape = RoundedCornerShape(12.dp),
                )
                .padding(horizontal = 16.dp, vertical = 12.dp),
            verticalAlignment = Alignment.CenterVertically,
        ) {
            BasicTextField(
                value = value,
                onValueChange = onValueChange,
                cursorBrush = SolidColor(PosterPrimary),
                textStyle = TextStyle(
                    color = PosterTextPrimary,
                    fontSize = 15.sp,
                    lineHeight = 21.sp,
                ),
                modifier = Modifier.weight(1f),
            )

            Spacer(modifier = Modifier.width(12.dp))

            Icon(
                imageVector = Icons.Outlined.Edit,
                contentDescription = null,
                tint = PosterTextSecondary,
                modifier = Modifier.size(20.dp),
            )
        }
    }

private fun getInitials(name: String): String {
    val parts = name.trim().split(" ").filter { it.isNotBlank() }

    return when {
        parts.size >= 2 -> "${parts[0].first()}${parts[1].first()}.uppercase()
        parts.size == 1 -> parts[0].take(2).uppercase()
        else -> "YN"
    }
}

```

```
@Preview(showBackground = true, widthDp = 393, heightDp = 852)
@Composable
private fun MyProfileScreenPreview() {
    MyProfileScreen(
        onBackClick = {},
        onAvatarClick = {},
    )
}
```

Листинг 105 – Код файла `app/src/main/java/com/example/poster/presentation/profile/ProfileScreen.kt`

```
package com.example.poster.presentation.profile

import androidx.compose.foundation.background
import androidx.compose.foundation.border
import androidx.compose.foundation.clickable
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.PaddingValues
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxHeight
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.navigationBarsPadding
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.layout.statusBarsPadding
import androidx.compose.foundation.layout.width
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.foundation.lazy.grid.GridCells
import androidx.compose.foundation.lazy.grid.LazyVerticalGrid
import androidx.compose.foundation.lazy.grid.items as gridItems
import androidx.compose.foundation.shape.CircleShape
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.outlined.ArrowBack
import androidx.compose.material.icons.outlined.Description
import androidx.compose.material.icons.outlined.Image
import androidx.compose.material3.Icon
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.draw.clip
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Brush
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.layout.ContentScale
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.style.TextOverflow
import androidx.compose.ui.tooling.preview.Preview
```

```

import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import coil.compose.AsyncImage
import com.example.poster.core.media.RemoteImageConfig
import com.example.poster.domain.model.Attachment
import com.example.poster.domain.model.AttachmentType

private val PosterBackground = Color(0xFF030306)
private val PosterTopBar = Color(0xFF101014)
private val PosterSurface = Color(0xFF1B1A31)
private val PosterStroke = Color(0xFF302B68)
private val PosterPrimary = Color(0xFF625BFF)
private val PosterPrimaryDark = Color(0xFF493CCB)
private val PosterTextPrimary = Color(0xFFFF7F7FF)
private val PosterTextSecondary = Color(0xFFA5A6BA)
private val PosterTextMuted = Color(0xFF7B7D92)
private val PosterDivider = Color(0xFF151522)

enum class ProfileTab {
    MEDIA,
    FILES,
}

data class ProfileFileUi(
    val id: String,
    val name: String,
    val size: String,
    val date: String,
)

@Composable
fun ProfileScreen(
    initials: String = "AJ",
    name: String = "Alice Johnson",
    username: String = "@alice.johnson",
    bio: String = "Product designer passionate about creating beautiful and functional user experience s.",
    attachments: List<Attachment> = emptyList(),
    files: List<ProfileFileUi> = emptyList(),
    onBackClick: () -> Unit = {},
) {
    var selectedTab by remember { mutableStateOf(ProfileTab.MEDIA) }
    val mediaAttachments = remember(attachments) {
        attachments.filter { attachment ->
            attachment.type == AttachmentType.IMAGE || attachment.type == AttachmentType.VIDE
        }
    }

    Column(
        modifier = Modifier

```

```

        .fillMaxSize()
        .background(PosterBackground)
        .navigationBarPadding(),
    ) {
        ProfileTopBar(
            onBackClick = onBackClick,
        )

        Box(
            modifier = Modifier
                .fillMaxWidth()
                .height(220.dp),
        ) {
            Box(
                modifier = Modifier
                    .fillMaxWidth()
                    .height(160.dp)
                    .background(
                        brush = Brush.horizontalGradient(
                            colors = listOf(
                                PosterPrimary,
                                PosterPrimaryDark,
                            ),
                        ),
                    ),
            ),
        ),
    )

    Box(
        modifier = Modifier
            .size(126.dp)
            .align(Alignment.BottomCenter)
            .background(PosterBackground, CircleShape),
        contentAlignment = Alignment.Center,
    ) {
        Box(
            modifier = Modifier
                .size(118.dp)
                .background(
                    brush = Brush.linearGradient(
                        listOf(PosterPrimary, PosterPrimaryDark),
                    ),
                    shape = CircleShape,
                ),
            contentAlignment = Alignment.Center,
        ) {
            Text(
                text = initials,
                color = Color.White,
                fontSize = 36.sp,
                fontWeight = FontWeight.Medium,
            )
        }
    }

```

```

    }
  }
}

```

```

Spacer(modifier = Modifier.height(18.dp))

```

```

Column(
  modifier = Modifier
    .fillMaxWidth()
    .padding(horizontal = 28.dp),
  horizontalAlignment = Alignment.CenterHorizontally,
) {
  Text(
    text = name,
    color = PosterTextPrimary,
    fontSize = 24.sp,
    fontWeight = FontWeight.Bold,
  )

```

```

Spacer(modifier = Modifier.height(6.dp))

```

```

Text(
  text = username,
  color = Color(0xFF7F8BFF),
  fontSize = 16.sp,
)

```

```

Spacer(modifier = Modifier.height(18.dp))

```

```

Text(
  text = bio,
  color = PosterTextSecondary,
  fontSize = 16.sp,
  lineHeight = 23.sp,
)
}

```

```

Spacer(modifier = Modifier.height(36.dp))

```

```

ProfileTabs(
  selectedTab = selectedTab,
  onTabClick = { selectedTab = it },
)

```

```

Box(
  modifier = Modifier
    .fillMaxWidth()
    .height(1.dp)
    .background(PosterStroke),
)

```

```

        when (selectedTab) {
            ProfileTab.MEDIA -> MediaGrid(attachments = mediaAttachments)
            ProfileTab.FILES -> FilesList(
                files = files,
                onFileClick = { file ->
                    // TODO: open/download file
                },
            )
        }
    }
}

```

```

@Composable
private fun ProfileTopBar(
    onBackClick: () -> Unit,
) {
    Row(
        modifier = Modifier
            .fillMaxWidth()
            .background(PosterTopBar)
            .statusBarsPadding()
            .height(49.dp)
            .padding(horizontal = 8.dp),
        verticalAlignment = Alignment.CenterVertically,
    ) {
        Icon(
            imageVector = Icons.Outlined.ArrowBack,
            contentDescription = "Back",
            tint = PosterTextSecondary,
            modifier = Modifier
                .size(28.dp)
                .clickable { onBackClick() },
        )

        Spacer(modifier = Modifier.width(8.dp))

        Text(
            text = "Profile",
            color = PosterTextPrimary,
            fontSize = 20.sp,
            fontWeight = FontWeight.Bold,
        )
    }
}

```

```

@Composable
private fun ProfileTabs(
    selectedTab: ProfileTab,
    onTabClick: (ProfileTab) -> Unit,
) {
    Row(

```

```

        modifier = Modifier
            .fillMaxWidth()
            .height(58.dp),
    ) {
        ProfileTabItem(
            text = "Media",
            selected = selectedTab == ProfileTab.MEDIA,
            icon = {
                Icon(
                    imageVector = Icons.Outlined.Image,
                    contentDescription = null,
                    tint = if (selectedTab == ProfileTab.MEDIA) {
                        Color(0xFF7F8BFF)
                    } else {
                        PosterTextMuted
                    },
                    modifier = Modifier.size(20.dp),
                )
            },
            onClick = { onTabClick(ProfileTab.MEDIA) },
            modifier = Modifier.weight(1f),
        )

        ProfileTabItem(
            text = "Files",
            selected = selectedTab == ProfileTab.FILES,
            icon = {
                Icon(
                    imageVector = Icons.Outlined.Description,
                    contentDescription = null,
                    tint = if (selectedTab == ProfileTab.FILES) {
                        Color(0xFF7F8BFF)
                    } else {
                        PosterTextMuted
                    },
                    modifier = Modifier.size(20.dp),
                )
            },
            onClick = { onTabClick(ProfileTab.FILES) },
            modifier = Modifier.weight(1f),
        )
    }
}

```

```

@Composable
private fun ProfileTabItem(
    text: String,
    selected: Boolean,
    icon: @Composable () -> Unit,
    onClick: () -> Unit,
    modifier: Modifier = Modifier,

```

```

) {
    Box(
        modifier = modifier
        .fillMaxHeight()
        .clickable { onClick() },
        contentAlignment = Alignment.Center,
    ) {
        Column(
            modifier = Modifier.fillMaxSize(),
            horizontalAlignment = Alignment.CenterHorizontally,
        ) {
            Spacer(modifier = Modifier.weight(1f))

            Row(
                verticalAlignment = Alignment.CenterVertically,
            ) {
                icon()

                Spacer(modifier = Modifier.width(8.dp))

                Text(
                    text = text,
                    color = if (selected) Color(0xFF7F8BFF) else PosterTextMuted,
                    fontSize = 16.sp,
                    fontWeight = FontWeight.SemiBold,
                )
            }

            Spacer(modifier = Modifier.weight(1f))

            Box(
                modifier = Modifier
                    .fillMaxWidth()
                    .height(2.dp)
                    .background(
                        if (selected) Color(0xFF7F8BFF) else Color.Transparent,
                    ),
            )
        }
    }
}

```

```

@Composable
private fun MediaGrid(attachments: List<Attachment>) {
    if (attachments.isEmpty()) {
        EmptyProfileTab(
            icon = {
                Icon(
                    imageVector = Icons.Outlined.Image,
                    contentDescription = null,
                    tint = PosterTextMuted,
                )
            }
        )
    }
}

```

```

        modifier = Modifier.size(42.dp),
    )
},
text = "No media yet",
)
return
}

```

```

LazyVerticalGrid(
    columns = GridCells.Fixed(3),
    modifier = Modifier.fillMaxSize(),
    contentPadding = PaddingValues(6.dp),
    horizontalArrangement = Arrangement.spacedBy(8.dp),
    verticalArrangement = Arrangement.spacedBy(8.dp),
) {
    gridItems(attachments, key = { it.id }) { attachment ->
        val context = LocalContext.current
        val imageModel = remember(context, attachment.localUri, attachment.remoteUri) {
            attachment.localUri ?: attachment.remoteUri?.let { remoteUri ->
                RemoteImageConfig.buildImageModel(
                    context = context,
                    remoteUri = remoteUri,
                )
            }
        }
    }
    val tileShape = RoundedCornerShape(7.dp)
}

```

```

Box(
    modifier = Modifier
        .fillMaxWidth()
        .height(132.dp)
        .clip(tileShape)
        .background(color = PosterSurface, shape = tileShape)
        .border(
            width = 0.5.dp,
            color = PosterDivider,
            shape = tileShape,
        ),
) {
    if (imageModel != null) {
        AsyncImage(
            model = imageModel,
            contentDescription = attachment.fileName,
            modifier = Modifier.fillMaxSize(),
            contentScale = ContentScale.Crop,
        )
    } else {
        Box(
            modifier = Modifier.fillMaxSize(),
            contentAlignment = Alignment.Center,
        ) {

```



```

        contentPadding = PaddingValues(
            start = 4.dp,
            end = 4.dp,
            top = 14.dp,
            bottom = 24.dp,
        ),
        verticalArrangement = Arrangement.spacedBy(8.dp),
    ) {
        items(
            items = files,
            key = { it.id },
        ) { file ->
            ProfileFileRow(
                file = file,
                onClick = { onFileClick(file) },
            )
        }
    }
}

```

```

@Composable
private fun ProfileFileRow(
    file: ProfileFileUi,
    onClick: () -> Unit,
) {
    Row(
        modifier = Modifier
            .fillMaxWidth()
            .height(80.dp)
            .background(
                color = PosterSurface,
                shape = RoundedCornerShape(10.dp),
            )
        .border(
            width = 0.5.dp,
            color = PosterDivider,
            shape = RoundedCornerShape(10.dp),
        )
        .clickable { onClick() }
        .padding(horizontal = 18.dp),
        verticalAlignment = Alignment.CenterVertically,
    ) {
        Box(
            modifier = Modifier
                .size(40.dp)
                .background(
                    brush = Brush.linearGradient(
                        colors = listOf(
                            PosterPrimary,
                            PosterPrimaryDark,
                        ),
                    ),
                )
            )
    }
}

```

```

        ),
        shape = RoundedCornerShape(9.dp),
    ),
    contentAlignment = Alignment.Center,
) {
    Icon(
        imageVector = Icons.Outlined.Description,
        contentDescription = null,
        tint = Color.White,
        modifier = Modifier.size(22.dp),
    )
}

Spacer(modifier = Modifier.width(14.dp))

Column(modifier = Modifier.weight(1f)) {
    Text(
        text = file.name,
        color = PosterTextPrimary,
        fontSize = 17.sp,
        fontWeight = FontWeight.Bold,
        maxLines = 1,
        overflow = TextOverflow.Ellipsis,
    )

    Spacer(modifier = Modifier.height(5.dp))

    Text(
        text = "${file.size} • ${file.date}",
        color = PosterTextMuted,
        fontSize = 14.sp,
    )
}
}

@Composable
private fun EmptyProfileTab(
    icon: @Composable () -> Unit,
    text: String,
) {
    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(top = 72.dp),
        horizontalAlignment = Alignment.CenterHorizontally,
    ) {
        icon()

        Spacer(modifier = Modifier.height(14.dp))
    }
}

```

```
        Text(  
            text = text,  
            color = PosterTextMuted,  
            fontSize = 16.sp,  
        )  
    }  
}
```

```
@Preview(showBackground = true, widthDp = 393, heightDp = 852)
```

```
@Composable
```

```
private fun ProfileScreenPreview() {
```

```
    ProfileScreen(  
        initials = "AJ",  
        name = "Alice Johnson",  
        username = "@alice.johnson",  
        bio = "Product designer passionate about creating beautiful and functional user experiences.",  
        onBackClick = {},  
    )  
}
```

Листинг 106 – Код файла `app/src/main/java/com/example/poster/presentation/settings/AccessTokenSettingsScreen.kt`

```
package com.example.poster.presentation.settings

import androidx.compose.foundation.background
import androidx.compose.foundation.border
import androidx.compose.foundation.clickable
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.layout.statusBarsPadding
import androidx.compose.foundation.layout.width
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.text.BasicTextField
import androidx.compose.foundation.text.KeyboardOptions
import androidx.compose.foundation.verticalScroll
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.outlined.ArrowBack
import androidx.compose.material.icons.outlined.HelpOutline
import androidx.compose.material.icons.outlined.Key
import androidx.compose.material.icons.outlined.Visibility
import androidx.compose.material.icons.outlined.VisibilityOff
import androidx.compose.material3.Icon
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.shadow
import androidx.compose.ui.graphics.Brush
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.SolidColor
import androidx.compose.ui.text.TextStyle
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.input.KeyboardType
import androidx.compose.ui.text.input.PasswordVisualTransformation
import androidx.compose.ui.text.input.VisualTransformation
import androidx.compose.ui.tooling.preview.Preview
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import com.example.poster.domain.model.MailAccessSettings
```

```

private val PosterBackground = Color(0xFF030306)
private val PosterTopBar = Color(0xFF101014)
private val PosterSurface = Color(0xFF1B1A31)
private val PosterInfoSurface = Color(0xFF0F0D27)
private val PosterStroke = Color(0xFF302B68)
private val PosterPrimary = Color(0xFF625BFF)
private val PosterPrimaryDark = Color(0xFF342983)
private val PosterTextPrimary = Color(0xFFFF7F7FF)
private val PosterTextSecondary = Color(0xFFA5A6BA)
private val PosterTextMuted = Color(0xFF7B7D92)
private val PosterDivider = Color(0xFF1B1A25)

```

@Composable

```

fun AccessTokenSettingsScreen(
    initialToken: String = "",
    initialSmtpHost: String = "",
    initialSmtpPort: String = "",
    initialImapHost: String = "",
    initialImapPort: String = "",
    onBackClick: () -> Unit,
    onSaveTokenClick: (MailAccessSettings) -> Unit,
    onHowToGetTokenClick: () -> Unit,
) {
    var token by remember { mutableStateOf(initialToken) }
    var smtpHost by remember { mutableStateOf(initialSmtpHost) }
    var smtpPort by remember { mutableStateOf(initialSmtpPort) }
    var imapHost by remember { mutableStateOf(initialImapHost) }
    var imapPort by remember { mutableStateOf(initialImapPort) }
    var isTokenVisible by remember { mutableStateOf(false) }
    val scrollState = rememberScrollState()

    val canSave = token.trim().isNotEmpty() &&
        smtpPort.isBlankOrValidPort() &&
        imapPort.isBlankOrValidPort()

```

```

Column(
    modifier = Modifier
        .fillMaxSize()
        .background(
            Brush.radialGradient(
                colors = listOf(
                    Color(0xFF08051A),
                    PosterBackground,
                    Color(0xFF030306),
                ),
                radius = 1100f,
            )
        ),
) {
    AccessTokenTopBar(

```

```

        onBackClick = onBackClick,
    )

    Column(
        modifier = Modifier
            .fillMaxSize()
            .verticalScroll(scrollState)
            .padding(horizontal = 23.dp),
    ) {
        Spacer(modifier = Modifier.height(24.dp))

        AccessTokenInfoCard()

        Spacer(modifier = Modifier.height(28.dp))

        Text(
            text = "Access Token",
            color = PosterTextPrimary,
            fontSize = 16.sp,
            fontWeight = FontWeight.Bold,
        )

        Spacer(modifier = Modifier.height(12.dp))

        AccessTokenInput(
            value = token,
            onValueChange = { token = it },
            isTokenVisible = isTokenVisible,
            onVisibilityClick = {
                isTokenVisible = !isTokenVisible
            },
        )

        Spacer(modifier = Modifier.height(22.dp))

        Text(
            text = "Mail Server Settings",
            color = PosterTextPrimary,
            fontSize = 16.sp,
            fontWeight = FontWeight.Bold,
        )

        Spacer(modifier = Modifier.height(12.dp))

        MailConfigInput(
            label = "SMTP Host",
            value = smtpHost,
            onValueChange = { smtpHost = it },
            placeholder = "smtp.example.com",
        )
    }

```

```
Spacer(modifier = Modifier.height(12.dp))
```

```
MailConfigInput(  
    label = "SMTP Port",  
    value = smtpPort,  
    onChange = { smtpPort = it.filter(Char::isDigit).take(5) },  
    placeholder = "587",  
    keyboardType = TextInputType.Number,  
)
```

```
Spacer(modifier = Modifier.height(12.dp))
```

```
MailConfigInput(  
    label = "IMAP Host",  
    value = imapHost,  
    onChange = { imapHost = it },  
    placeholder = "imap.example.com",  
)
```

```
Spacer(modifier = Modifier.height(12.dp))
```

```
MailConfigInput(  
    label = "IMAP Port",  
    value = imapPort,  
    onChange = { imapPort = it.filter(Char::isDigit).take(5) },  
    placeholder = "993",  
    keyboardType = TextInputType.Number,  
)
```

```
Spacer(modifier = Modifier.height(22.dp))
```

```
HowToGetTokenButton(  
    onClick = onHowToGetTokenClick,  
)
```

```
Spacer(modifier = Modifier.height(36.dp))
```

```
SaveTokenButton(  
    enabled = canSave,  
    onClick = {  
        onSaveTokenClick(  
            MailAccessSettings(  
                token = token.trim(),  
                smtpHost = smtpHost.trim().ifBlank { null },  
                smtpPort = smtpPort.toIntOrNull(),  
                imapHost = imapHost.trim().ifBlank { null },  
                imapPort = imapPort.toIntOrNull(),  
            )  
        )  
    },  
)
```

```
Text(
  text = "Your token is encrypted and stored securely on your device",
  color = PosterTextMuted,
  fontSize = 13.sp,
  modifier = Modifier.align(Alignment.CenterHorizontally),
)
}
```

```
Spacer(modifier = Modifier.height(8.dp))
```

```
BasicTextField(
    value = value,
    onValueChange = onValueChange,
    singleLine = true,
    cursorBrush = SolidColor(PosterPrimary),
    textStyle = TextStyle(
        color = PosterTextPrimary,
        fontSize = 14.sp,
    ),
    keyboardOptions = KeyboardOptions(keyboardType = keyboardType),
    decorationBox = { innerTextField ->
        Box(
            modifier = Modifier
                .fillMaxWidth()
                .height(52.dp)
                .background(
                    color = PosterSurface,
                    shape = androidx.compose.foundation.shape.RoundedCornerShape(13.dp),
                )
            .border(
```

```

        width = 1.dp,
        color = PosterStroke.copy(alpha = 0.7f),
        shape = androidx.compose.foundation.shape.RoundedCornerShape(13.dp),
    )
    .padding(horizontal = 16.dp),
    contentAlignment = Alignment.CenterStart,
) {
    if (value.isEmpty()) {
        Text(
            text = placeholder,
            color = PosterTextMuted,
            fontSize = 14.sp,
        )
    }
    innerTextField()
}
},
)
}
}

```

```

@Composable
private fun AccessTokenTopBar(
    onBackClick: () -> Unit,
) {
    Row(
        modifier = Modifier
            .fillMaxWidth()
            .background(PosterTopBar)
            .statusBarsPadding()
            .height(56.dp)
            .border(width = 0.5.dp, color = PosterDivider)
            .padding(horizontal = 16.dp),
        verticalAlignment = Alignment.CenterVertically,
    ) {
        Icon(
            imageVector = Icons.Outlined.ArrowBack,
            contentDescription = "Back",
            tint = PosterTextSecondary,
            modifier = Modifier
                .size(28.dp)
                .clickable { onBackClick() },
        )

        Spacer(modifier = Modifier.width(12.dp))

        Text(
            text = "Access Token Settings",
            color = PosterTextPrimary,
            fontSize = 20.sp,
            fontWeight = FontWeight.Bold,

```

```

    )
  }
}

```

@Composable

private fun AccessTokenInfoCard() {

Row(

modifier = Modifier

.fillMaxWidth()

.background(

color = PosterInfoSurface,

shape = androidx.compose.foundation.shape.RoundedCornerShape(13.dp),

)

.border(

width = 1.dp,

color = PosterStroke,

shape = androidx.compose.foundation.shape.RoundedCornerShape(13.dp),

)

.padding(

start = 24.dp,

end = 22.dp,

top = 26.dp,

bottom = 24.dp,

),

) {

Icon(

imageVector = Icons.Outlined.Key,

contentDescription = null,

tint = PosterPrimary,

modifier = Modifier

.padding(top = 2.dp)

.size(25.dp),

)

Spacer(modifier = Modifier.width(14.dp))

Column {

Text(

text = "Why do I need an access token?",

color = PosterTextPrimary,

fontSize = 18.sp,

fontWeight = FontWeight.Bold,

)

Spacer(modifier = Modifier.height(12.dp))

Text(

text = "Poster uses SMTP/IMAP protocols to send and receive messages through your email provider. An access token allows the app to securely connect to your email without storing your password.",

color = PosterTextSecondary,

```

        fontSize = 15.sp,
        lineHeight = 23.sp,
    )
}
}
}

```

```

@Composable
private fun AccessTokenInput(
    value: String,
    onChange: (String) -> Unit,
    isVisible: Boolean,
    onVisibilityClick: () -> Unit,
) {
    Row(
        modifier = Modifier
            .fillMaxWidth()
            .height(55.dp)
            .background(
                color = PosterSurface,
                shape = androidx.compose.foundation.shape.RoundedCornerShape(13.dp),
            )
            .border(
                width = 1.dp,
                color = PosterStroke.copy(alpha = 0.7f),
                shape = androidx.compose.foundation.shape.RoundedCornerShape(13.dp),
            )
            .padding(start = 16.dp, end = 14.dp),
        verticalAlignment = Alignment.CenterVertically,
    ) {
        BasicTextField(
            value = value,
            onChange = onChange,
            singleLine = true,
            cursorBrush = SolidColor(PosterPrimary),
            textStyle = TextStyle(
                color = PosterTextPrimary,
                fontSize = 14.sp,
                letterSpacing = 1.sp,
            ),
            keyboardOptions = KeyboardOptions.Default,
            visualTransformation = if (isVisible) {
                VisualTransformation.None
            } else {
                PasswordVisualTransformation()
            },
            decorationBox = { innerTextField ->
                Box(
                    modifier = Modifier.fillMaxWidth(),
                    contentAlignment = Alignment.CenterStart,
                ) {

```

```

        if (value.isEmpty()) {
            Text(
                text = "Paste your access token here",
                color = PosterTextMuted,
                fontSize = 14.sp,
                letterSpacing = 1.sp,
            )
        }

        innerTextField()
    },
    modifier = Modifier.weight(1f),
)

Spacer(modifier = Modifier.width(12.dp))

Icon(
    imageVector = if (isTokenVisible) {
        Icons.Outlined.VisibilityOff
    } else {
        Icons.Outlined.Visibility
    },
    contentDescription = "Toggle token visibility",
    tint = PosterTextSecondary,
    modifier = Modifier
        .size(22.dp)
        .clickable { onVisibilityClick() },
)
}
}

@Composable
private fun HowToGetTokenButton(
    onClick: () -> Unit,
) {
    Row(
        modifier = Modifier.clickable { onClick() },
        verticalAlignment = Alignment.CenterVertically,
    ) {
        Icon(
            imageVector = Icons.Outlined.HelpOutline,
            contentDescription = null,
            tint = Color(0xFF7F8BFF),
            modifier = Modifier.size(21.dp),
        )

        Spacer(modifier = Modifier.width(8.dp))

        Text(
            text = "How to get an access token?",

```

```

        color = Color(0xFF7F8BFF),
        fontSize = 16.sp,
        fontWeight = FontWeight.Bold,
    )
}
}

@Composable
private fun SaveTokenButton(
    enabled: Boolean,
    onClick: () -> Unit,
) {
    Box(
        modifier = Modifier
            .fillMaxWidth()
            .height(56.dp)
            .shadow(
                elevation = if (enabled) 24.dp else 0.dp,
                shape = androidx.compose.foundation.shape.RoundedCornerShape(13.dp),
                spotColor = PosterPrimary.copy(alpha = 0.35f),
            )
        .background(
            brush = if (enabled) {
                Brush.horizontalGradient(
                    colors = listOf(
                        PosterPrimary,
                        PosterPrimaryDark,
                    ),
                )
            } else {
                Brush.horizontalGradient(
                    colors = listOf(
                        Color(0xFF312A7A),
                        Color(0xFF261D65),
                    ),
                )
            },
            shape = androidx.compose.foundation.shape.RoundedCornerShape(13.dp),
        )
        .clickable(enabled = enabled) { onClick() },
        contentAlignment = Alignment.Center,
    ) {
        Text(
            text = "Save Token",
            color = if (enabled) Color.White else PosterTextMuted,
            fontSize = 16.sp,
            fontWeight = FontWeight.Bold,
        )
    }
}

```

```
private fun String.isBlankOrValidPort(): Boolean {  
    return isBlank() || toIntOrNull()?.let { it in 1..65535 } == true  
}
```

```
@Preview(showBackground = true, widthDp = 393, heightDp = 852)  
@Composable  
private fun AccessTokenSettingsScreenPreview() {  
    AccessTokenSettingsScreen(  
        onBackClick = {},  
        onSaveTokenClick = { _ -> },  
        onHowToGetTokenClick = {},  
    )  
}
```

Листинг 107 – Код файла `app/src/main/java/com/example/poster/presentation/settings/SettingsDetailScreens.kt`

```
package com.example.poster.presentation.settings

import androidx.compose.foundation.background
import androidx.compose.foundation.border
import androidx.compose.foundation.clickable
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.PaddingValues
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.heightIn
import androidx.compose.foundation.layout.navigationBarsPadding
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.layout.statusBarsPadding
import androidx.compose.foundation.layout.width
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.shape.CircleShape
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.foundation.text.BasicTextField
import androidx.compose.foundation.text.KeyboardOptions
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.outlined.ArrowBack
import androidx.compose.material.icons.outlined.CheckCircle
import androidx.compose.material.icons.outlined.Info
import androidx.compose.material.icons.outlined.RadioButtonUnchecked
import androidx.compose.material3.Icon
import androidx.compose.material3.Switch
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.Immutable
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.saveable.rememberSaveable
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.shadow
import androidx.compose.ui.graphics.Brush
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.SolidColor
import androidx.compose.ui.text.TextStyle
import androidx.compose.ui.text.font.FontWeight
```

```

import androidx.compose.ui.text.input.KeyboardType
import androidx.compose.ui.text.style.TextOverflow
import androidx.compose.ui.tooling.preview.Preview
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp

private val PosterBackground = Color(0xFF030306)
private val PosterTopBar = Color(0xFF101014)
private val PosterSurface = Color(0xFF1B1A31)
private val PosterInfoSurface = Color(0xFF0F0D27)
private val PosterStroke = Color(0xFF302B68)
private val PosterPrimary = Color(0xFF625BFF)
private val PosterPrimaryDark = Color(0xFF493CCB)
private val PosterTextPrimary = Color(0xFFFF7F7FF)
private val PosterTextSecondary = Color(0xFFA5A6BA)
private val PosterTextMuted = Color(0xFF7B7D92)
private val PosterDivider = Color(0xFF1B1A25)
private val PosterAccent = Color(0xFF7F8BFF)

```

```

@Immutable
data class PrivacySettingsUi(
    val showOnlineStatus: Boolean,
    val sendReadReceipts: Boolean,
    val confirmBeforeOpeningFiles: Boolean,
)

```

```

@Composable
fun SettingsTextEditScreen(
    title: String,
    label: String,
    value: String,
    placeholder: String,
    onBackClick: () -> Unit,
    onSaveClick: (String) -> Unit,
    modifier: Modifier = Modifier,
    singleLine: Boolean = true,
    allowEmpty: Boolean = false,
    keyboardType: KeyboardType = KeyboardType.Text,
) {
    var editedValue by remember(value) { mutableStateOf(value) }
    val normalizedValue = editedValue.trim()
    val canSave = allowEmpty || normalizedValue.isNotEmpty()

```

```

SettingsDetailScaffold(
    title = title,
    onBackClick = onBackClick,
    modifier = modifier,
) {
    item {
        Text(
            text = label,

```

```

        color = PosterTextPrimary,
        fontSize = 16.sp,
        fontWeight = FontWeight.Bold,
        modifier = Modifier.padding(horizontal = 23.dp),
    )
}

item {
    Spacer(modifier = Modifier.height(12.dp))
}

item {
    SettingsEditField(
        value = editedValue,
        onChange = { editedValue = it },
        placeholder = placeholder,
        singleLine = singleLine,
        keyboardType = keyboardType,
        modifier = Modifier.padding(horizontal = 23.dp),
    )
}

item {
    Spacer(modifier = Modifier.height(24.dp))
}

item {
    SettingsPrimaryButton(
        text = "Save",
        enabled = canSave,
        onClick = { onSaveClick(if (allowEmpty) editedValue.trim() else normalizedValue) },
        modifier = Modifier.padding(horizontal = 23.dp),
    )
}
}
}

```

```

@Composable
fun LanguageSettingsScreen(
    currentLanguage: String,
    onBackClick: () -> Unit,
    onSaveLanguageClick: (String) -> Unit,
    modifier: Modifier = Modifier,
) {
    var selectedLanguage by rememberSaveable(currentLanguage) {
        mutableStateOf(currentLanguage)
    }
    val languages = listOf(
        SettingsOption(
            value = "English",
            title = "English",

```

```

        subtitle = "Interface labels and placeholders stay in English",
    ),
    SettingsOption(
        value = "Русский",
        title = "Русский",
        subtitle = "Основной язык интерфейса: русский",
    ),
    SettingsOption(
        value = "Deutsch",
        title = "Deutsch",
        subtitle = "UI-Sprache auf Deutsch umschalten",
    ),
)

SettingsDetailScaffold(
    title = "Language",
    onBackPressed = onBackPressed,
    modifier = modifier,
) {
    items(languages.size) { index ->
        val option = languages[index]
        SettingsSelectableRow(
            title = option.title,
            subtitle = option.subtitle,
            selected = selectedLanguage == option.value,
            onClick = { selectedLanguage = option.value },
            modifier = Modifier.padding(horizontal = 16.dp),
        )
    }

    item {
        Spacer(modifier = Modifier.height(24.dp))
    }

    item {
        SettingsPrimaryButton(
            text = "Save Language",
            enabled = true,
            onClick = { onSaveLanguageClick(selectedLanguage) },
            modifier = Modifier.padding(horizontal = 23.dp),
        )
    }
}

@Composable
fun PrivacySettingsScreen(
    settings: PrivacySettingsUi,
    onBackPressed: () -> Unit,
    onSavePrivacyClick: (PrivacySettingsUi) -> Unit,
    modifier: Modifier = Modifier,

```

```

) {
    var showOnlineStatus by rememberSaveable(settings.showOnlineStatus) {
        mutableStateOf(settings.showOnlineStatus)
    }
    var sendReadReceipts by rememberSaveable(settings.sendReadReceipts) {
        mutableStateOf(settings.sendReadReceipts)
    }
    var confirmBeforeOpeningFiles by rememberSaveable(settings.confirmBeforeOpeningFiles) {
        mutableStateOf(settings.confirmBeforeOpeningFiles)
    }

    SettingsDetailScaffold(
        title = "Privacy & Security",
        onBackPressed = onBackPressed,
        modifier = modifier,
    ) {
        item {
            SettingsInfoCard(
                title = "Security controls",
                description = "These preferences are applied immediately in this session and stay active while the app process is alive.",
                modifier = Modifier.padding(horizontal = 16.dp),
            )
        }

        item {
            Spacer(modifier = Modifier.height(20.dp))
        }

        item {
            SettingsSwitchRow(
                title = "Show online status",
                subtitle = "Contacts can see that you are online",
                checked = showOnlineStatus,
                onCheckedChange = { showOnlineStatus = it },
                modifier = Modifier.padding(horizontal = 16.dp),
            )
        }

        item {
            SettingsSwitchRow(
                title = "Read receipts",
                subtitle = "Allow contacts to see when messages are read",
                checked = sendReadReceipts,
                onCheckedChange = { sendReadReceipts = it },
                modifier = Modifier.padding(horizontal = 16.dp, vertical = 10.dp),
            )
        }

        item {
            SettingsSwitchRow(

```

```

        title = "Confirm files",
        subtitle = "Ask before opening downloaded attachments",
        checked = confirmBeforeOpeningFiles,
        onCheckedChange = { confirmBeforeOpeningFiles = it },
        modifier = Modifier.padding(horizontal = 16.dp),
    )
}

item {
    Spacer(modifier = Modifier.height(24.dp))
}

item {
    SettingsPrimaryButton(
        text = "Save Privacy Settings",
        enabled = true,
        onClick = {
            onSavePrivacyClick(
                PrivacySettingsUi(
                    showOnlineStatus = showOnlineStatus,
                    sendReadReceipts = sendReadReceipts,
                    confirmBeforeOpeningFiles = confirmBeforeOpeningFiles,
                )
            )
        },
        modifier = Modifier.padding(horizontal = 23.dp),
    )
}
}
}
}

```

@Composable

```

fun AccessTokenGuideScreen(
    onBackClick: () -> Unit,
    modifier: Modifier = Modifier,
) {
    SettingsDetailScaffold(
        title = "How to get token",
        onBackClick = onBackClick,
        modifier = modifier,
    ) {
        item {
            SettingsInfoCard(
                title = "Use an app password",
                description = "For most mail providers this token is an app-specific password. Create it in your email account security settings and paste it into Poster.",
                modifier = Modifier.padding(horizontal = 16.dp),
            )
        }

        item {

```

```

        Spacer(modifier = Modifier.height(20.dp))
    }

    items(tokenSteps.size) { index ->
        TokenStepRow(
            index = index + 1,
            text = tokenSteps[index],
            modifier = Modifier.padding(horizontal = 16.dp),
        )
    }
}

@Composable
private fun SettingsDetailScaffold(
    title: String,
    onClick: () -> Unit,
    modifier: Modifier = Modifier,
    content: androidx.compose.foundation.lazy.LazyListScope.() -> Unit,
) {
    Column(
        modifier = modifier
        .fillMaxSize()
        .background(
            Brush.radialGradient(
                colors = listOf(
                    Color(0xFF09051E),
                    PosterBackground,
                    Color(0xFF030306),
                ),
                radius = 1200f,
            )
        ),
    ) {
        SettingsDetailTopBar(
            title = title,
            onClick = onClick,
        )

        LazyColumn(
            modifier = Modifier
                .fillMaxSize()
                .navigationBarPadding(),
            contentPadding = PaddingValues(top = 24.dp, bottom = 32.dp),
            verticalArrangement = Arrangement.Top,
            content = content,
        )
    }
}

```

@Composable

```

private fun SettingsDetailTopBar(
    title: String,
    onBackPressed: () -> Unit,
) {
    Row(
        modifier = Modifier
            .fillMaxWidth()
            .background(PosterTopBar)
            .statusBarsPadding()
            .height(56.dp)
            .border(width = 0.5.dp, color = PosterDivider)
            .padding(horizontal = 16.dp),
        verticalAlignment = Alignment.CenterVertically,
    ) {
        Icon(
            imageVector = Icons.Outlined.ArrowBack,
            contentDescription = "Back",
            tint = PosterTextSecondary,
            modifier = Modifier
                .size(28.dp)
                .clickable { onBackPressed() },
        )

        Spacer(modifier = Modifier.width(12.dp))

        Text(
            text = title,
            color = PosterTextPrimary,
            fontSize = 20.sp,
            fontWeight = FontWeight.Bold,
            maxLines = 1,
            overflow = TextOverflow.Ellipsis,
        )
    }
}

```

```

@Composable
private fun SettingsEditField(
    value: String,
    onValueChange: (String) -> Unit,
    placeholder: String,
    singleLine: Boolean,
    keyboardType: KeyboardType,
    modifier: Modifier = Modifier,
) {
    Box(
        modifier = modifier
            .fillMaxWidth()
            .heightIn(min = if (singleLine) 56.dp else 148.dp)
            .background(
                color = PosterSurface,
            )
    )
}

```

```

        shape = RoundedCornerShape(13.dp),
    )
    .border(
        width = 1.dp,
        color = PosterStroke.copy(alpha = 0.75f),
        shape = RoundedCornerShape(13.dp),
    )
    .padding(horizontal = 16.dp, vertical = 14.dp),
    contentAlignment = Alignment.TopStart,
) {
    BasicTextField(
        value = value,
        onValueChange = onValueChange,
        singleLine = singleLine,
        cursorBrush = SolidColor(PosterPrimary),
        keyboardOptions = KeyboardOptions(keyboardType = keyboardType),
        textStyle = TextStyle(
            color = PosterTextPrimary,
            fontSize = 15.sp,
            lineHeight = 22.sp,
        ),
        decorationBox = { innerTextField ->
            Box(
                modifier = Modifier.fillMaxWidth(),
                contentAlignment = Alignment.TopStart,
            ) {
                if (value.isEmpty()) {
                    Text(
                        text = placeholder,
                        color = PosterTextMuted,
                        fontSize = 15.sp,
                    )
                }

                innerTextField()
            }
        },
        modifier = Modifier.fillMaxWidth(),
    )
}

```

```

@Composable
private fun SettingsSelectableRow(
    title: String,
    subtitle: String,
    selected: Boolean,
    onClick: () -> Unit,
    modifier: Modifier = Modifier,
) {
    Row(

```

```

        modifier = modifier
        .fillMaxWidth()
        .padding(bottom = 10.dp)
        .background(
            color = if (selected) PosterPrimary.copy(alpha = 0.2f) else PosterSurface.copy(alpha = 0.8
2f),
            shape = RoundedCornerShape(15.dp),
        )
        .border(
            width = 1.dp,
            color = if (selected) PosterAccent else PosterStroke.copy(alpha = 0.7f),
            shape = RoundedCornerShape(15.dp),
        )
        .clickable { onClick() }
        .padding(horizontal = 18.dp, vertical = 16.dp),
        verticalAlignment = Alignment.CenterVertically,
    ) {
        Column(modifier = Modifier.weight(1f)) {
            Text(
                text = title,
                color = PosterTextPrimary,
                fontSize = 15.sp,
                fontWeight = FontWeight.Bold,
            )

            Spacer(modifier = Modifier.height(5.dp))

            Text(
                text = subtitle,
                color = PosterTextSecondary,
                fontSize = 13.sp,
                lineHeight = 18.sp,
            )
        }

        Icon(
            imageVector = if (selected) Icons.Outlined.CheckCircle else Icons.Outlined.RadioButtonUn
checked,
            contentDescription = null,
            tint = if (selected) PosterAccent else PosterTextMuted,
            modifier = Modifier.size(24.dp),
        )
    }
}

```

```

@Composable
private fun SettingsSwitchRow(
    title: String,
    subtitle: String,
    checked: Boolean,
    onCheckedChange: (Boolean) -> Unit,

```

```

        modifier: Modifier = Modifier,
    ) {
        Row(
            modifier = modifier
                .fillMaxWidth()
                .background(
                    color = PosterSurface.copy(alpha = 0.82f),
                    shape = RoundedCornerShape(15.dp),
                )
                .border(
                    width = 1.dp,
                    color = PosterStroke.copy(alpha = 0.7f),
                    shape = RoundedCornerShape(15.dp),
                )
                .clickable { onCheckedChange(!checked) }
                .padding(horizontal = 18.dp, vertical = 16.dp),
            verticalAlignment = Alignment.CenterVertically,
        ) {
            Column(modifier = Modifier.weight(1f)) {
                Text(
                    text = title,
                    color = PosterTextPrimary,
                    fontSize = 15.sp,
                    fontWeight = FontWeight.Bold,
                )

                Spacer(modifier = Modifier.height(5.dp))

                Text(
                    text = subtitle,
                    color = PosterTextSecondary,
                    fontSize = 13.sp,
                    lineHeight = 18.sp,
                )
            }

            Switch(
                checked = checked,
                onCheckedChange = onCheckedChange,
            )
        }
    }
}

```

```

@Composable
private fun SettingsInfoCard(
    title: String,
    description: String,
    modifier: Modifier = Modifier,
) {
    Row(
        modifier = modifier
    )
}

```

```

        .fillMaxWidth()
        .background(
            color = PosterInfoSurface,
            shape = RoundedCornerShape(15.dp),
        )
        .border(
            width = 1.dp,
            color = PosterStroke.copy(alpha = 0.75f),
            shape = RoundedCornerShape(15.dp),
        )
        .padding(horizontal = 18.dp, vertical = 18.dp),
    ) {
        Box(
            modifier = Modifier
                .size(34.dp)
                .background(PosterPrimary.copy(alpha = 0.2f), CircleShape),
            contentAlignment = Alignment.Center,
        ) {
            Icon(
                imageVector = Icons.Outlined.Info,
                contentDescription = null,
                tint = PosterAccent,
                modifier = Modifier.size(20.dp),
            )
        }
    }

    Spacer(modifier = Modifier.width(14.dp))

    Column(modifier = Modifier.weight(1f)) {
        Text(
            text = title,
            color = PosterTextPrimary,
            fontSize = 16.sp,
            fontWeight = FontWeight.Bold,
        )

        Spacer(modifier = Modifier.height(8.dp))

        Text(
            text = description,
            color = PosterTextSecondary,
            fontSize = 14.sp,
            lineHeight = 20.sp,
        )
    }
}

@Composable
private fun TokenStepRow(
    index: Int,

```

```

text: String,
modifier: Modifier = Modifier,
) {
    Row(
        modifier = modifier
        .fillMaxWidth()
        .padding(bottom = 12.dp)
        .background(
            color = PosterSurface.copy(alpha = 0.82f),
            shape = RoundedCornerShape(15.dp),
        )
        .border(
            width = 1.dp,
            color = PosterStroke.copy(alpha = 0.7f),
            shape = RoundedCornerShape(15.dp),
        )
        .padding(horizontal = 16.dp, vertical = 16.dp),
        verticalAlignment = Alignment.Top,
    ) {
        Box(
            modifier = Modifier
            .size(28.dp)
            .background(
                brush = Brush.linearGradient(
                    colors = listOf(PosterPrimary, PosterPrimaryDark),
                ),
                shape = CircleShape,
            ),
            contentAlignment = Alignment.Center,
        ) {
            Text(
                text = index.toString(),
                color = Color.White,
                fontSize = 13.sp,
                fontWeight = FontWeight.Bold,
            )
        }
    }

    Spacer(modifier = Modifier.width(12.dp))

    Text(
        text = text,
        color = PosterTextSecondary,
        fontSize = 14.sp,
        lineHeight = 20.sp,
        modifier = Modifier.weight(1f),
    )
}
}

```

@Composable

```

private fun SettingsPrimaryButton(
    text: String,
    enabled: Boolean,
    onClick: () -> Unit,
    modifier: Modifier = Modifier,
) {
    Box(
        modifier = modifier
            .fillMaxWidth()
            .height(56.dp)
            .shadow(
                elevation = if (enabled) 24.dp else 0.dp,
                shape = RoundedCornerShape(13.dp),
                spotColor = PosterPrimary.copy(alpha = 0.35f),
            )
            .background(
                brush = Brush.horizontalGradient(
                    colors = if (enabled) {
                        listOf(PosterPrimary, PosterPrimaryDark)
                    } else {
                        listOf(Color(0xFF312A7A), Color(0xFF261D65))
                    },
                ),
                shape = RoundedCornerShape(13.dp),
            )
            .clickable(enabled = enabled) { onClick() },
        contentAlignment = Alignment.Center,
    ) {
        Text(
            text = text,
            color = if (enabled) Color.White else PosterTextMuted,
            fontSize = 16.sp,
            fontWeight = FontWeight.Bold,
        )
    }
}

private data class SettingsOption(
    val value: String,
    val title: String,
    val subtitle: String,
)

private val tokenSteps = listOf(
    "Open your email provider account settings.",
    "Find Security, App passwords, SMTP/IMAP access, or External apps.",
    "Create a new app password for Poster and copy the generated token.",
    "Return to Poster, paste the token, then press Save Token.",
)

@Preview(showBackground = true, widthDp = 393, heightDp = 852)

```

```
@Composable
private fun SettingsTextEditScreenPreview() {
    SettingsTextEditScreen(
        title = "Name",
        label = "Name",
        value = "Your Name",
        placeholder = "Enter your name",
        onBackClick = {},
        onSaveClick = {},
    )
}
```

Листинг 108 – Код файла `app/src/main/java/com/example/poster/presentation/settings/SettingsScreen.kt`

```
package com.example.poster.presentation.settings

import androidx.compose.foundation.background
import androidx.compose.foundation.border
import androidx.compose.foundation.clickable
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.PaddingValues
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.navigationBarsPadding
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.layout.statusBarsPadding
import androidx.compose.foundation.layout.width
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.shape.CircleShape
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.foundation.text.BasicTextField
import androidx.compose.foundation.text.KeyboardOptions
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.outlined.AccountCircle
import androidx.compose.material.icons.outlined.AlternateEmail
import androidx.compose.material.icons.outlined.ArrowBack
import androidx.compose.material.icons.outlined.Article
import androidx.compose.material.icons.outlined.CalendarMonth
import androidx.compose.material.icons.outlined.CameraAlt
import androidx.compose.material.icons.outlined.Check
import androidx.compose.material.icons.outlined.ChevronRight
import androidx.compose.material.icons.outlined.Close
import androidx.compose.material.icons.outlined.Email
import androidx.compose.material.icons.outlined.Language
import androidx.compose.material.icons.outlined.Person
import androidx.compose.material.icons.outlined.Security
import androidx.compose.material.icons.outlined.VpnKey
import androidx.compose.material3.Icon
import androidx.compose.material3.IconButton
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.Immutable
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.saveable.rememberSaveable
import androidx.compose.runtime.setValue
```

```
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.shadow
import androidx.compose.ui.graphics.Brush
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.SolidColor
import androidx.compose.ui.graphics.vector.ImageVector
import androidx.compose.ui.text.TextStyle
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.input.KeyboardType
import androidx.compose.ui.text.style.TextOverflow
import androidx.compose.ui.tooling.preview.Preview
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
```

```
private val PosterBackground = Color(0xFF030306)
private val PosterTopBar = Color(0xFF101014)
private val PosterSurface = Color(0xFF101014)
private val PosterSection = Color(0xFF090716)
private val PosterDivider = Color(0xFF1B1A25)
private val PosterPrimary = Color(0xFF625BFF)
private val PosterPrimaryDark = Color(0xFF493CCB)
private val PosterIconBg = Color(0xFF1D1A4A)
private val PosterTextPrimary = Color(0xFFFF7F7FF)
private val PosterTextSecondary = Color(0xFFA5A6BA)
private val PosterTextMuted = Color(0xFF7B7D92)
private val PosterAccent = Color(0xFF7F8BFF)
```

```
@Immutable
data class SettingsItemUi(
    val id: String,
    val title: String,
    val subtitle: String,
    val icon: ImageVector,
)
```

```
@Composable
fun SettingsScreen(
    name: String = "Your Name",
    username: String = "@your.username",
    email: String = "your.email@example.com",
    birthday: String = "January 1, 2000",
    bio: String = "Hey there! I'm using Poster.",
    language: String = "English",
    accessTokenStatus: String = "Configured",
    onBackClick: () -> Unit = {},
    onAccessTokenClick: () -> Unit = {},
    onPrivacyClick: () -> Unit = {},
    onLanguageClick: () -> Unit = {},
    onProfilePhotoClick: () -> Unit = {},
    onAccountValueSave: (String, String) -> Unit = { _, _ -> },
)
```

```

) {
    var currentName by rememberSaveable(name) { mutableStateOf(name) }
    var currentUsername by rememberSaveable(username) { mutableStateOf(username) }
    var currentEmail by rememberSaveable(email) { mutableStateOf(email) }
    var currentBirthday by rememberSaveable(birthday) { mutableStateOf(birthday) }
    var currentBio by rememberSaveable(bio) { mutableStateOf(bio) }
    var editingAccountItemId by rememberSaveable { mutableStateOf<String?>(null) }
    var editingAccountValue by rememberSaveable { mutableStateOf("") }

    fun accountValueFor(id: String): String {
        return when (id) {
            "name" -> currentName
            "username" -> currentUsername
            "email" -> currentEmail
            "birthday" -> currentBirthday
            "bio" -> currentBio
            else -> ""
        }
    }

    fun saveAccountValue(id: String, value: String) {
        val normalizedValue = normalizeAccountValue(id, value)
        when (id) {
            "name" -> currentName = normalizedValue
            "username" -> currentUsername = normalizedValue
            "email" -> currentEmail = normalizedValue
            "birthday" -> currentBirthday = normalizedValue
            "bio" -> currentBio = normalizedValue
        }
        editingAccountItemId = null
        editingAccountValue = ""
        onAccountValueSave(id, normalizedValue)
    }

    val accountItems = remember(
        currentName,
        currentUsername,
        currentEmail,
        currentBirthday,
        currentBio,
    ) {
        listOf(
            SettingsItemUi(
                id = "name",
                title = "Name",
                subtitle = currentName,
                icon = Icons.Outlined.AccountCircle,
            ),
            SettingsItemUi(
                id = "username",
                title = "Username",

```

```

        subtitle = currentUsername,
        icon = Icons.Outlined.AlternateEmail,
    ),
    SettingsItemUi(
        id = "email",
        title = "Email",
        subtitle = currentEmail,
        icon = Icons.Outlined.Email,
    ),
    SettingsItemUi(
        id = "birthday",
        title = "Birthday",
        subtitle = currentBirthday,
        icon = Icons.Outlined.CalendarMonth,
    ),
    SettingsItemUi(
        id = "bio",
        title = "Bio",
        subtitle = currentBio,
        icon = Icons.Outlined.Article,
    ),
)
}

```

```

val preferenceItems = listOf(
    SettingsItemUi(
        id = "language",
        title = "Language",
        subtitle = language,
        icon = Icons.Outlined.Language,
    ),
)

```

```

val securityItems = listOf(
    SettingsItemUi(
        id = "access_token",
        title = "Access Token",
        subtitle = accessTokenStatus,
        icon = Icons.Outlined.VpnKey,
    ),
    SettingsItemUi(
        id = "privacy",
        title = "Privacy & Security",
        subtitle = "",
        icon = Icons.Outlined.Security,
    ),
)

```

```

Column(
    modifier = Modifier
        .fillMaxSize()

```

```

        .background(
            Brush.radialGradient(
                colors = listOf(
                    Color(0xFF09051E),
                    PosterBackground,
                    Color(0xFF030306),
                ),
                radius = 1200f,
            )
        ),
    ) {
        SettingsTopBar(
            onBackClick = onBackClick,
        )

        LazyColumn(
            modifier = Modifier
                .weight(1f)
                .fillMaxWidth()
                .navigationBarsPadding(),
            contentPadding = PaddingValues(bottom = 32.dp),
        ) {
            item {
                SettingsProfileHeader(
                    name = currentName,
                    username = currentUsername,
                    onProfilePhotoClick = onProfilePhotoClick,
                )
            }

            item {
                SettingsSection(
                    title = "ACCOUNT",
                    items = accountItems,
                    editingItemId = editingAccountId,
                    editingValue = editingAccountValue,
                    onEditingValueChange = { editingAccountValue = it },
                    onEditClick = { item ->
                        editingAccountId = item.id
                        editingAccountValue = accountValueFor(item.id)
                    },
                    onSaveEdit = { item ->
                        if (canSaveAccountValue(item.id, editingAccountValue)) {
                            saveAccountValue(item.id, editingAccountValue)
                        }
                    },
                    onCancelEdit = {
                        editingAccountId = null
                        editingAccountValue = ""
                    },
                    onItemClick = { item ->

```



```

        tint = PosterTextSecondary,
        modifier = Modifier
            .size(28.dp)
            .clickable { onBackPressed() },
    )

    Spacer(modifier = Modifier.width(12.dp))

    Text(
        text = "Settings",
        color = PosterTextPrimary,
        fontSize = 20.sp,
        fontWeight = FontWeight.Bold,
    )
}
}

@Composable
private fun SettingsProfileHeader(
    name: String,
    username: String,
    onProfilePhotoClick: () -> Unit,
) {
    Column(
        modifier = Modifier
            .fillMaxWidth()
            .background(
                Brush.verticalGradient(
                    colors = listOf(
                        Color(0xFF0D0A26),
                        Color(0xFF030306),
                    ),
                ),
            )
        .padding(top = 28.dp, bottom = 34.dp),
        horizontalAlignment = Alignment.CenterHorizontally,
    ) {
        Box {
            Box(
                modifier = Modifier
                    .size(96.dp)
                    .background(
                        brush = Brush.linearGradient(
                            colors = listOf(
                                PosterPrimary,
                                PosterPrimaryDark,
                            ),
                        ),
                    ),
                shape = CircleShape,
            ),
            contentAlignment = Alignment.Center,

```

```

    ) {
        Icon(
            imageVector = Icons.Outlined.Person,
            contentDescription = null,
            tint = Color.White,
            modifier = Modifier.size(48.dp),
        )
    }

    Box(
        modifier = Modifier
            .size(32.dp)
            .align(Alignment.BottomEnd)
            .shadow(
                elevation = 12.dp,
                shape = CircleShape,
                spotColor = PosterPrimary.copy(alpha = 0.45f),
            )
            .background(
                brush = Brush.linearGradient(
                    colors = listOf(
                        PosterPrimary,
                        PosterPrimaryDark,
                    ),
                ),
                shape = CircleShape,
            )
            .border(
                width = 3.dp,
                color = PosterBackground,
                shape = CircleShape,
            )
            .clickable { onProfilePhotoClick() },
        contentAlignment = Alignment.Center,
    ) {
        Icon(
            imageVector = Icons.Outlined.CameraAlt,
            contentDescription = "Change photo",
            tint = Color.White,
            modifier = Modifier.size(17.dp),
        )
    }
}

```

```

Spacer(modifier = Modifier.height(16.dp))

```

```

Text(
    text = name,
    color = PosterTextPrimary,
    fontSize = 18.sp,
    fontWeight = FontWeight.Bold,

```

```

    )

    Spacer(modifier = Modifier.height(6.dp))

    Text(
        text = username,
        color = PosterAccent,
        fontSize = 14.sp,
        fontWeight = FontWeight.Medium,
    )
}
}

@Composable
private fun SettingsSection(
    title: String,
    items: List<SettingsItemUi>,
    onItemClick: (SettingsItemUi) -> Unit,
    editingItemId: String? = null,
    editingValue: String = "",
    onEditingValueChange: (String) -> Unit = {},
    onEditClick: (SettingsItemUi) -> Unit = onItemClick,
    onSaveEdit: (SettingsItemUi) -> Unit = {},
    onCancelEdit: () -> Unit = {},
) {
    Column(
        modifier = Modifier
            .fillMaxWidth()
            .padding(bottom = 26.dp),
    ) {
        Box(
            modifier = Modifier
                .fillMaxWidth()
                .background(PosterSection.copy(alpha = 0.75f))
                .padding(horizontal = 30.dp, vertical = 12.dp),
        ) {
            Text(
                text = title,
                color = PosterAccent,
                fontSize = 13.sp,
                fontWeight = FontWeight.Medium,
                letterSpacing = 1.sp,
            )
        }
    }

    Column(
        modifier = Modifier
            .fillMaxWidth()
            .background(PosterSurface.copy(alpha = 0.78f))
            .border(width = 0.5.dp, color = PosterDivider),
    ) {

```

```

items.forEachIndexed { index, item ->
    val isEditing = item.id == editingItemId
    SettingsRow(
        item = item,
        showDivider = index < items.lastIndex,
        isEditing = isEditing,
        editingValue = if (isEditing) editingValue else "",
        onEditingValueChange = onEditingValueChange,
        onSaveEdit = { onSaveEdit(item) },
        onCancelEdit = onCancelEdit,
        onClick = {
            if (isEditing) {
                onCancelEdit()
            } else {
                onEditClick(item)
            }
        },
    )
}
}
}
}
}

```

```

@Composable
private fun SettingsRow(
    item: SettingsItemUi,
    showDivider: Boolean,
    onClick: () -> Unit,
    isEditing: Boolean = false,
    editingValue: String = "",
    onEditingValueChange: (String) -> Unit = {},
    onSaveEdit: () -> Unit = {},
    onCancelEdit: () -> Unit = {},
) {
    Column(
        modifier = Modifier.fillMaxWidth(),
    ) {
        Row(
            modifier = Modifier
                .fillMaxWidth()
                .height(65.dp)
                .clickable { onClick() }
                .padding(start = 30.dp, end = 22.dp),
            verticalAlignment = Alignment.CenterVertically,
        ) {
            Box(
                modifier = Modifier
                    .size(36.dp)
                    .background(
                        color = PosterIconBg,
                        shape = CircleShape,

```

```

        ),
        contentAlignment = Alignment.Center,
    ) {
        Icon(
            imageVector = item.icon,
            contentDescription = null,
            tint = PosterAccent,
            modifier = Modifier.size(19.dp),
        )
    }

    Spacer(modifier = Modifier.width(16.dp))

    Column(
        modifier = Modifier.weight(1f),
    ) {
        Text(
            text = item.title,
            color = PosterTextPrimary,
            fontSize = 14.sp,
            fontWeight = FontWeight.Bold,
            maxLines = 1,
            overflow = TextOverflow.Ellipsis,
        )

        if (item.subtitle.isNotEmpty()) {
            Spacer(modifier = Modifier.height(4.dp))

            Text(
                text = item.subtitle,
                color = PosterTextSecondary,
                fontSize = 13.sp,
                fontWeight = FontWeight.SemiBold,
                maxLines = 1,
                overflow = TextOverflow.Ellipsis,
            )
        }
    }

    Spacer(modifier = Modifier.width(12.dp))

    if (isEditing) {
        IconButton(
            onClick = onCancelEdit,
            modifier = Modifier.size(34.dp),
        ) {
            Icon(
                imageVector = Icons.Outlined.Close,
                contentDescription = "Cancel",
                tint = PosterTextMuted,
                modifier = Modifier.size(19.dp),
            )
        }
    }

```

```

        )
    }
} else {
    Icon(
        imageVector = Icons.Outlined.ChevronRight,
        contentDescription = null,
        tint = PosterTextMuted,
        modifier = Modifier.size(22.dp),
    )
}
}

if (isEditing) {
    InlineSettingsEditor(
        itemId = item.id,
        value = editingValue,
        onValueChange = onEditingValueChange,
        onSaveClick = onSaveEdit,
        canSave = canSaveAccountValue(item.id, editingValue),
        modifier = Modifier.padding(
            start = 82.dp,
            end = 22.dp,
            bottom = 14.dp,
        ),
    )
}

if (showDivider) {
    Box(
        modifier = Modifier
            .fillMaxWidth()
            .padding(start = 82.dp)
            .height(0.7.dp)
            .background(PosterDivider),
    )
}
}
}

```

```

@Composable
private fun InlineSettingsEditor(
    itemId: String,
    value: String,
    onValueChange: (String) -> Unit,
    onSaveClick: () -> Unit,
    canSave: Boolean,
    modifier: Modifier = Modifier,
) {
    val singleLine = itemId != "bio"
    Row(
        modifier = modifier.fillMaxWidth(),
    )
}

```

```

        verticalAlignment = Alignment.CenterVertically,
    ) {
        Box(
            modifier = Modifier
                .weight(1f)
                .height(if (singleLine) 46.dp else 92.dp)
                .background(
                    color = PosterSection.copy(alpha = 0.86f),
                    shape = RoundedCornerShape(12.dp),
                )
                .border(
                    width = 1.dp,
                    color = PosterPrimary.copy(alpha = 0.45f),
                    shape = RoundedCornerShape(12.dp),
                )
                .padding(horizontal = 14.dp, vertical = 11.dp),
            contentAlignment = Alignment.TopStart,
        ) {
            BasicTextField(
                value = value,
                onValueChange = onValueChange,
                singleLine = singleLine,
                cursorBrush = SolidColor(PosterPrimary),
                keyboardOptions = KeyboardOptions(
                    keyboardType = accountKeyboardType(itemId),
                ),
                textStyle = TextStyle(
                    color = PosterTextPrimary,
                    fontSize = 14.sp,
                    lineHeight = 20.sp,
                    fontWeight = FontWeight.SemiBold,
                ),
                decorationBox = { innerTextField ->
                    Box(
                        modifier = Modifier.fillMaxWidth(),
                        contentAlignment = Alignment.TopStart,
                    ) {
                        if (value.isEmpty()) {
                            Text(
                                text = accountPlaceholder(itemId),
                                color = PosterTextMuted,
                                fontSize = 14.sp,
                            )
                        }

                        innerTextField()
                    }
                },
                modifier = Modifier.fillMaxWidth(),
            )
        }
    }

```

```

Spacer(modifier = Modifier.width(10.dp))

IconButton(
    onClick = onSaveClick,
    enabled = canSave,
    modifier = Modifier
        .size(42.dp)
        .background(
            color = if (canSave) PosterPrimary.copy(alpha = 0.2f) else PosterDivider,
            shape = CircleShape,
        ),
) {
    Icon(
        imageVector = Icons.Outlined.Check,
        contentDescription = "Save",
        tint = if (canSave) PosterAccent else PosterTextMuted,
        modifier = Modifier.size(21.dp),
    )
}
}
}

private fun accountKeyboardType(itemId: String): KeyboardType {
    return when (itemId) {
        "email" -> KeyboardType.Email
        else -> KeyboardType.Text
    }
}

private fun accountPlaceholder(itemId: String): String {
    return when (itemId) {
        "name" -> "Enter name"
        "username" -> "@username"
        "email" -> "email@example.com"
        "birthday" -> "Birthday"
        "bio" -> "Write a short bio"
        else -> ""
    }
}

private fun canSaveAccountValue(itemId: String, value: String): Boolean {
    return itemId == "bio" || value.trim().isEmpty()
}

private fun normalizeAccountValue(itemId: String, value: String): String {
    val trimmedValue = value.trim()
    return when {
        itemId == "bio" -> trimmedValue
        itemId == "username" && trimmedValue.isNotEmpty() && !trimmedValue.startsWith("@") -
        > "@$trimmedValue"
    }
}

```

```
        else -> trimmedValue
    }
}
```

```
@Preview(showBackground = true, widthDp = 393, heightDp = 852)
@Composable
private fun SettingsScreenPreview() {
    SettingsScreen(
        onBackClick = {},
        onAccessTokenClick = {},
    )
}
```

Листинг 109 – Код файла `app/src/main/java/com/example/poster/ui/theme/Color.kt`

```
package com.example.poster.ui.theme
```

```
import androidx.compose.ui.graphics.Color
```

```
val Purple80 = Color(0xFFD0BCFF)
```

```
val PurpleGrey80 = Color(0xFFCCC2DC)
```

```
val Pink80 = Color(0xFFE8B888)
```

```
val Purple40 = Color(0xFF6650a4)
```

```
val PurpleGrey40 = Color(0xFF625b71)
```

```
val Pink40 = Color(0xFF7D5260)
```

Листинг 110 – Код файла app/src/main/java/com/example/poster/ui/theme/Theme.kt

```
package com.example.poster.ui.theme

import android.app.Activity
import android.os.Build
import androidx.compose.foundation.isSystemInDarkTheme
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.darkColorScheme
import androidx.compose.material3.dynamicDarkColorScheme
import androidx.compose.material3.dynamicLightColorScheme
import androidx.compose.material3.lightColorScheme
import androidx.compose.runtime.Composable
import androidx.compose.ui.platform.LocalContext

private val DarkColorScheme = darkColorScheme(
    primary = Purple80,
    secondary = PurpleGrey80,
    tertiary = Pink80
)

private val LightColorScheme = lightColorScheme(
    primary = Purple40,
    secondary = PurpleGrey40,
    tertiary = Pink40

    /* Other default colors to override
    background = Color(0xFFFFFBFE),
    surface = Color(0xFFFFFBFE),
    onPrimary = Color.White,
    onSecondary = Color.White,
    onTertiary = Color.White,
    onBackground = Color(0xFF1C1B1F),
    onSurface = Color(0xFF1C1B1F),
    */
)

@Composable
fun PosterTheme(
    darkTheme: Boolean = isSystemInDarkTheme(),
    // Dynamic color is available on Android 12+
    dynamicColor: Boolean = true,
    content: @Composable () -> Unit
) {
    val colorScheme = when {
        dynamicColor && Build.VERSION.SDK_INT >= Build.VERSION_CODES.S -> {
            val context = LocalContext.current
            if (darkTheme) dynamicDarkColorScheme(context) else dynamicLightColorScheme(context)
        }
        else -> {
            if (darkTheme) DarkColorScheme else LightColorScheme
        }
    }
    MaterialTheme(
        colorScheme = colorScheme,
        typography = Typography,
        content = content
    )
}
```

```
        darkTheme -> DarkColorScheme
        else -> LightColorScheme
    }

    MaterialTheme(
        colorScheme = colorScheme,
        typography = Typography,
        content = content
    )
}
```

Листинг 111 – Код файла `app/src/main/java/com/example/poster/ui/theme/Type.kt`

```
package com.example.poster.ui.theme

import androidx.compose.material3.Typography
import androidx.compose.ui.text.TextStyle
import androidx.compose.ui.text.font.FontFamily
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.sp

// Set of Material typography styles to start with
val Typography = Typography(
    bodyLarge = TextStyle(
        fontFamily = FontFamily.Default,
        fontWeight = FontWeight.Normal,
        fontSize = 16.sp,
        lineHeight = 24.sp,
        letterSpacing = 0.5.sp
    )
    /* Other default text styles to override
    titleLarge = TextStyle(
        fontFamily = FontFamily.Default,
        fontWeight = FontWeight.Normal,
        fontSize = 22.sp,
        lineHeight = 28.sp,
        letterSpacing = 0.sp
    ),
    labelSmall = TextStyle(
        fontFamily = FontFamily.Default,
        fontWeight = FontWeight.Medium,
        fontSize = 11.sp,
        lineHeight = 16.sp,
        letterSpacing = 0.5.sp
    )
    */
)
```

Листинг 112 – Код файла app/src/main/res/drawable/ic_launcher_background.xml

```
<?xml version="1.0" encoding="utf-8"?>
<vector xmlns:android="http://schemas.android.com/apk/res/android"
    android:width="108dp"
    android:height="108dp"
    android:viewportWidth="108"
    android:viewportHeight="108">
    <path
        android:fillColor="#3DDC84"
        android:pathData="M0,0h108v108h-108z" />
    <path
        android:fillColor="#00000000"
        android:pathData="M9,0L9,108"
        android:strokeWidth="0.8"
        android:strokeColor="#33FFFFFF" />
    <path
        android:fillColor="#00000000"
        android:pathData="M19,0L19,108"
        android:strokeWidth="0.8"
        android:strokeColor="#33FFFFFF" />
    <path
        android:fillColor="#00000000"
        android:pathData="M29,0L29,108"
        android:strokeWidth="0.8"
        android:strokeColor="#33FFFFFF" />
    <path
        android:fillColor="#00000000"
        android:pathData="M39,0L39,108"
        android:strokeWidth="0.8"
        android:strokeColor="#33FFFFFF" />
    <path
        android:fillColor="#00000000"
        android:pathData="M49,0L49,108"
        android:strokeWidth="0.8"
        android:strokeColor="#33FFFFFF" />
    <path
        android:fillColor="#00000000"
        android:pathData="M59,0L59,108"
        android:strokeWidth="0.8"
        android:strokeColor="#33FFFFFF" />
    <path
        android:fillColor="#00000000"
        android:pathData="M69,0L69,108"
        android:strokeWidth="0.8"
        android:strokeColor="#33FFFFFF" />
    <path
        android:fillColor="#00000000"
        android:pathData="M79,0L79,108"
        android:strokeWidth="0.8"
        android:strokeColor="#33FFFFFF" />
```

```

<path
  android:fillColor="#00000000"
  android:pathData="M89,0L89,108"
  android:strokeWidth="0.8"
  android:strokeColor="#33FFFFFF" />
<path
  android:fillColor="#00000000"
  android:pathData="M99,0L99,108"
  android:strokeWidth="0.8"
  android:strokeColor="#33FFFFFF" />
<path
  android:fillColor="#00000000"
  android:pathData="M0,9L108,9"
  android:strokeWidth="0.8"
  android:strokeColor="#33FFFFFF" />
<path
  android:fillColor="#00000000"
  android:pathData="M0,19L108,19"
  android:strokeWidth="0.8"
  android:strokeColor="#33FFFFFF" />
<path
  android:fillColor="#00000000"
  android:pathData="M0,29L108,29"
  android:strokeWidth="0.8"
  android:strokeColor="#33FFFFFF" />
<path
  android:fillColor="#00000000"
  android:pathData="M0,39L108,39"
  android:strokeWidth="0.8"
  android:strokeColor="#33FFFFFF" />
<path
  android:fillColor="#00000000"
  android:pathData="M0,49L108,49"
  android:strokeWidth="0.8"
  android:strokeColor="#33FFFFFF" />
<path
  android:fillColor="#00000000"
  android:pathData="M0,59L108,59"
  android:strokeWidth="0.8"
  android:strokeColor="#33FFFFFF" />
<path
  android:fillColor="#00000000"
  android:pathData="M0,69L108,69"
  android:strokeWidth="0.8"
  android:strokeColor="#33FFFFFF" />
<path
  android:fillColor="#00000000"
  android:pathData="M0,79L108,79"
  android:strokeWidth="0.8"
  android:strokeColor="#33FFFFFF" />
<path

```

```

        android:fillColor="#00000000"
        android:pathData="M0,89L108,89"
        android:strokeWidth="0.8"
        android:strokeColor="#33FFFFFF" />
<path
    android:fillColor="#00000000"
    android:pathData="M0,99L108,99"
    android:strokeWidth="0.8"
    android:strokeColor="#33FFFFFF" />
<path
    android:fillColor="#00000000"
    android:pathData="M19,29L89,29"
    android:strokeWidth="0.8"
    android:strokeColor="#33FFFFFF" />
<path
    android:fillColor="#00000000"
    android:pathData="M19,39L89,39"
    android:strokeWidth="0.8"
    android:strokeColor="#33FFFFFF" />
<path
    android:fillColor="#00000000"
    android:pathData="M19,49L89,49"
    android:strokeWidth="0.8"
    android:strokeColor="#33FFFFFF" />
<path
    android:fillColor="#00000000"
    android:pathData="M19,59L89,59"
    android:strokeWidth="0.8"
    android:strokeColor="#33FFFFFF" />
<path
    android:fillColor="#00000000"
    android:pathData="M19,69L89,69"
    android:strokeWidth="0.8"
    android:strokeColor="#33FFFFFF" />
<path
    android:fillColor="#00000000"
    android:pathData="M19,79L89,79"
    android:strokeWidth="0.8"
    android:strokeColor="#33FFFFFF" />
<path
    android:fillColor="#00000000"
    android:pathData="M29,19L29,89"
    android:strokeWidth="0.8"
    android:strokeColor="#33FFFFFF" />
<path
    android:fillColor="#00000000"
    android:pathData="M39,19L39,89"
    android:strokeWidth="0.8"
    android:strokeColor="#33FFFFFF" />
<path
    android:fillColor="#00000000"

```

```
        android:pathData="M49,19L49,89"
        android:strokeWidth="0.8"
        android:strokeColor="#33FFFFFF" />
    <path
        android:fillColor="#00000000"
        android:pathData="M59,19L59,89"
        android:strokeWidth="0.8"
        android:strokeColor="#33FFFFFF" />
    <path
        android:fillColor="#00000000"
        android:pathData="M69,19L69,89"
        android:strokeWidth="0.8"
        android:strokeColor="#33FFFFFF" />
    <path
        android:fillColor="#00000000"
        android:pathData="M79,19L79,89"
        android:strokeWidth="0.8"
        android:strokeColor="#33FFFFFF" />
</vector>
```

Листинг 113 – Код файла app/src/main/res/drawable/ic_launcher_foreground.xml

```
<vector xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:aapt="http://schemas.android.com/aapt"
    android:width="108dp"
    android:height="108dp"
    android:viewportWidth="108"
    android:viewportHeight="108">
    <path android:pathData="M31,63.928c0,0 6.4,-11 12.1,-13.1c7.2,-2.6 26,-1.4 26,-1.4l38.1,38.1L
    107,108.928l-32,-1L31,63.928z">
        <aapt:attr name="android:fillColor">
            <gradient
                android:endX="85.84757"
                android:endY="92.4963"
                android:startX="42.9492"
                android:startY="49.59793"
                android:type="linear">
                <item
                    android:color="#44000000"
                    android:offset="0.0" />
                <item
                    android:color="#00000000"
                    android:offset="1.0" />
            </gradient>
        </aapt:attr>
    </path>
    <path
        android:fillColor="#FFFFFF"
        android:fillType="nonZero"
        android:pathData="M65.3,45.828l3.8,-6.6c0.2,-0.4 0.1,-0.9 -0.3,-1.1c-0.4,-0.2 -0.9,-0.1 -1.1,0.
        3l-3.9,6.7c-6.3,-2.8 -13.4,-2.8 -19.7,0l-3.9,-6.7c-0.2,-0.4 -0.7,-0.5 -1.1,-0.3C38.8,38.328 38.7,38.
        828 38.9,39.228l3.8,6.6C36.2,49.428 31.7,56.028 31,63.928h46C76.3,56.028 71.8,49.428 65.3,4
        5.828zM43.4,57.328c-0.8,0 -1.5,-0.5 -1.8,-1.2c-0.3,-0.7 -0.1,-1.5 0.4,-2.1c0.5,-0.5 1.4,-0.7 2.1,-0.
        4c0.7,0.3 1.2,1 1.2,1.8C45.3,56.528 44.5,57.328 43.4,57.328L43.4,57.328zM64.6,57.328c-0.8,0
        -1.5,-0.5 -1.8,-1.2s-0.1,-1.5 0.4,-2.1c0.5,-0.5 1.4,-0.7 2.1,-0.4c0.7,0.3 1.2,1 1.2,1.8C66.5,56.528 6
        5.6,57.328 64.6,57.328L64.6,57.328z"
        android:strokeWidth="1"
        android:strokeColor="#00000000" />
</vector>
```

Листинг 114 – Код файла app/src/main/res/mipmap-anydpi/ic_launcher.xml

```
<?xml version="1.0" encoding="utf-8"?>
<adaptive-icon xmlns:android="http://schemas.android.com/apk/res/android">
  <background android:drawable="@drawable/ic_launcher_background" />
  <foreground android:drawable="@drawable/ic_launcher_foreground" />
  <monochrome android:drawable="@drawable/ic_launcher_foreground" />
</adaptive-icon>
```

Листинг 115 – Код файла `app/src/main/res/mipmap-anydpi/ic_launcher_round.xml`

```
<?xml version="1.0" encoding="utf-8"?>
<adaptive-icon xmlns:android="http://schemas.android.com/apk/res/android">
  <background android:drawable="@drawable/ic_launcher_background" />
  <foreground android:drawable="@drawable/ic_launcher_foreground" />
  <monochrome android:drawable="@drawable/ic_launcher_foreground" />
</adaptive-icon>
```

Листинг 116 – Код файла app/src/main/res/values/colors.xml

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <color name="purple_200">#FFBB86FC</color>
    <color name="purple_500">#FF6200EE</color>
    <color name="purple_700">#FF3700B3</color>
    <color name="teal_200">#FF03DAC5</color>
    <color name="teal_700">#FF018786</color>
    <color name="black">#FF000000</color>
    <color name="white">#FFFFFFFF</color>
</resources>
```

Листинг 117 – Код файла app/src/main/res/values/strings.xml

```
<resources>  
    <string name="app_name">Poster</string>  
</resources>
```

Листинг 118 – Код файла app/src/main/res/values/themes.xml

```
<?xml version="1.0" encoding="utf-8"?>
<resources>

    <style name="Theme.Poster" parent="android:Theme.Material.NoActionBar">
        <item name="android:windowBackground">#030306</item>
        <item name="android:colorAccent">#625BFF</item>
        <item name="android:navigationBarColor">#030306</item>
        <item name="android:statusBarColor">#030306</item>
        <item name="android:windowLightStatusBar">false</item>
        <item name="android:windowLightNavigationBar">false</item>
    </style>
</resources>
```

Листинг 119 – Код файла app/src/main/res/xml/backup_rules.xml

```
<?xml version="1.0" encoding="utf-8"?><!--  
    Sample backup rules file; uncomment and customize as necessary.  
    See https://developer.android.com/guide/topics/data/autobackup  
    for details.  
    Note: This file is ignored for devices older than API 31  
    See https://developer.android.com/about/versions/12/backup-restore  
-->  
<full-backup-content>  
    <!--  
    <include domain="sharedpref" path="."/>  
    <exclude domain="sharedpref" path="device.xml"/>  
    -->  
</full-backup-content>
```

Листинг 120 – Код файла app/src/main/res/xml/data_extraction_rules.xml

```
<?xml version="1.0" encoding="utf-8"?><!--  
  Sample data extraction rules file; uncomment and customize as necessary.  
  See https://developer.android.com/about/versions/12/backup-restore#xml-changes  
  for details.  
-->  
<data-extraction-rules>  
  <cloud-backup>  
    <!-- TODO: Use <include> and <exclude> to control what is backed up.  
    <include .../>  
    <exclude .../>  
    -->  
  </cloud-backup>  
  <!--  
  <device-transfer>  
    <include .../>  
    <exclude .../>  
  </device-transfer>  
  -->  
</data-extraction-rules>
```

Листинг 121 – Код файла app/src/test/java/com/example/poster/ExampleUnitTest.kt

```
package com.example.poster

import org.junit.Test

import org.junit.Assert.*

/**
 * Example local unit test, which will execute on the development machine (host).
 *
 * See [testing documentation](http://d.android.com/tools/testing).
 */
class ExampleUnitTest {
    @Test
    fun addition_isCorrect() {
        assertEquals(4, 2 + 2)
    }
}
```